

Title

Δίκτυα Επικοινωνιών I

Κεφάλαιο 1: Εισαγωγή

1. Στόχος κεφαλαίου

Στόχος είναι να δούμε τη «μεγάλη εικόνα» των δικτύων:

- τι είναι το Διαδίκτυο
- τι είναι πρωτόκολλο
- τι υπάρχει στην περιφέρεια του δικτύου
- τι υπάρχει στον πυρήνα του δικτύου
- τι είναι packet switching και circuit switching
- τι είναι καθυστέρηση, απώλειες, throughput
- γιατί χρησιμοποιούμε layers
- τι είναι encapsulation
- βασικές έννοιες ασφάλειας
- σύντομη ιστορία Διαδικτύου

2. Τι είναι το Διαδίκτυο

Το Διαδίκτυο είναι:

Auto ▾

δίκτυο δικτύων

Αποτελείται από πολλά διασυνδεδεμένα δίκτυα που ανήκουν σε διαφορετικούς οργανισμούς.

Κύρια συστατικά

Συστατικό	Τι είναι
Hosts / τερματικά συστήματα	υπολογιστές, κινητά, servers, IoT συσκευές

Packet switches	routers και switches
Communication links	οπτική ίνα, χαλκός, ραδιοζεύξεις, δορυφόροι
Networks	σύνολα συσκευών, routers και ζεύξεων
ISPs	πάροχοι Διαδικτύου

3. Hosts

Οι hosts είναι οι τελικές συσκευές.

Παραδείγματα:

- laptop
- κινητό
- server
- smart TV
- κάμερα ασφαλείας
- smartwatch
- gaming device
- αυτοκίνητο
- αισθητήρας
- Amazon Echo
- pacemaker monitor
- IoT συσκευές

Οι hosts τρέχουν δικτυακές εφαρμογές.

4. Packet switches

Οι packet switches προωθούν πακέτα.

Υπάρχουν δύο βασικοί τύποι:

Auto ▾



routers
switches

Οι routers δουλεύουν κυρίως στο επίπεδο δικτύου.

Οι switches δουλεύουν κυρίως στο επίπεδο ζεύξης.

5. Communication links

Οι ζεύξεις συνδέουν συσκευές.

Μπορεί να είναι:

- οπτική ίνα
- χαλκός
- ραδιοζεύξη
- δορυφορική ζεύξη

Ο ρυθμός μετάδοσης λέγεται:

Auto ▾



bandwidth / εύρος ζώνης

και μετριέται σε:

Auto ▾



bits per second

6. Το Διαδίκτυο από την οπτική εφαρμογών

Το Διαδίκτυο παρέχει υποδομή για εφαρμογές όπως:

- Web
- email
- streaming video
- games
- social media
- e-commerce
- τηλεδιασκέψεις
- IoT
- cloud services

Παρέχει επίσης API σε κατανεμημένες εφαρμογές.

Δηλαδή επιτρέπει σε εφαρμογές αποστολέα και δέκτη να χρησιμοποιούν υπηρεσίες μεταφοράς.

7. Πρωτόκολλο

Πρωτόκολλο είναι σύνολο κανόνων που καθορίζει:

- μορφή μηνυμάτων
- σειρά μηνυμάτων
- ενέργειες όταν στέλνονται μηνύματα
- ενέργειες όταν λαμβάνονται μηνύματα
- ενέργειες όταν συμβαίνουν γεγονότα

Ορισμός:

Auto ▾



Τα πρωτόκολλα καθορίζουν τη μορφή και τη σειρά των μηνυμάτων που στέλνονται και λαμβάνονται, καθώς και τις ενέργειες κατά την αποστολή/λήψη.

Παραδείγματα:

- HTTP
- TCP
- IP
- WiFi
- Ethernet
- 4G/5G
- DNS
- SMTP

8. Ανθρώπινο παράδειγμα πρωτοκόλλου

Άνθρωπος A:

Auto ▾



Γεια

Άνθρωπος B:

Auto ▾



Γεια

Άνθρωπος A:

Auto ▾



Τι ώρα είναι;

Ανθρωπος Β:

Auto ▾



2:00

Αυτό έχει σειρά και κανόνες, άρα μοιάζει με πρωτόκολλο.

9. Δικτυακό παράδειγμα πρωτοκόλλου

Auto (SQL) ▾



TCP connection request
TCP connection response
HTTP **GET** request
HTTP response με αρχείο

10. Περιφέρεια δικτύου

Η περιφέρεια περιλαμβάνει:

- hosts
- clients
- servers

Οι servers συχνά βρίσκονται σε data centers.

Παράδειγμα:

Auto ▾



browser = client
Google web server = server

11. Δίκτυα πρόσβασης

Το δίκτυο πρόσβασης συνδέει ένα τερματικό σύστημα με τον πρώτο router.

Παραδείγματα:

- οικιακό δίκτυο
- εταιρικό/πανεπιστημιακό δίκτυο
- WiFi
- 4G/5G
- καλωδιακό δίκτυο
- DSL
- οπτική ίνα

12. Τι κοιτάμε στα δίκτυα πρόσβασης

Δύο βασικά ερωτήματα:

3380. Ποιος είναι ο ρυθμός μετάδοσης;

3381. Είναι το μέσο διαμοιραζόμενο ή αποκλειστικό;

13. Cable access / καλωδιακό δίκτυο

Χρησιμοποιεί HFC:

Auto ▾



Hybrid Fiber Coax

Δηλαδή συνδυασμό:

- οπτικής ίνας
- ομοαξονικού καλωδίου

Χαρακτηριστικά:

- διαμοιραζόμενο μέσο
- ασύμμετρο
- downstream μεγαλύτερο από upstream
- δεδομένα και TV σε διαφορετικές συχνότητες

Χρησιμοποιεί FDM:

Auto ▾



Frequency Division Multiplexing

Δηλαδή διαφορετικά κανάλια σε διαφορετικές ζώνες συχνотήτων.

14. DSL

DSL = Digital Subscriber Line.

Χρησιμοποιεί την υπάρχουσα τηλεφωνική γραμμή.

Χαρακτηριστικά:

- δεδομένα και φωνή σε διαφορετικές συχνότητες
 - δεδομένα προς DSLAM
 - φωνή προς τηλεφωνικό δίκτυο
 - μη διαμοιραζόμενο μέσο
 - αποκλειστική γραμμή προς το κέντρο
-

15. Optical Fiber

Η οπτική ίνα:

- έχει πολύ υψηλές ταχύτητες
- χαμηλό ρυθμό σφαλμάτων
- μεγάλη χωρητικότητα
- απαιτεί μεγάλη επένδυση υποδομής

Ταχύτητες:

Auto ▾



300 Mbps έως 1000+ Mbps

16. Οικιακό δίκτυο

Συνήθως περιλαμβάνει:

- modem
- router
- firewall
- NAT
- WiFi access point
- Ethernet

Συχνά όλα είναι σε μία συσκευή.

17. Ασύρματα δίκτυα πρόσβασης

WiFi

- μικρή εμβέλεια
- περίπου μέσα/γύρω από κτίρια
- 802.11 b/g/n
- ταχύτητες 11, 54, 450 Mbps

Κυψελωτά δίκτυα

- 4G/5G
- μεγάλη περιοχή κάλυψης
- δεκάδες Mbps ή και περισσότερο
- 5G: υψηλότερες ταχύτητες και μικρότερη καθυστέρηση

18. Δίκτυα επιχειρήσεων

Χρησιμοποιούν συνδυασμό:

- Ethernet
- WiFi
- switches
- routers
- servers

Παραδείγματα ταχυτήτων Ethernet:

Auto ▾



100 Mbps, 1 Gbps, 10 Gbps

19. Data center networks

Συνδέουν εκατοντάδες ή χιλιάδες servers.

Χρησιμοποιούν υψηλές ταχύτητες:

Auto ▾



10s έως 100s Gbps

Στόχος:

- υψηλή χωρητικότητα
 - μικρή καθυστέρηση
 - σύνδεση servers μεταξύ τους και με Internet
-

20. Τερματικό που στέλνει πακέτα

Η εφαρμογή δημιουργεί μήνυμα.

Ο host:

5279. παίρνει το μήνυμα

5280. το σπάει σε πακέτα μήκους L bits

5281. μεταδίδει κάθε πακέτο σε ζεύξη ρυθμού R

Χρόνος μετάδοσης:

Auto ▾



$$d_{trans} = L / R$$

όπου:

- L = μήκος πακέτου σε bits
 - R = ρυθμός μετάδοσης σε bits/sec
-

21. Φυσικά μέσα

Το bit διαδίδεται από πομπό σε δέκτη μέσω φυσικού μέσου.

Υπάρχουν δύο κατηγορίες:

Guided media

Το σήμα ταξιδεύει μέσα σε φυσικό οδηγό.

Παραδείγματα:

- χαλκός
- οπτική ίνα
- ομοαξονικό καλώδιο

Unguided media

Το σήμα ταξιδεύει ελεύθερα στον χώρο.

Παραδείγματα:

- ραδιοκύματα

- WiFi
 - κυψελωτά δίκτυα
 - δορυφορικές ζεύξεις
-

22. Συνεστραμμένο ζεύγος

Twisted pair:

- δύο μονωμένα σύρματα χαλκού
 - Category 5: 100 Mbps ή 1 Gbps Ethernet
 - Category 6: 10 Gbps Ethernet
-

23. Ομοαξονικό καλώδιο

Χαρακτηριστικά:

- δύο ομόκεντροι χάλκινοι αγωγοί
 - διπλής κατεύθυνσης
 - broadband
 - πολλά κανάλια συχνοτήτων
 - εκατοντάδες Mbps ανά κανάλι
-

24. Οπτική ίνα

Χαρακτηριστικά:

- γυάλινη ίνα
 - μεταφέρει παλμούς φωτός
 - κάθε παλμός = bit
 - πολύ υψηλή ταχύτητα
 - 10s έως 100s Gbps
 - χαμηλά σφάλματα
 - ανθεκτική σε ηλεκτρομαγνητικό θόρυβο
-

25. Ασύρματα ραδιοκανάλια

Χαρακτηριστικά:

- δεν έχουν καλώδιο
- χρησιμοποιούν ηλεκτρομαγνητικό φάσμα
- συνήθως broadcast
- συχνά half-duplex

Προβλήματα:

- ανάκλαση
- εμπόδια
- παρεμβολές
- θόρυβος

Τύποι:

- WiFi
- 4G/5G
- Bluetooth
- επίγεια μικροκύματα
- δορυφορικές ζεύξεις

26. Δορυφορικές ζεύξεις

Χαρακτηριστικά:

- κανάλια μέχρι περίπου 45 Mbps
- μεγάλη καθυστέρηση
- περίπου 270 ms end-to-end delay

27. Πυρήνας δικτύου

Ο πυρήνας είναι πλέγμα διασυνδεδεμένων routers.

Κάνει packet switching.

Τα μηνύματα σπάνε σε πακέτα και τα πακέτα προωθούνται από router σε router.

28. Δύο βασικές λειτουργίες πυρήνα

Forwarding / προώθηση

Τοπική λειτουργία router.

Auto ▾



πακέτο από είσοδο → σωστή έξοδο

Χρησιμοποιεί forwarding table.

Routing / δρομολόγηση

Καθορίζει ολόκληρο μονοπάτι από πηγή σε προορισμό.

Χρησιμοποιεί routing algorithms.

29. Packet switching

Στη μεταγωγή πακέτου:

- τα μηνύματα σπάνε σε πακέτα
 - κάθε πακέτο μεταδίδεται ανεξάρτητα
 - οι routers κάνουν store-and-forward
 - τα πακέτα μοιράζονται τους πόρους του δικτύου
-

30. Store-and-forward

Ο router πρέπει να λάβει ολόκληρο το πακέτο πριν το προωθήσει στην επόμενη ζεύξη.

Αν πακέτο έχει L bits και ζεύξη R bps:

Auto ▾



$$\text{χρόνος μετάδοσης} = L/R$$

Παράδειγμα:

Auto ▾



$L = 10 \text{ Kbits}$
 $R = 100 \text{ Mbps}$
 $d_{\text{trans}} = 0.1 \text{ ms}$

31. Queueing

Αναμονή εμφανίζεται όταν:

Auto ▾



τα πακέτα φτάνουν πιο γρήγορα από όσο μπορεί να τα στείλει η έξοδος

Τότε:

- δημιουργείται ουρά

- έχουμε καθυστέρηση
 - αν γεμίσει ο buffer, έχουμε απώλειες
-

32. Circuit switching

Στη μεταγωγή κυκλώματος:

- δεσμεύονται πόροι από άκρο σε άκρο
- η σύνδεση έχει εγγυημένη απόδοση
- οι πόροι δεν μοιράζονται
- αν δεν χρησιμοποιούνται, μένουν αδρανείς

Χρησιμοποιήθηκε πολύ σε τηλεφωνικά δίκτυα.

33. FDM και TDM στη μεταγωγή κυκλώματος

FDM

Frequency Division Multiplexing.

Η συχνότητα χωρίζεται σε ζώνες.

Κάθε κλήση παίρνει δική της ζώνη.

TDM

Time Division Multiplexing.

Ο χρόνος χωρίζεται σε χρονοθυρίδες.

Κάθε κλήση παίρνει περιοδικά slots.

34. Packet switching vs circuit switching

Παράδειγμα:

- ζεύξη 1 Gbps
- κάθε χρήστης χρειάζεται 100 Mbps όταν είναι ενεργός
- κάθε χρήστης ενεργός 10% του χρόνου

Circuit switching

Μπορούν να υποστηριχθούν:

1 Gbps / 100 Mbps = 10 χρήστες

Packet switching

Με 35 χρήστες, η πιθανότητα να είναι πάνω από 10 ενεργοί ταυτόχρονα είναι μικρότερη από:

Auto ▾



0.0004

Αυτό λέγεται:

Auto ▾



statistical multiplexing

35. Γιατί packet switching;

Είναι καλό για bursty data.

Πλεονεκτήματα:

- κοινή χρήση πόρων
- απλό
- δεν χρειάζεται call setup
- χρησιμοποιεί καλύτερα το δίκτυο όταν οι χρήστες δεν είναι συνεχώς ενεργοί

Μειονεκτήματα:

- συμφόρηση
- καθυστέρηση
- απώλειες
- χρειάζεται TCP/έλεγχος συμφόρησης για αξιοπιστία

36. Δομή Διαδικτύου

Το Διαδίκτυο αποτελείται από:

- access ISPs
- regional ISPs
- global/tier-1 ISPs
- IXPs

- content provider networks

37. Γιατί δεν συνδέονται όλοι με όλους;

Αν κάθε access ISP συνδεόταν απευθείας με κάθε άλλο:

Auto ▾



$O(N^2)$ συνδέσεις

Δεν κλιμακώνει.

Άρα δημιουργήθηκε ιεραρχική δομή.

38. Global ISP

Κάθε access ISP μπορεί να συνδέεται με global ISP.

Ο access ISP είναι πελάτης.

Ο global ISP είναι πάροχος.

39. Ανταγωνισμός ISPs

Αν υπάρχει ένας global ISP, θα εμφανιστούν ανταγωνιστές.

Οι global ISPs πρέπει να διασυνδεθούν μεταξύ τους.

Αυτό γίνεται μέσω:

Auto ▾



IXP = Internet Exchange Point

και peering links.

40. Regional ISPs

Περιφερειακά δίκτυα συνδέουν access networks με μεγαλύτερους ISPs.

41. Content provider networks

Μεγάλοι πάροχοι περιεχομένου, όπως Google, Microsoft, Akamai, έχουν δικά τους δίκτυα.

Στόχος:

- φέρνουν περιεχόμενο πιο κοντά στους χρήστες
 - παρακάμπτουν κάποιους ISPs
 - μειώνουν καθυστέρηση
 - αυξάνουν απόδοση
-

42. Απόδοση δικτύου

Βασικές έννοιες:

- καθυστέρηση
 - απώλεια
 - throughput
-

43. Γιατί συμβαίνουν καθυστερήσεις;

Τα πακέτα περιμένουν σε buffers των routers.

Αν ο ρυθμός άφιξης είναι μεγαλύτερος από τη χωρητικότητα της εξόδου:

- αυξάνεται η ουρά
 - αυξάνεται η καθυστέρηση
 - μπορεί να υπάρξει απώλεια
-

44. Τέσσερις συνιστώσες καθυστέρησης

Η κομβική καθυστέρηση είναι:

Auto ▾



$$dnodal = dproc + dqueue + dtrans + dprop$$

dproc

Processing delay.

Περιλαμβάνει:

- έλεγχο bit errors
- καθορισμό εξόδου

Συνήθως:

Auto ▾



< microseconds

dqueue

Queueing delay.

Χρόνος αναμονής στην ουρά.

Εξαρτάται από τη συμφόρηση.

dtrans

Transmission delay.

Χρόνος για να μπει όλο το πακέτο στη ζεύξη.

Auto ▾


$$d_{trans} = L / R$$

dprop

Propagation delay.

Χρόνος για να διαδοθεί το σήμα στο μέσο.

Auto ▾


$$d_{prop} = d / s$$

όπου:

- d = μήκος φυσικής ζεύξης
- s = ταχύτητα διάδοσης, περίπου 2×10^8 m/s

45. Transmission delay vs propagation delay

Είναι διαφορετικά.

Auto ▾



$dtrans$ = χρόνος για να βάλω τα bits στη γραμμή
 $dprop$ = χρόνος για να ταξιδέψουν τα bits στη γραμμή

Αναλογία:

- $dtrans$ = χρόνος για να περάσουν όλα τα αυτοκίνητα από τα διόδια
- $dprop$ = χρόνος για να ταξιδέψει ένα αυτοκίνητο μέχρι τα επόμενα διόδια

46. Ένταση κίνησης

Ορίζεται:

Auto ▾

 L_a/R

όπου:

- L = μήκος πακέτου σε bits
- a = μέσος ρυθμός άφιξης πακέτων/sec
- R = ρυθμός μετάδοσης bits/sec

Ερμηνεία

Auto (PowerShell) ▾



$L_a/R \approx 0 \rightarrow$ μικρή καθυστέρηση
 $L_a/R \rightarrow 1 \rightarrow$ μεγάλη καθυστέρηση
 $L_a/R > 1 \rightarrow$ άπειρη μέση καθυστέρηση

Γιατί όταν φτάνει περισσότερη δουλειά από όση μπορεί να εξυπηρετηθεί, η ουρά μεγαλώνει χωρίς όριο.

47. Traceroute

Το traceroute μετρά καθυστερήσεις από την πηγή μέχρι κάθε router στο μονοπάτι.

Στέλνει 3 πακέτα για κάθε TTL.

TTL:

Auto ▾



1, 2, 3, ...

Ο router όπου λήγει το TTL επιστρέφει μήνυμα στην πηγή.

Η πηγή μετρά RTT.

Το * σημαίνει:

- χάθηκε πακέτο
- ή ο router δεν απάντησε

48. Απώλεια πακέτων

Οι buffers έχουν πεπερασμένη χωρητικότητα.

Αν φτάσει πακέτο και ο buffer είναι γεμάτος:

Auto ▾



το πακέτο απορρίπτεται

Μπορεί να επαναμεταδοθεί:

- από προηγούμενο κόμβο
- από την πηγή
- ή να μη μεταδοθεί ξανά

49. Throughput

Throughput = ρυθμός με τον οποίο μεταφέρονται bits από αποστολέα σε παραλήπτη.

Μονάδα:

Auto ▾



bits/sec

Στιγμιαίο throughput

Ρυθμός σε μια συγκεκριμένη στιγμή.

Μέσο throughput

Ρυθμός σε μεγάλο χρονικό διάστημα.

50. Bottleneck link

Η από άκρο σε άκρο ρυθμαπόδοση περιορίζεται από τη μικρότερη χωρητικότητα στη διαδρομή.

Αν έχουμε:

Auto ▾



$$R_s < R_c$$

throughput = R_s .

Αν:

Auto ▾



$$R_s > R_c$$

throughput = R_c .

Γενικά:

Auto ▾



$$\text{throughput} = \min(R_s, R_c)$$

51. Throughput σε δίκτυο με 10 συνδέσεις

Αν 10 συνδέσεις μοιράζονται ζεύξη R :

Auto ▾



$$\text{throughput ανά σύνδεση} = \min(R_c, R_s, R/10)$$

52. Layers

Τα δίκτυα είναι πολύπλοκα.

Έχουν:

- hosts
- routers

- links
- applications
- protocols
- hardware
- software

Για να οργανωθούν, χρησιμοποιούμε επίπεδα.

53. Γιατί layering;

Πλεονεκτήματα:

- καλύτερη οργάνωση
 - ευκολότερη κατανόηση
 - διαχωρισμός λειτουργιών
 - ευκολότερη συντήρηση
 - δυνατότητα αλλαγής ενός επιπέδου χωρίς να αλλάζουν όλα
-

54. Αναλογία αεροπορικού ταξιδιού

Ταξίδι περιλαμβάνει επίπεδα:

- αγορά εισιτηρίου
- έλεγχος αποσκευών
- επιβίβαση
- απογείωση
- δρομολόγηση αεροπλάνου
- προσγείωση
- αποβίβαση
- παραλαβή αποσκευών

Κάθε επίπεδο προσφέρει υπηρεσία στο ανώτερο επίπεδο.

55. Internet protocol stack

Το Internet έχει 5 επίπεδα:

Auto ▾



Application
Transport
Network
Link
Physical

56. Application layer

Υποστηρίζει δικτυακές εφαρμογές.

Παραδείγματα:

- HTTP
- IMAP
- SMTP
- DNS

Μονάδα δεδομένων:

Auto ▾



message

57. Transport layer

Μεταφέρει δεδομένα από διεργασία σε διεργασία.

Πρωτόκολλα:

- TCP
- UDP

Μονάδα δεδομένων:

Auto ▾



segment

58. Network layer

Δρομολογεί datagrams από πηγή σε προορισμό.

Πρωτόκολλα:

- IP

- routing protocols

Μονάδα δεδομένων:

Auto ▾



datagram

59. Link layer

Μεταφέρει δεδομένα ανάμεσα σε γειτονικά δικτυακά στοιχεία.

Πρωτόκολλα:

- Ethernet
- WiFi
- PPP

Μονάδα δεδομένων:

Auto ▾



frame

60. Physical layer

Μεταφέρει bits πάνω στο φυσικό μέσο.

Μονάδα:

Auto ▾



bit

61. Encapsulation

Κάθε επίπεδο προσθέτει δική του κεφαλίδα.

Ξεκινάμε με μήνυμα εφαρμογής:

Auto ▾



M

To transport προσθέτει header:

Auto ▾



Ht | M

To network προσθέτει header:

Auto ▾



Hn | Ht | M

To link προσθέτει header:

Auto ▾



Hl | Hn | Ht | M

62. Ονόματα PDU ανά επίπεδο

Επίπεδο	PDU
Application	message
Transport	segment
Network	datagram
Link	frame
Physical	bits

63. Encapsulation end-to-end

Σε host:

- υπάρχουν όλα τα επίπεδα

Σε switch:

- κυρίως link και physical

Σε router:

- physical
- link
- network

Ο router αφαιρεί link header και βάζει νέο link header για την επόμενη ζεύξη.

Το IP datagram παραμένει, αλλά το frame αλλάζει από link σε link.

64. Ασφάλεια δικτύου

Το Διαδίκτυο δεν σχεδιάστηκε αρχικά με ασφάλεια.

Αρχική λογική:

Auto ▾



χρήστες με αμοιβαία εμπιστοσύνη σε διαφανές δίκτυο

Σήμερα χρειάζεται ασφάλεια σε όλα τα επίπεδα.

65. Packet sniffing

Ο επιτιθέμενος καταγράφει πακέτα.

Γίνεται εύκολα σε:

- broadcast Ethernet
- ασύρματο δίκτυο

Μπορεί να διαβάσει:

- δεδομένα
- κωδικούς
- μηνύματα

Παράδειγμα εργαλείου:

Auto ▾



66. IP spoofing

Ο επιτιθέμενος στέλνει πακέτα με ψεύτικη IP πηγής.

Δηλαδή:

Auto ▾



src IP ≠ πραγματικός αποστολέας

67. Denial of Service

DoS = Denial of Service.

Στόχος:

Auto ▾



να γίνει ένας πόρος μη διαθέσιμος

Ο επιτιθέμενος υπερφορτώνει:

- server
- bandwidth
- δίκτυο

Βήματα:

16098. Επιλογή στόχου.

16099. Παραβίαση πολλών υπολογιστών.

16100. Αποστολή πολλών πακέτων προς στόχο.

68. Γραμμές άμυνας

Μέτρα ασφάλειας:

- αυθεντικοποίηση
- κρυπτογράφηση
- έλεγχος ακεραιότητας

- ψηφιακές υπογραφές
 - VPN
 - firewalls
 - περιορισμός πρόσβασης
 - ανίχνευση DoS
-

69. Αυθεντικοποίηση

Αποδεικνύεις ότι είσαι αυτός που λες.

Παράδειγμα:

- SIM authentication στα κινητά δίκτυα
-

70. Εμπιστευτικότητα

Προστασία περιεχομένου μέσω κρυπτογράφησης.

71. Ακεραιότητα

Έλεγχος ότι το μήνυμα δεν αλλοιώθηκε.

Χρησιμοποιούνται:

- checks
 - digital signatures
-

72. Firewalls

Firewalls είναι middleboxes που φιλτράρουν πακέτα.

Μπορούν να περιορίζουν:

- αποστολείς
 - παραλήπτες
 - εφαρμογές
 - θύρες
-

73. Ιστορία Διαδικτύου

1961–1972: αρχές packet switching

- 1961: Kleinrock, θεωρία ουρών, αποτελεσματικότητα packet switching
- 1964: Baran, packet switching για στρατιωτικά δίκτυα

- 1967: ιδέα ARPAnet
 - 1969: πρώτος κόμβος ARPAnet
 - 1972: δημόσια επίδειξη ARPAnet
 - 1972: NCP, πρώτο host-to-host protocol
 - 1972: πρώτο email πρόγραμμα
 - ARPAnet: 15 κόμβοι
-

1972–1980: διαδικτύωση

- 1970: ALOHAnet στη Χαβάη
- 1974: Cerf και Kahn
- 1976: Ethernet στο Xerox PARC
- τέλη 1970s: DECnet, SNA, XNA
- 1979: ARPAnet με 200 κόμβους

Αρχές Cerf και Kahn:

- μινιμαλισμός
 - αυτονομία
 - best effort service
 - stateless routers
 - decentralized control
-

1980–1990

- 1983: TCP/IP
 - 1982: SMTP για email
 - 1983: DNS
 - 1985: FTP
 - 1988: TCP congestion control
 - νέα δίκτυα: CSnet, BITnet, NSFnet, Minitel
 - περίπου 100.000 hosts
-

1990–2000s

- ARPAnet παροπλίζεται
- NSFnet ανοίγει για εμπορική χρήση
- Web εμφανίζεται
- HTML και HTTP από Berners-Lee
- 1994: Mosaic
- μετά Netscape
- εμπορευματοποίηση Web
- instant messaging

- P2P file sharing
 - ασφάλεια στο προσκήνιο
 - 50 εκατομμύρια hosts
 - πάνω από 100 εκατομμύρια χρήστες
 - Gbps links στον πυρήνα
-

2005–σήμερα

- ευρυζωνική πρόσβαση
 - 2008: SDN
 - 4G/5G
 - WiFi
 - δίκτυα παρόχων περιεχομένου
 - cloud services
 - AWS
 - Microsoft Azure
 - περισσότερες κινητές από σταθερές συσκευές από το 2017
 - περίπου 18 δισ. συσκευές στο Internet το 2017
-

74. SOS για εξέταση

Auto ▾



Internet = δίκτυο δικτύων.

Auto ▾



Protocol = κανόνες για μορφή, σειρά μηνυμάτων και ενέργειες.

Auto ▾



Host = τερματικό σύστημα που τρέχει εφαρμογές.

Auto ▾



Access network = συνδέει host με πρώτο router.

Auto ▾



Network core = routers που προωθούν πακέτα.

Auto ▾



Forwarding = τοπική μεταφορά πακέτου από είσοδο σε έξοδο router.

Auto ▾



Routing = υπολογισμός μονοπατιού από πηγή σε προορισμό.

Auto ▾



Packet switching = σπάω μήνυμα σε πακέτα, μοιράζομαι πόρους.

Auto ▾



Circuit switching = δεσμεύω πόρους από άκρο σε άκρο.

Auto (PowerShell) ▾



Store-and-forward = ο router λαμβάνει όλο το πακέτο πριν το προωθήσει.

Auto ▾



$dtrans = L/R$

Auto ▾



$dprop = d/s$

Auto ▾



$dnodal = dproc + dqueue + dtrans + dprop$

Auto ▾



Traffic intensity = $\lambda a/R$

Auto ▾



$\lambda a/R > 1 \rightarrow$ η ουρά μεγαλώνει χωρίς όριο.

Auto ▾



Throughput = ρυθμός μεταφοράς bits από sender σε receiver.

Auto ▾



Bottleneck link = η πιο αργή ζεύξη που περιορίζει το throughput.

Auto (C++) ▾



Internet stack:

Application → Transport → Network → Link → Physical

Auto ▾



Encapsulation:

M → HtM → HnHtM → HlHnHtM

Auto ▾



Packet sniffing = καταγραφή πακέτων.

Auto ▾



IP spoofing = ψεύτικη IP πηγής.

Auto ▾



DoS = υπερφόρτωση πόρου ώστε να μη λειτουργεί.

75. Βασικοί τύποι

Transmission delay

Auto ▾


$$d_{trans} = L / R$$

Propagation delay

Auto ▾



$$dprop = d / s$$

Nodal delay

Auto ▾



$$dnodal = dproc + dqueue + dtrans + dprop$$

Traffic intensity

Auto ▾



$$La / R$$

Throughput απλής διαδρομής

Auto ▾



$$\text{throughput} = \min(Rs, Rc)$$

Throughput με 10 συνδέσεις

Auto ▾



$$\text{throughput ανά σύνδεση} = \min(Rc, Rs, R/10)$$

76. Μικρό μνημονικό

Auto ▾



Host φτιάχνει μήνυμα.
Transport το κάνει segment.
Network το κάνει datagram.
Link το κάνει frame.
Physical το κάνει bits.

Auto ▾



Routing βρίσκει δρόμο.
Forwarding κάνει τοπική προώθηση.

Auto ▾



Transmission = βάζω bits στη γραμμή.
Propagation = ταξιδεύουν τα bits στη γραμμή.

Auto (Java) ▾



Packet switching = στατιστικό μοίρασμα.
Circuit switching = δεσμευμένο κύκλωμα

Δίκτυα Επικοινωνιών Ι

Ενότητα 2: Επίπεδο Εφαρμογής

1. Στόχοι ενότητας

Στο επίπεδο εφαρμογής μαθαίνουμε:

- πώς σχεδιάζονται δικτυακές εφαρμογές
- τι είναι client-server
- τι είναι peer-to-peer
- τι υπηρεσίες ζητούν οι εφαρμογές από το επίπεδο μεταφοράς
- TCP και UDP
- HTTP
- cookies
- web caching
- email / SMTP / IMAP / POP3
- DNS
- P2P / BitTorrent
- video streaming / DASH / CDNs
- socket API

2. Παραδείγματα δικτυακών εφαρμογών

- απομακρυσμένη σύνδεση
- e-mail
- Web
- αναζήτηση στον Ιστό
- κοινωνικά δίκτυα
- instant messaging
- VoIP, π.χ. Skype
- Zoom
- online games
- YouTube / Netflix / Hulu
- P2P file sharing

3. Πώς φτιάχνουμε δικτυακή εφαρμογή

Γράφουμε προγράμματα που:

- τρέχουν σε διαφορετικά τερματικά συστήματα
- επικοινωνούν μέσω δικτύου
- ανταλλάσσουν μηνύματα

Δεν γράφουμε εφαρμογές για routers του πυρήνα.

Οι εφαρμογές βρίσκονται στα άκρα του δικτύου.

Auto ▾



Η πολυπλοκότητα βρίσκεται στα άκρα.

4. Αρχιτεκτονικές εφαρμογών

Υπάρχουν δύο βασικές:

Auto ▾



Client-server

Peer-to-peer

5. Client-server

Server

Ο server:

- είναι διαρκώς ενεργός
- έχει μόνιμη IP
- συχνά βρίσκεται σε data center
- εξυπηρετεί πολλούς clients

Client

Ο client:

- επικοινωνεί με server
- μπορεί να έχει δυναμική IP
- μπορεί να συνδέεται διακοπτόμενα
- δεν επικοινωνεί συνήθως απευθείας με άλλους clients

Παραδείγματα:

- HTTP
 - FTP
 - IMAP
-

6. Peer-to-peer

Στο P2P:

- δεν υπάρχει διαρκώς ενεργός server
- οι peers επικοινωνούν απευθείας
- κάθε peer μπορεί να ζητά και να προσφέρει υπηρεσία
- οι peers μπαίνουν και βγαίνουν συχνά
- αλλάζουν IP
- έχει αυτοκλιμάκωση

Παράδειγμα:

Auto ▾



BitTorrent

7. Διεργασίες

Διεργασία = πρόγραμμα που τρέχει σε υπολογιστή.

Auto ▾



process = instance of a program

Διεργασίες στον ίδιο υπολογιστή επικοινωνούν μέσω λειτουργικού.

Διεργασίες σε διαφορετικούς υπολογιστές επικοινωνούν με μηνύματα.

8. Client process και server process

Auto (Java) ▾



Client process = ξεκινά την επικοινωνία
Server process = περιμένει να επικοινωνήσουν μαζί του

Σε P2P εφαρμογές, μια διεργασία μπορεί να είναι και client και server.

9. Socket

Socket = «πόρτα» της διεργασίας προς το δίκτυο.

Η διεργασία:

- στέλνει μηνύματα στο socket

- λαμβάνει μηνύματα από το socket

Το socket είναι API μεταξύ:

Auto ▾



Application layer

Transport layer

10. Διευθυνσιοδότηση διεργασιών

Για να βρούμε μια διεργασία χρειάζονται:

Auto ▾



IP address + port number

Η IP μόνη της δεν αρκεί, γιατί σε έναν υπολογιστή τρέχουν πολλές διεργασίες.

Παραδείγματα ports:

Auto ▾



HTTP server: 80

Mail server SMTP: 25

Παράδειγμα:

Auto ▾



gaia.cs.umass.edu

IP: 128.119.245.12

Port: 80

11. Τι ορίζει ένα πρωτόκολλο εφαρμογής

Ένα πρωτόκολλο επιπέδου εφαρμογής ορίζει:

- είδη μηνυμάτων
- σύνταξη μηνυμάτων
- πεδία μηνυμάτων

- σημασία πεδίων
- κανόνες αποστολής
- κανόνες απόκρισης

12. Ανοιχτά και ιδιοταγή πρωτόκολλα

Ανοιχτά πρωτόκολλα

- ορίζονται σε RFCs
- όλοι έχουν πρόσβαση στις λεπτομέρειες
- επιτρέπουν διαλειτουργικότητα

Παραδείγματα:

Auto ▾



HTTP, SMTP

Ιδιοταγή πρωτόκολλα

Ανήκουν σε εταιρείες.

Παραδείγματα:

Auto ▾



Skype, Zoom

13. Απαιτήσεις εφαρμογών από το επίπεδο μεταφοράς

Οι εφαρμογές μπορεί να ζητούν:

- αξιοπιστία
- χαμηλή καθυστέρηση
- ελάχιστη ρυθμαπόδοση
- ασφάλεια

14. Ακεραιότητα δεδομένων

Κάποιες εφαρμογές δεν αντέχουν απώλειες:

- file transfer
- e-mail
- web documents
- online συναλλαγές

Άλλες αντέχουν κάποιες απώλειες:

- ήχος
 - βίντεο
 - παιχνίδια
-

15. Χρονισμός

Εφαρμογές ευαίσθητες στον χρόνο:

- VoIP
- online games
- Zoom

Χρειάζονται χαμηλή καθυστέρηση.

16. Throughput

Κάποιες εφαρμογές χρειάζονται ελάχιστο throughput:

- video streaming
- multimedia

Άλλες είναι ελαστικές:

- email
- web
- file transfer

Ελαστική εφαρμογή = δουλεύει με όση ρυθμαπόδοση πάρει.

17. TCP

Το TCP παρέχει:

- αξιόπιστη μεταφορά
- έλεγχο ροής
- έλεγχο συμφόρησης
- connection-oriented επικοινωνία

Δεν παρέχει:

- εγγυημένο timing
- εγγυημένο minimum throughput
- ασφάλεια

18. UDP

Το UDP παρέχει:

- απλή αποστολή datagrams
- χωρίς σύνδεση
- χωρίς αξιοπιστία
- χωρίς έλεγχο ροής
- χωρίς έλεγχο συμφόρησης
- χωρίς εγγύηση χρόνου
- χωρίς εγγύηση throughput
- χωρίς ασφάλεια

Γιατί υπάρχει;

Επειδή είναι απλό, γρήγορο και χρήσιμο όταν η εφαρμογή θέλει να ελέγχει η ίδια τι θα κάνει.

19. Εφαρμογές και πρωτόκολλα μεταφοράς

Εφαρμογή	Application protocol	Transport
File transfer	FTP	TCP
E-mail	SMTP	TCP
Web	HTTP	TCP
Internet telephony	SIP/RTP/proprietary	TCP ή UDP
Streaming	HTTP/DASH	TCP
Games	proprietary	UDP ή TCP

20. TLS

TCP και UDP από μόνα τους δεν κρυπτογραφούν.

Το TLS προσθέτει:

- κρυπτογράφηση
- αυθεντικοποίηση
- ακεραιότητα

Το TLS υλοποιείται στο επίπεδο εφαρμογής και χρησιμοποιεί TCP.

Auto ▾



HTTP + TLS = HTTPS

21. Web και HTTP

Μια ιστοσελίδα αποτελείται από αντικείμενα.

Αντικείμενο μπορεί να είναι:

- HTML
- εικόνα JPEG
- ήχος
- video
- script

Η ιστοσελίδα έχει URL.

Παράδειγμα:

Auto ▾



`www.someschool.edu/someDept/pic.gif`

όπου:

Auto ▾



`www.someschool.edu` = hostname
`/someDept/pic.gif` = path

22. HTTP

HTTP = HyperText Transfer Protocol.

Είναι πρωτόκολλο επιπέδου εφαρμογής του Web.

Χρησιμοποιεί client-server μοντέλο.

Client

Ο browser:

- ζητά αντικείμενα
- λαμβάνει αντικείμενα
- εμφανίζει ιστοσελίδες

Server

Ο web server:

- λαμβάνει requests
- στέλνει responses

23. HTTP και TCP

Το HTTP χρησιμοποιεί TCP.

Βήματα:

5615. Ο client ανοίγει TCP σύνδεση με server στη θύρα 80.
5616. Ο server αποδέχεται τη σύνδεση.
5617. Ανταλλάσσονται HTTP μηνύματα.
5618. Η TCP σύνδεση κλείνει ή παραμένει ανοιχτή.

24. HTTP είναι stateless

Το HTTP είναι:

Auto ▾



stateless

Δηλαδή ο server δεν θυμάται προηγούμενες αιτήσεις του client.

Κάθε HTTP request είναι ανεξάρτητο.

25. Non-persistent HTTP

Σε non-persistent HTTP:

6014. ανοίγει TCP σύνδεση
6015. στέλνεται ένα αντικείμενο

6016. κλείνει TCP σύνδεση

Για πολλά αντικείμενα χρειάζονται πολλές TCP συνδέσεις.

Χρόνος απόκρισης:

Auto ▾



2 RTT + χρόνος μετάδοσης αρχείου

26. Persistent HTTP

Σε persistent HTTP:

- ανοίγει μία TCP σύνδεση
- στέλνονται πολλά αντικείμενα
- κλείνει η σύνδεση μετά

Πλεονεκτήματα:

- λιγότερα RTT
- μικρότερο overhead
- πιο γρήγορη λήψη πολλών αντικειμένων

27. HTTP request

Παράδειγμα:

Auto (Visual Basic .NET) ▾



```
GET /index.html HTTP/1.1
Host: www-net.cs.umass.edu
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
```

Το request έχει:

- request line
- header lines
- blank line
- προαιρετικό body

28. HTTP μέθοδοι

GET

Ζητά αντικείμενο.

Μπορεί να στείλει δεδομένα μέσα στο URL:

Auto ▾



/search?monkeys&banana

POST

Στέλνει δεδομένα στο entity body, π.χ. φόρμα.

HEAD

Ζητά μόνο headers, όχι το αντικείμενο.

PUT

Ανεβάζει ή αντικαθιστά αρχείο στον server.

29. HTTP response

Παράδειγμα:

Auto (Visual Basic .NET) ▾



```
HTTP/1.1 200 OK
Date: ...
Server: Apache
Last-Modified: ...
Content-Length: 2651
Content-Type: text/html
```

Περιέχει:

- status line
- header lines
- body με τα δεδομένα

30. HTTP status codes

Κωδικός	Σημασία
200 OK	επιτυχία
301 Moved Permanently	μετακινήθηκε μόνιμα
400 Bad Request	λάθος αίτημα
404 Not Found	δεν βρέθηκε
505 HTTP Version Not Supported	μη υποστηριζόμενη έκδοση

31. Cookies

Τα cookies επιτρέπουν διατήρηση κατάστασης σε stateless HTTP.

Χρησιμοποιούνται για:

- login
- καλάθια αγορών
- συστάσεις
- web mail
- tracking

32. Πώς δουλεύουν τα cookies

Υπάρχουν 4 στοιχεία:

7679. **Set-Cookie** στο HTTP response.

7680. **Cookie** σε επόμενα HTTP requests.

7681. Αρχείο cookies στον browser.

7682. Βάση δεδομένων στον server.

33. Cookies και ιδιωτικότητα

Τα cookies μπορούν να παρακολουθούν:

- συμπεριφορά σε ένα site
- συμπεριφορά σε πολλά sites

First-party cookies

Από το site που επέλεξες να επισκεφθείς.

Third-party cookies

Από τρίτο site, π.χ. διαφημιστικό δίκτυο.

Τα third-party cookies επιτρέπουν tracking σε πολλούς ιστοτόπους.

Σχετίζονται με GDPR και προσωπικά δεδομένα.

34. Web cache / proxy server

Η web cache αποθηκεύει αντίγραφα αντικειμένων.

Ο browser στέλνει requests στην cache.

Αν το αντικείμενο υπάρχει στην cache:

Auto ▾



cache hit

Η cache το επιστρέφει.

Αν δεν υπάρχει:

Auto ▾



cache miss

Η cache το ζητά από τον origin server.

35. Γιατί web caching;

Πλεονεκτήματα:

- μικρότερος χρόνος απόκρισης
 - λιγότερη κίνηση στη ζεύξη πρόσβασης
 - μικρότερο φορτίο στον origin server
 - καλύτερη εξυπηρέτηση περιεχομένου
-

36. Παράδειγμα caching

Δεδομένα:

Auto (Bash) ▾



```
access link = 1.54 Mbps  
μέσο αντικείμενο = 100 Kbits  
15 requests/sec  
μέσος ρυθμός = 1.50 Mbps
```

Αξιοποίηση ζεύξης:

Auto ▾


$$1.50 / 1.54 = 0.97$$

Πολύ υψηλή → μεγάλες ουρές.

Αν cache hit rate = 0.4:

Auto (Bash) ▾



```
60% περνά από access link  
0.6 * 1.5 Mbps = 0.9 Mbps  
0.9 / 1.54 = 0.58
```

Άρα μικρότερη καθυστέρηση.

Μέση καθυστέρηση:

Auto ▾


$$0.6 * 2.01 + 0.4 * \text{λίγα ms} \approx 1.2 \text{ sec}$$

37. Conditional GET

Σκοπός:

Auto ▾



να μη στέλνεται ξανά αντικείμενο αν η cache έχει ενημερωμένο αντίγραφο

Ο client στέλνει:

Auto ▾



If-Modified-Since: <date>

Αν δεν άλλαξε:

Auto ▾



304 Not Modified

Αν άλλαξε:

Auto ▾



200 OK
<data>

38. HTTP/2

Στόχος:

Auto ▾



μείωση καθυστέρησης όταν υπάρχουν πολλά αντικείμενα

Πρόβλημα HTTP/1.1:

- FCFS απαντήσεις
- μικρά αντικείμενα μπορεί να περιμένουν πίσω από μεγάλο
- αυτό λέγεται HOL blocking

HTTP/2:

- χωρίζει αντικείμενα σε frames
 - κάνει interleaving
 - επιτρέπει καλύτερο scheduling
 - υποστηρίζει server push
 - αποφεύγει application-level HOL blocking
-

39. HTTP/3

Το HTTP/2 πάνω από TCP ακόμη παγώνει λόγω απώλειας TCP segment.

HTTP/3:

- βασίζεται πάνω από UDP
 - προσθέτει ασφάλεια
 - έλεγχο σφαλμάτων
 - έλεγχο συμφόρησης
 - καλύτερο pipelining ανά αντικείμενο
-

40. Email

Το email έχει 3 βασικά μέρη:

10020. User agents

10021. Mail servers

10022. SMTP

41. User agent

Ο user agent είναι το πρόγραμμα email.

Παραδείγματα:

- Outlook
- Thunderbird
- iPhone Mail

Κάνει:

- σύνθεση email
 - ανάγνωση
 - επεξεργασία
-

42. Mail server

Ο mail server έχει:

- mailbox για εισερχόμενα
 - ουρά εξερχόμενων μηνυμάτων
-

43. SMTP

SMTP = Simple Mail Transfer Protocol.

Χρησιμοποιεί:

Auto ▾



TCP port 25

Μεταφέρει email από mail server αποστολέα σε mail server παραλήπτη.

Φάσεις:

10502. χειραψία

10503. μεταφορά μηνύματος

10504. τερματισμός

Είναι ASCII command/response πρωτόκολλο.

44. Σενάριο email

Η Αλίκη στέλνει email στον Βασίλη:

10656. Η Αλίκη γράφει μήνυμα στον user agent.

10657. Ο user agent στέλνει το μήνυμα στον mail server της Αλίκης.

10658. Ο mail server της Αλίκης ανοίγει TCP σύνδεση με mail server Βασίλη.

10659. SMTP μεταφέρει το μήνυμα.

10660. Ο server Βασίλη το βάζει στο mailbox.

10661. Ο Βασίλης το διαβάζει με user agent.

45. Mail access protocols

Το SMTP στέλνει email στον server παραλήπτη.

Για να το διαβάσει ο χρήστης χρειάζεται mail access protocol.

IMAP

- τα μηνύματα μένουν στον server
- υποστηρίζει ανάκτηση
- διαγραφή
- οργάνωση φακέλων

POP3

- authentication
- download μηνυμάτων

HTTP

Χρησιμοποιείται από webmail:

- Gmail
 - Yahoo Mail
 - Hotmail
-

46. DNS

DNS = Domain Name System.

Μεταφράζει:

Auto ▾



hostname → IP address

Παράδειγμα:

Auto ▾



di.uoa.gr → IP address

Είναι:

- κατανεμημένη βάση
 - ιεραρχική
 - πρωτόκολλο εφαρμογής
 - βασική λειτουργία του Internet
-

47. Γιατί όχι κεντρικό DNS;

Ένα κεντρικό DNS δεν κλιμακώνει γιατί θα είχε:

- μοναδικό σημείο αποτυχίας
 - τεράστια κίνηση
 - μεγάλη καθυστέρηση
 - δύσκολη συντήρηση
-

48. Υπηρεσίες DNS

Το DNS παρέχει:

- hostname to IP translation

- host aliasing
 - mail server aliasing
 - load distribution
-

49. Ιεραρχία DNS

Υπάρχουν:

Auto (SQL) ▾



Root DNS servers

TLD DNS servers

Authoritative DNS servers

Local DNS servers

50. Root DNS servers

Οι root servers δείχνουν προς TLD servers.

Υπάρχουν 13 λογικοί root servers με πολλά αντίγραφα παγκοσμίως.

Το ICANN διαχειρίζεται το root DNS domain.

51. TLD servers

TLD = Top Level Domain.

Παραδείγματα:

Auto ▾



.com

.org

.net

.edu

.gr

.uk

.fr

Οι TLD servers δείχνουν στους authoritative servers.

52. Authoritative DNS servers

Έχουν τις αυθεντικές εγγραφές ενός οργανισμού.

Παράδειγμα:

Auto ▾



umass.edu authoritative DNS

Ξέρει τις IP για hosts του umass.edu.

53. Local DNS server

Κάθε ISP/οργανισμός έχει local DNS.

Ο host πρώτα ρωτά τον local DNS.

Ο local DNS:

- απαντά από cache
- ή λειτουργεί ως proxy προς την ιεραρχία DNS

54. Iterative DNS query

Ο server απαντά:

Auto ▾



Δεν ξέρω, ρώτα αυτόν.

Παράδειγμα:

12772. Local DNS ρωτά root.

12773. Root δείχνει TLD.

12774. TLD δείχνει authoritative.

12775. Authoritative δίνει IP.

55. Recursive DNS query

Ο server που ρωτήθηκε αναλαμβάνει να βρει όλη την απάντηση.

Έχει μεγαλύτερο φορτίο στα ανώτερα επίπεδα.

Χρησιμοποιείται λιγότερο.

56. DNS caching

Όταν ένας DNS server μάθει αντιστοίχιση, την αποθηκεύει προσωρινά.

Πλεονεκτήματα:

- μικρότερος χρόνος απόκρισης
- λιγότερα queries

Οι εγγραφές λήγουν με TTL.

Μειονέκτημα:

Auto ▾



μπορεί προσωρινά να μην είναι ενημερωμένες

57. DNS Resource Records

Μορφή:

Auto ▾



(name, value, type, ttl)

Type A

Auto ▾



name = hostname
value = IP address

Type NS

Auto ▾



name = domain
value = authoritative name server

Type CNAME

Auto ▾



```
name = alias  
value = canonical name
```

Type MX

Auto ▾



```
name = domain  
value = mail server
```

58. DNS messages

DNS query και reply έχουν ίδιο format.

Περιλαμβάνουν:

- identification
- flags
- number of questions
- number of answers
- authority RRs
- additional RRs

To identification επιτρέπει να ταιριάζει request με response.

59. P2P file distribution

Ερώτηση:

Auto ▾



Πόσος χρόνος χρειάζεται για διανομή αρχείου F σε N peers;

60. Client-server χρόνος διανομής

Ο server πρέπει να ανεβάσει N αντίγραφα:

Auto ▾



NF / us

Ο πιο αργός client πρέπει να κατεβάσει:

Auto ▾



$F / d_{\min}$

Άρα:

Auto ▾



$D_{c-s} \geq \max \{ NF/us, F/d_{\min} \}$

Αυξάνει γραμμικά με N .

61. P2P χρόνος διανομής

Ο server πρέπει να ανεβάσει τουλάχιστον ένα αντίγραφο:

Auto ▾



F/us

Ο πιο αργός peer πρέπει να κατεβάσει:

Auto ▾



$F/d_{\min}$

Συνολικά πρέπει να ανέβουν NF bits από server και peers:

Auto ▾


$$NF / (u_s + \sum u_i)$$

Άρα:

Auto ▾


$$DP2P \geq \max \{ F/u_s, F/d_{\min}, NF/(u_s + \sum u_i) \}$$

Το P2P κλιμακώνει καλύτερα γιατί κάθε νέος peer φέρνει και upload capacity.

62. BitTorrent

Το αρχείο χωρίζεται σε chunks:

Auto ▾



256 KB

Υπάρχει torrent:

Auto ▾



ομάδα peers που ανταλλάσσουν chunks

Υπάρχει tracker:

Auto ▾



παρακολουθεί ποιοι peers συμμετέχουν

63. Πώς δουλεύει BitTorrent

Ο peer:

14899. μπαίνει στο torrent

14900. παίρνει λίστα peers από tracker

14901. συνδέεται με subset peers

14902. κατεβάζει chunks

14903. ανεβάζει chunks σε άλλους

Οι peers μπορούν να φύγουν ή να μπουν συνεχώς.

Αυτό λέγεται:

Auto ▾



churn

64. Rarest first

Ο peer ζητά πρώτα τα πιο σπάνια chunks.

Σκοπός:

Auto ▾



να μη χαθούν σπάνια κομμάτια και να διαδοθούν γρήγορα

65. Tit-for-tat

Η Αλίκη στέλνει chunks στους 4 peers που της στέλνουν με υψηλότερο ρυθμό.

Κάθε 10 sec επαναξιολογεί top 4.

Κάθε 30 sec επιλέγει τυχαία έναν ακόμη peer:

Auto ▾



optimistic unchoke

Σκοπός:

- να βρει καλύτερους συνεργάτες
- να δώσει ευκαιρία σε νέους peers

66. Video streaming

Το video είναι ακολουθία εικόνων.

Παράδειγμα:

Auto ▾



24 frames/sec

Κάθε εικόνα αποτελείται από pixels.

Κάθε pixel περιγράφεται με bits.

67. Κωδικοποίηση video

Χρησιμοποιεί πλεονασμό για μείωση bits.

Spatial coding

Μέσα στην ίδια εικόνα.

Παράδειγμα:

αντί να στείλω 100 ίδιες μωβ τιμές, στέλνω:

Auto ▾



μωβ, 100 φορές

Temporal coding

Μεταξύ διαδοχικών εικόνων.

Στέλνω μόνο αλλαγές από frame σε frame.

68. CBR και VBR

CBR

Constant Bit Rate.

Σταθερός ρυθμός κωδικοποίησης.

VBR

Variable Bit Rate.

Μεταβλητός ρυθμός ανάλογα με το περιεχόμενο.

Παραδείγματα:

- MPEG1: 1.5 Mbps

- MPEG2: 3–6 Mbps
 - MPEG4: 64 Kbps – 12 Mbps
-

69. Προκλήσεις video streaming

Προβλήματα:

- μεταβαλλόμενο throughput
 - συμφόρηση
 - απώλειες
 - καθυστέρηση
 - jitter
 - ανάγκη συνεχούς αναπαραγωγής
-

70. Client buffer

Ο client αποθηκεύει προσωρινά video πριν το παίξει.

Σκοπός:

Auto ▾



να απορροφήσει καθυστερήσεις και jitter

Αν ο buffer αδειάσει:

Auto ▾



starvation / πάγωμα video

71. DASH

DASH = Dynamic Adaptive Streaming over HTTP.

Ο server:

- χωρίζει video σε chunks
- κωδικοποιεί κάθε chunk σε πολλούς ρυθμούς
- αποθηκεύει διαφορετικές εκδόσεις
- δίνει manifest file με URLs

Ο client:

- μετρά διαθέσιμο bandwidth
 - διαλέγει chunk
 - διαλέγει ποιότητα
 - αλλάζει δυναμικά ποιότητα
 - μπορεί να ζητήσει chunks από διαφορετικούς servers
-

72. Manifest file

Το manifest περιέχει:

- URLs για chunks
- διαθέσιμες ποιότητες
- διαφορετικούς ρυθμούς encoding

Είναι σαν «χάρτης» του video.

73. Εξυπνάδα στο DASH

Η εξυπνάδα είναι στον client.

Ο client αποφασίζει:

- πότε να ζητήσει chunk
 - ποια ποιότητα να ζητήσει
 - από πού να το ζητήσει
-

74. CDN

CDN = Content Delivery Network.

Σκοπός:

Auto ▾



να διανέμει περιεχόμενο σε πολλούς χρήστες από κοντινούς servers

Χωρίς CDN, ένας mega-server έχει:

- single point of failure
 - συμφόρηση
 - μεγάλα μονοπάτια
 - κακή κλιμάκωση
-

75. CDN στρατηγικές

Enter deep

Πολλοί servers μέσα σε πολλά access networks.

Πιο κοντά στους χρήστες.

Παράδειγμα:

Auto ▾



Akamai

Bring home

Λιγότερα αλλά μεγάλα clusters κοντά σε IXPs.

Παράδειγμα:

Auto ▾



Limelight

76. CDN και streaming

Το CDN αποθηκεύει αντίγραφα video.

Ο χρήστης παίρνει manifest και κατεβάζει chunks από CDN server.

Αν υπάρχει συμφόρηση, μπορεί να αλλάξει:

- ποιότητα
- server
- διαδρομή

77. OTT

OTT = Over The Top.

Σημαίνει υπηρεσία πάνω από IP υποδομή ISPs.

Προκλήσεις:

- ποιο περιεχόμενο να μπει σε ποιο CDN node
- από ποιο CDN node να εξυπηρετηθεί ο χρήστης
- με τι ρυθμό

78. Παράδειγμα Netflix

18219. Ο χρήστης διαχειρίζεται λογαριασμό σε Netflix servers.

18220. Περιηγείται στα διαθέσιμα videos.

18221. Παίρνει manifest για το video.

18222. Το video παραδίδεται μέσω DASH από CDN servers.

18223. Πολλαπλές εκδόσεις του video είναι αποθηκευμένες στο CDN.

79. SOS τύποι

Non-persistent HTTP

Auto ▾



```
response time = 2RTT + transmission time
```

Cache utilization

Auto ▾



```
utilization = traffic rate / link rate
```

Cache hit example

Auto ▾



```
traffic through access link = miss rate * total traffic
```

Client-server distribution

Auto ▾



$Dc-s \geq \max \{ NF/us , F/dmin \}$

P2P distribution

Auto ▾



$DP2P \geq \max \{ F/us , F/dmin , NF/(us + \sum u_i) \}$

80. SOS ορισμοί

Auto ▾



Socket = πόρτα της εφαρμογής προς το δίκτυο.

Auto ▾



IP + port = ταυτοποίηση διεργασίας.

Auto ▾



Client-server = μόνιμος server, clients ζητούν υπηρεσίες.

Auto ▾



P2P = peers ζητούν και προσφέρουν υπηρεσίες μεταξύ τους.

Auto ▾



TCP = αξιόπιστο, connection-oriented.

Auto ▾



UDP = αναξιόπιστο, απλό, χωρίς σύνδεση.

Auto ▾



HTTP = πρωτόκολλο Web, stateless, πάνω από TCP.

Auto ▾



Cookie = τρόπος διατήρησης κατάστασης στο HTTP.

Auto ▾



Web cache = proxy που αποθηκεύει αντικείμενα.

Auto ▾



Conditional GET = έλεγχος αν το cached object είναι ενημερωμένο.

Auto ▾



SMTP = αποστολή email μεταξύ mail servers.

Auto ▾



IMAP/POP = ανάκτηση email από server.

Auto ▾



DNS = μετάφραση hostname σε IP.

Auto ▾



BitTorrent = P2P διανομή αρχείων με chunks.

Auto ▾



DASH = adaptive streaming over HTTP.

Auto ▾



CDN = κατανεμημένο δίκτυο servers για γρήγορη παράδοση περιεχομένου.

81. Μικρό μνημονικό

Auto (Java) ▾



HTTP = Web

SMTP = στέλνω mail

IMAP/POP = διαβάζω mail

DNS = βρίσκω IP

DASH = βλέπω video προσαρμοστικά

CDN = κοντινός server για περιεχόμενο

BitTorrent = κατεβάζω και ανεβάζω chunks

82. Τελική σύνοψη

Στο επίπεδο εφαρμογής:

- οι εφαρμογές τρέχουν στα άκρα
- επικοινωνούν μέσω sockets
- χρησιμοποιούν TCP ή UDP
- το HTTP εξυπηρετεί το Web
- τα cookies δίνουν state στο stateless HTTP
- οι caches μειώνουν καθυστέρηση
- το SMTP στέλνει email
- το IMAP/POP ανακτά email
- το DNS μετατρέπει ονόματα σε IP
- το P2P κλιμακώνει καλύτερα από client-server
- το BitTorrent χρησιμοποιεί chunks και tit-for-tat
- το video streaming χρειάζεται buffering
- το DASH προσαρμόζει ποιότητα
- τα CDNs φέρνουν περιεχόμενο κοντά στον χρήστ

Δίκτυα Επικοινωνιών I — Επίπεδο Μεταφοράς

1. Τι κάνει το επίπεδο μεταφοράς

Το **επίπεδο μεταφοράς** βρίσκεται πάνω από το επίπεδο δικτύου και κάτω από το επίπεδο εφαρμογής.

Η βασική του δουλειά είναι:

Να προσφέρει λογική επικοινωνία μεταξύ διεργασιών εφαρμογών που τρέχουν σε διαφορετικούς υπολογιστές.

Δηλαδή:

Επίπεδο	Επικοινωνία μεταξύ
Δίκτυο / IP	υπολογιστών
Μεταφορά / TCP-UDP	διεργασιών / εφαρμογών

Παράδειγμα:

Έχεις έναν υπολογιστή που τρέχει:

- Chrome
- Netflix

- Skype
- Discord

Όταν έρχεται ένα πακέτο από το δίκτυο, το επίπεδο μεταφοράς πρέπει να ξέρει **σε ποια εφαρμογή/socket να το παραδώσει**.

2. Αναλογία με νοικοκυριό

Η αναλογία των διαφανειών:

Δίκτυο	Αναλογία
Υπολογιστές	σπίτια
Διεργασίες	παιδιά μέσα στα σπίτια
Μηνύματα εφαρμογής	γράμματα
Επίπεδο δικτύου	ταχυδρομείο
Επίπεδο μεταφοράς	η Άννα και ο Βασίλης που μοιράζουν τα γράμματα στα σωστά παιδιά

Άρα:

Το IP φέρνει το μήνυμα **στο σωστό σπίτι**.

Το TCP/UDP το παραδίδει **στο σωστό παιδί**, δηλαδή στη σωστή διεργασία.

3. Τι κάνει ο αποστολέας και ο παραλήπτης

Στον αποστολέα

Το επίπεδο μεταφοράς:

1278. Παίρνει μήνυμα από την εφαρμογή.
1279. Το σπάει, αν χρειάζεται, σε τμήματα.
1280. Βάζει κεφαλίδα μεταφοράς.
1281. Δημιουργεί segment.
1282. Το περνάει στο IP.

Μορφή:

Auto (CSS) ▾

Application message



↓
Transport **header** + Application message
↓
Segment

Στον παραλήπτη

Το επίπεδο μεταφοράς:

- 1585. Παίρνει segment από το IP.
- 1586. Ελέγχει την κεφαλίδα.
- 1587. Βγάζει το μήνυμα εφαρμογής.
- 1588. Το αποπολυπλέκει.
- 1589. Το παραδίδει στο σωστό socket.

4. Τα δύο βασικά πρωτόκολλα μεταφοράς

Υπάρχουν δύο βασικά πρωτόκολλα στο Internet:

Πρωτόκολλο	Χαρακτηριστικά
TCP	αξιόπιστο, συνδεσμικό, με σειρά, flow control, congestion control
UDP	απλό, ασυνδεσμικό, best effort, χωρίς εγγύηση

TCP

Το TCP προσφέρει:

- αξιόπιστη μεταφορά
- παράδοση με σωστή σειρά
- έλεγχο ροής
- έλεγχο συμφόρησης
- εγκαθίδρυση σύνδεσης
- πλήρως αμφίδρομη επικοινωνία

Χρησιμοποιείται π.χ. σε:

- HTTP/1.1
 - HTTP/2
 - email
 - file transfer
 - SSH
-

UDP

Το UDP προσφέρει:

- απλή αποστολή datagrams
- χωρίς σύνδεση
- χωρίς εγγύηση παράδοσης
- χωρίς εγγύηση σειράς
- χωρίς congestion control
- μικρή κεφαλίδα

Χρησιμοποιείται π.χ. σε:

- DNS
 - SNMP
 - streaming
 - VoIP
 - HTTP/3 μέσω QUIC
-

5. Πολύπλεξη και αποπολύπλεξη

Πολύπλεξη

Πολύπλεξη γίνεται στον αποστολέα.

Σημαίνει:

Το επίπεδο μεταφοράς παίρνει δεδομένα από πολλές εφαρμογές/sockets και τα στέλνει προς το δίκτυο, βάζοντας κατάλληλες κεφαλίδες.

Παράδειγμα:

Auto ▾



Chrome \
Netflix → Transport → IP
Skype /

Αποπολύπλεξη

Αποπολύπλεξη γίνεται στον παραλήπτη.

Σημαίνει:

Το επίπεδο μεταφοράς κοιτάει την κεφαλίδα του segment και αποφασίζει σε ποιο socket θα παραδώσει τα δεδομένα.

Παράδειγμα:

Auto ▾



IP → Transport → Chrome ή Netflix ή Skype

Πεδία που χρησιμοποιούνται

Κάθε segment έχει:

Auto ▾



source port
destination port

Επίσης, το IP datagram έχει:

Auto ▾



source IP
destination IP

Άρα το επίπεδο μεταφοράς μπορεί να χρησιμοποιήσει:

- source IP
- destination IP
- source port
- destination port

6. UDP αποπολύπλεξη

Στο UDP, η socket αναγνωρίζεται κυρίως από τη:

Auto ▾



destination port

Αν δύο UDP datagrams έχουν ίδια destination port, πάνε στην ίδια socket, ακόμα και αν έχουν διαφορετικό source IP ή source port.

Παράδειγμα:

Auto ▾



```
DatagramSocket serverSocket = new DatagramSocket(6428);
```

Αυτό σημαίνει:

Όσα UDP segments φτάσουν με destination port 6428 θα πάνε σε αυτή τη socket.

Source port στο UDP

Το source port λειτουργεί σαν:

Auto ▾



διεύθυνση επιστροφής

Δηλαδή ο παραλήπτης ξέρει σε ποια θύρα να απαντήσει.

7. TCP αποπολύπλεξη

Στο TCP, η socket αναγνωρίζεται από τετράδα:

Auto ▾



```
source IP  
source port  
destination IP  
destination port
```

Άρα δύο πελάτες μπορούν να συνδεθούν στον ίδιο server, στην ίδια θύρα 80, αλλά να έχουν διαφορετικές TCP sockets.

Παράδειγμα:

Auto (CSS) ▾



```
Client A: A:9157 → B:80  
Client C: C:5775 → B:80  
Client C: C:9157 → B:80
```

Όλα πάνε στον server B στη θύρα 80, αλλά είναι διαφορετικές TCP συνδέσεις.

8. Σύγκριση UDP και TCP στην αποπολύπλεξη

Χαρακτηριστικό	UDP	TCP
Τύπος	ασυνδεσμικό	συνδεσμικό
Socket ID	destination port	source IP, source port, dest IP, dest port
Πολλοί clients στην ίδια θύρα server	ίδια socket	διαφορετικές sockets
Χρήση	απλή αποστολή datagrams	σύνδεση client-server

9. UDP

Τι είναι το UDP

UDP = **U**ser **D**atagram **P**rotocol

Είναι το απλούστερο πρωτόκολλο μεταφοράς του Internet.

Παρέχει υπηρεσία:

Auto ▾



best effort

Δηλαδή:

Το UDP προσπαθεί να στείλει το segment, αλλά δεν εγγυάται τίποτα.

Τι μπορεί να συμβεί σε UDP segments

Τα UDP segments μπορεί:

- να χαθούν

- να φτάσουν εκτός σειράς
- να φτάσουν αλλοιωμένα
- να φτάσουν κανονικά

Το UDP δεν κάνει αναμετάδοση.

Γιατί υπάρχει το UDP;

Παρότι δεν είναι αξιόπιστο, έχει πλεονεκτήματα:

- 5175. Δεν έχει χειραψία.
 - 5176. Δεν έχει καθυστέρηση εγκαθίδρυσης σύνδεσης.
 - 5177. Είναι απλό.
 - 5178. Δεν κρατάει κατάσταση σύνδεσης.
 - 5179. Έχει μικρή κεφαλίδα.
 - 5180. Δεν έχει congestion control.
 - 5181. Μπορεί να στέλνει πολύ γρήγορα.
 - 5182. Είναι καλό για εφαρμογές που αντέχουν απώλειες.
-

Πού χρησιμοποιείται το UDP;

Χρησιμοποιείται σε:

- streaming
 - real-time audio/video
 - DNS
 - SNMP
 - HTTP/3 μέσω QUIC
-

Αν θέλουμε αξιοπιστία πάνω από UDP;

Τότε η αξιοπιστία πρέπει να μπει στο επίπεδο εφαρμογής.

Παράδειγμα:

Auto ▾



HTTP/3 = QUIC πάνω από UDP

Το QUIC προσθέτει:

- αξιοπιστία
- congestion control
- σύνδεση
- ασφάλεια

πάνω από UDP.

10. UDP segment header

Η κεφαλίδα UDP έχει 4 βασικά πεδία:

Auto ▾



```
source port
destination port
length
checksum
```

Μορφή:

Auto (Bash) ▾



```
source port # / dest port #
length      | checksum
application data
```

UDP Length

Το πεδίο length δείχνει το συνολικό μήκος του UDP segment σε bytes:

Auto ▾



```
UDP header + data
```

UDP Checksum

Το checksum χρησιμοποιείται για:

Auto ▾



```
ανίχνευση σφαλμάτων
```

Δηλαδή αν έχουν αναστραφεί bits κατά τη μετάδοση.

11. Checksum UDP

Σκοπός

Το checksum ανιχνεύει αν το segment αλλοιώθηκε.

Δεν διορθώνει το λάθος.

Απλά λέει:

Auto ▾



Υπάρχει πιθανό σφάλμα ή όχι;

Πώς δουλεύει γενικά

Ο αποστολέας:

6495. Χωρίζει το segment σε λέξεις των 16 bits.

6496. Τις προσθέτει.

6497. Κάνει συμπλήρωμα ως προς 1.

6498. Βάζει το αποτέλεσμα στο checksum field.

Ο δέκτης:

6644. Ξαναυπολογίζει το checksum.

6645. Το συγκρίνει με αυτό που έλαβε.

6646. Αν δεν ταιριάζει, υπάρχει σφάλμα.

Σημαντικό

Το checksum δεν πιάνει όλα τα λάθη.

Μπορεί κάποια bits να αλλάξουν, αλλά το checksum να μείνει ίδιο.

Άρα:

Auto ▾



Checksum σωστό ≠ 100% σίγουρα χωρίς σφάλμα

12. Σύνοψη UDP

Το UDP:

- είναι απλό
 - είναι ασυνδεσμικό
 - δεν κάνει handshake
 - δεν εγγυάται παράδοση
 - δεν εγγυάται σειρά
 - δεν κάνει retransmission
 - δεν έχει congestion control
 - έχει checksum
 - χρησιμοποιείται όταν θέλουμε ταχύτητα ή μικρή καθυστέρηση
-

13. Αξιόπιστη μεταφορά δεδομένων

Το πρόβλημα:

Το επίπεδο δικτύου είναι αναξιόπιστο.

Μπορεί:

- να χάσει πακέτα
- να αλλοιώσει bits
- να παραδώσει πακέτα εκτός σειράς

Αρα το επίπεδο μεταφοράς πρέπει να δημιουργήσει μια αξιόπιστη υπηρεσία πάνω από ένα αναξιόπιστο κανάλι.

Reliable Data Transfer - RDT

Το RDT είναι θεωρητικό μοντέλο για να καταλάβουμε πώς χτίζεται η αξιοπιστία.

Χρησιμοποιεί:

- checksum
 - ACK
 - NAK
 - sequence numbers
 - timers
 - retransmissions
-

Βασικές συναρτήσεις RDT

Συνάρτηση	Ρόλος
rdt_send()	καλείται από την εφαρμογή στον

	αποστολέα
udt_send()	στέλνει πακέτο στο αναξιόπιστο κανάλι
rdt_rcv()	καλείται όταν φτάνει πακέτο
deliver_data()	παραδίδει δεδομένα στην εφαρμογή του δέκτη

14. FSM

Τα πρωτόκολλα RDT περιγράφονται με:

Auto ▾



Finite State Machines

Δηλαδή μηχανές πεπερασμένων καταστάσεων.

Κάθε FSM έχει:

- καταστάσεις
- γεγονότα
- ενέργειες
- μεταβάσεις

Μορφή:

Auto ▾



event / action

Παράδειγμα:

Auto ▾



λήψη ACK / στείλε επόμενο πακέτο

15. rdt1.0

Υπόθεση

Το κανάλι είναι τέλειο:

- δεν χάνει πακέτα
- δεν αλλοιώνει bits
- δεν αλλάζει σειρά

Άρα το rdt1.0 είναι πολύ απλό.

Ο αποστολέας στέλνει.

Ο δέκτης λαμβάνει και παραδίδει.

Δεν χρειάζονται:

- ACK
 - NAK
 - checksum
 - timers
 - retransmissions
-

16. rdt2.0

Υπόθεση

Το κανάλι μπορεί να αλλοιώσει bits, αλλά δεν χάνει πακέτα.

Άρα χρειαζόμαστε:

- checksum
 - ACK
 - NAK
 - retransmission
-

ACK

ACK = θετική επιβεβαίωση.

Σημαίνει:

Auto ▾



Το πακέτο ελήφθη σωστά.

NAK

NAK = αρνητική επιβεβαίωση.

Σημαίνει:

Auto ▾



Το πακέτο είχε σφάλμα, στείλ' το ξανά.

Stop-and-wait

Το rdt2.0 είναι stop-and-wait.

Δηλαδή:

8943. Ο αποστολέας στέλνει ένα πακέτο.

8944. Περιμένει ACK ή NAK.

8945. Μετά στέλνει το επόμενο.

Πρόβλημα rdt2.0

Το rdt2.0 έχει μοιραίο πρόβλημα:

Auto ▾



Τι γίνεται αν αλλοιωθεί το ACK ή το NAK;

Ο αποστολέας δεν ξέρει αν:

- το πακέτο έφτασε σωστά
- το πακέτο είχε λάθος
- το ACK/NAK αλλοιώθηκε

Αν ξαναστείλει, μπορεί να δημιουργήσει duplicate packet.

17. rdt2.1

Το rdt2.1 λύνει το πρόβλημα των αλλοιωμένων ACK/NAK.

Προσθέτει:

Auto ▾



sequence numbers

Δηλαδή αριθμούς ακολουθίας.

Χρησιμοποιεί μόνο δύο sequence numbers:

Auto ▾



0 και 1

Γιατί αρκούν;

Επειδή είναι stop-and-wait.

Ο αποστολέας έχει κάθε φορά μόνο ένα πακέτο σε εκκρεμότητα.

Άρα χρειάζεται να ξεχωρίζει μόνο:

Auto ▾



τρέχον πακέτο ή διπλό προηγούμενο

Τι κάνει ο δέκτης στο rdt2.1

Ο δέκτης:

- ελέγχει αν το πακέτο είναι corrupt
- ελέγχει sequence number
- αν είναι σωστό και αναμενόμενο, το παραδίδει
- αν είναι duplicate, το απορρίπτει
- στέλνει ACK ή NAK

18. rdt2.2

Το rdt2.2 καταργεί το NAK.

Χρησιμοποιεί μόνο ACKs.

Ιδέα:

Αν ο δέκτης λάβει λάθος ή διπλό πακέτο, ξαναστέλνει ACK για το τελευταίο σωστό πακέτο.

Άρα duplicate ACK λειτουργεί σαν NAK.

Αυτή είναι σημαντική ιδέα γιατί:

Auto ▾



Το TCP χρησιμοποιεί ACK-only προσέγγιση.

19. rdt3.0

Υπόθεση

Το κανάλι μπορεί:

- να αλλοιώσει bits
- να χάσει πακέτα
- να χάσει ACKs

Άρα χρειαζόμαστε:

- checksum
 - sequence numbers
 - ACKs
 - retransmissions
 - timer
 - timeout
-

Timeout

Ο αποστολέας περιμένει ACK για κάποιο χρονικό διάστημα.

Αν δεν έρθει ACK:

Auto ▾



timeout → retransmission

Δηλαδή ξαναστέλνει το πακέτο.

Αν το πακέτο δεν χάθηκε αλλά απλώς άργησε;

Τότε μπορεί να δημιουργηθεί duplicate.

Όμως τα sequence numbers βοηθούν τον δέκτη να το αναγνωρίσει και να μην το παραδώσει δεύτερη φορά.

20. Απόδοση rdt3.0 stop-and-wait

Το stop-and-wait έχει πολύ μικρή απόδοση.

Παράδειγμα:

- ζεύξη 1 Gbps
- RTT = 30 ms περίπου
- πακέτο = 8000 bits

Χρόνος μετάδοσης:

Auto ▾



$$D_{trans} = L / R$$

Auto ▾



$$D_{trans} = 8000 / 10^9 = 8 \mu s$$

Βαθμός χρήσης:

Auto ▾



$$U_{sender} = (L/R) / (RTT + L/R)$$

Με τα νούμερα των διαφανειών:

Auto ▾



$$U_{sender} = 0.008 / 30.008 = 0.00027$$

Δηλαδή ο αποστολέας στέλνει ελάχιστο χρόνο και μετά περιμένει.

21. Pipelining

Για να αυξηθεί η απόδοση, χρησιμοποιούμε:

Auto ▾



pipelining

Δηλαδή ο αποστολέας μπορεί να έχει πολλά πακέτα "in flight", χωρίς να περιμένει ACK για κάθε ένα ξεχωριστά.

Χρειάζονται:

- μεγαλύτερο εύρος sequence numbers
 - buffering στον αποστολέα
 - buffering στον δέκτη
 - window
-

22. Go-Back-N

Στο Go-Back-N ο αποστολέας έχει παράθυρο μεγέθους N.

Μπορεί να στείλει μέχρι N μη επιβεβαιωμένα πακέτα.

Χαρακτηριστικά Go-Back-N

Ο αποστολέας:

- έχει window έως N πακέτα
 - χρησιμοποιεί cumulative ACKs
 - έχει timer για το παλαιότερο unACKed πακέτο
 - αν γίνει timeout, ξαναστέλνει το χαμένο και όλα τα επόμενα στο παράθυρο
-

Cumulative ACK

ACK(n) σημαίνει:

Auto ▾



Έχω λάβει σωστά όλα τα πακέτα μέχρι και το n.

Άρα αν λάβω ACK(5), σημαίνει ότι έχουν επιβεβαιωθεί:

Auto ▾



0, 1, 2, 3, 4, 5

Δέκτης Go-Back-N

Ο δέκτης:

- δέχεται μόνο πακέτα στη σωστή σειρά
- αν έρθει out-of-order πακέτο, το απορρίπτει ή το ενταμιεύει ανάλογα με την υλοποίηση
- στέλνει ACK για το μεγαλύτερο in-order πακέτο

Συνήθως για τη θεωρία:

Auto ▾



out-of-order → discard

Παράδειγμα Go-Back-N

Αν χαθεί το pkt2 και έχουν σταλεί pkt3, pkt4, pkt5:

Ο δέκτης απορρίπτει pkt3, pkt4, pkt5 γιατί λείπει το pkt2.

Με timeout στο pkt2, ο αποστολέας ξαναστέλνει:

Auto ▾



pkt2, pkt3, pkt4, pkt5

Γι' αυτό λέγεται:

Auto ▾



Go-Back-N

23. Selective Repeat

Στο Selective Repeat ο δέκτης επιβεβαιώνει κάθε σωστό πακέτο ξεχωριστά.

Ο αποστολέας ξαναστέλνει μόνο όσα δεν επιβεβαιώθηκαν.

Χαρακτηριστικά Selective Repeat

Ο δέκτης:

- δέχεται out-of-order πακέτα
- τα βάζει σε buffer
- τα παραδίδει στην εφαρμογή όταν συμπληρωθούν τα κενά

Ο αποστολέας:

- έχει timer για κάθε unACKed πακέτο
- κάνει retransmission μόνο για το συγκεκριμένο χαμένο πακέτο

Παράδειγμα

Αν χαθεί το pkt2 αλλά φτάσουν pkt3, pkt4, pkt5:

Ο δέκτης:

- κρατάει pkt3, pkt4, pkt5 σε buffer
- στέλνει ACK3, ACK4, ACK5
- όταν φτάσει pkt2, παραδίδει:

Auto ▾



pkt2, pkt3, pkt4, pkt5

Δίλημμα Selective Repeat

Πρέπει το window size να μην είναι πολύ μεγάλο σε σχέση με το πλήθος sequence numbers.

Αλλιώς ο δέκτης μπορεί να μπερδέψει ένα παλιό retransmitted πακέτο με νέο πακέτο που έχει ίδιο sequence number.

Κανόνας:

Auto ▾



Window size $\leq$ μισό του εύρους των sequence numbers

Αν έχουμε k bits για sequence number:

Auto ▾



Sequence numbers = 2^k

Άρα:

Auto ▾



$N \leq 2^{(k-1)}$

24. TCP επισκόπηση

Το TCP είναι:

- συνδεδεσμένο
- αξιόπιστο
- byte-stream
- full duplex
- point-to-point
- με cumulative ACKs
- με pipelining
- με flow control
- με congestion control

TCP είναι byte-stream

Το TCP δεν βλέπει “μηνύματα”.

Βλέπει ροή από bytes.

Άρα:

Auto ▾



```
sequence number = αριθμός byte
```

και όχι αριθμός segment.

MSS

MSS = Maximum Segment Size.

Συνήθως:

Auto ▾



```
MSS = 1460 bytes
```

Είναι το μέγιστο payload TCP segment, όχι όλο το IP packet.

25. TCP segment header

Βασικά πεδία TCP κεφαλίδας:

Auto (TypeScript) ▾



```
source port  
destination port  
sequence number
```


acknowledgement **number**
header length
receive **window**
checksum
flags
urgent pointer
options
data

Σημαντικά TCP flags

Flag	Ρόλος
ACK	το segment περιέχει έγκυρο ACK
SYN	έναρξη σύνδεσης
FIN	κλείσιμο σύνδεσης
RST	reset σύνδεσης
PSH	push data
URG	urgent data
C/E	congestion notification

26. TCP sequence numbers και ACKs

Sequence number

Στο TCP:

Auto ▾



Seq = αριθμός του πρώτου byte δεδομένων στο segment

Παράδειγμα:

Αν στείλω segment με:

Auto ▾



Seq = 92

data = 8 bytes

τότε τα bytes είναι:

Auto ▾



92 έως 99

Το επόμενο αναμενόμενο byte είναι:

Auto ▾



100

Άρα ACK:

Auto ▾



ACK = 100

ACK number

Το ACK δείχνει:

Auto ▾



το επόμενο byte που περιμένω

Άρα:

Auto ▾



ACK = 100

σημαίνει:

Auto ▾



Έλαβα όλα τα bytes μέχρι 99.
Περιμένω το byte 100.

Cumulative ACK

Το TCP χρησιμοποιεί cumulative ACKs.

Δηλαδή ένα ACK μπορεί να επιβεβαιώνει πολλά προηγούμενα bytes μαζί.

27. Παράδειγμα Telnet

Αν ο Host A στείλει τον χαρακτήρα C:

Auto ▾



Host A → Host B:
Seq=42, ACK=79, data='C'

Το C είναι 1 byte.

Άρα ο Host B απαντά:

Auto ▾



Seq=79, ACK=43, data='C'

Γιατί ACK=43;

Επειδή έλαβε το byte 42 και τώρα περιμένει το 43.

Μετά ο Host A στέλνει:

Auto ▾



Seq=43, ACK=80

28. RTT και Timeout στο TCP

SampleRTT

SampleRTT είναι ο χρόνος από τότε που στέλνω ένα segment μέχρι να πάρω ACK.

Συνήθως αγνοούμε retransmitted segments, γιατί δεν ξέρουμε αν το ACK αφορά το αρχικό ή το retransmitted.

EstimatedRTT

Το TCP υπολογίζει ομαλοποιημένο RTT:

Auto ▾



$$\text{EstimatedRTT} = (1 - \alpha) \text{ EstimatedRTT} + \alpha \text{ SampleRTT}$$

Τυπικά:

Auto ▾



$$\alpha = 0.125$$

Αυτό είναι EWMA:

Auto ▾



Exponentially Weighted Moving Average

Δηλαδή τα παλιά δείγματα επηρεάζουν όλο και λιγότερο.

DevRTT

Μετράει την απόκλιση του SampleRTT από το EstimatedRTT.

Auto ▾



$$\text{DevRTT} = (1 - \beta) \text{ DevRTT} + \beta |\text{SampleRTT} - \text{EstimatedRTT}|$$

Τυπικά:

Auto ▾



$$\beta = 0.25$$

TimeoutInterval

Auto ▾


$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \text{ DevRTT}$$

Αν το timeout είναι πολύ μικρό:

- έχουμε πρόωρα timeouts
- άχρηστες αναμεταδόσεις

Αν είναι πολύ μεγάλο:

- αργούμε να αντιδράσουμε σε απώλειες

29. TCP αποστολέας

Ο TCP αποστολέας αντιδρά σε 3 βασικά γεγονότα.

1. Λήψη δεδομένων από εφαρμογή

Ο αποστολέας:

16633. Δημιουργεί segment.

16634. Βάζει sequence number.

16635. Αν δεν τρέχει timer, ξεκινά timer.

16636. Στέλνει segment.

2. Timeout

Αν λήξει ο χρόνος:

16781. Ξαναστέλνει το παλαιότερο μη επιβεβαιωμένο segment.

16782. Ξεκινά ξανά timer.

3. Λήψη ACK

Αν το ACK επιβεβαιώνει νέα δεδομένα:

16918. Ενημερώνει το SendBase.

16919. Αν υπάρχουν ακόμα μη επιβεβαιωμένα, συνεχίζει timer.

16920. Αλλιώς σταματά timer.

30. TCP δέκτης και ACK generation

Ο TCP δέκτης έχει κανόνες για ACK.

Περίπτωση 1

Φτάνει segment στη σωστή σειρά και δεν περιμένει άλλο ACK.

Ενέργεια:

Auto ▾



Delayed ACK

Περιμένει μέχρι 500 ms μήπως έρθει και επόμενο segment.

Περίπτωση 2

Φτάνει segment στη σωστή σειρά και υπάρχει ήδη ACK που περιμένει να σταλεί.

Ενέργεια:

Auto ▾



Στέλνει άμεσα cumulative ACK

Περίπτωση 3

Φτάνει segment εκτός σειράς, άρα υπάρχει κενό.

Ενέργεια:

Auto ▾



Στέλνει duplicate ACK

Το duplicate ACK δηλώνει το επόμενο byte που περιμένει.

Περίπτωση 4

Φτάνει segment που συμπληρώνει κενό.

Ενέργεια:

Auto ▾



Στέλνει άμεσα ACK

31. TCP retransmission scenarios

Lost ACK

Αν χαθεί ACK, μπορεί να γίνει timeout και retransmission.

Όμως αν αργότερα έρθει cumulative ACK, μπορεί να καλύψει και το χαμένο ACK.

Premature timeout

Αν το timeout γίνει πολύ νωρίς, το TCP μπορεί να ξαναστείλει segment που δεν είχε χαθεί.

Αυτό δημιουργεί duplicate.

Cumulative ACK covers lost ACK

Αν χαθεί ACK=100 αλλά μετά έρθει ACK=120, τότε το ACK=120 επιβεβαιώνει και τα προηγούμενα.

Άρα το χαμένο ACK δεν είναι πάντα πρόβλημα.

32. TCP Fast Retransmit

Το TCP δεν περιμένει πάντα timeout.

Αν λάβει:

Auto ▾



3 duplicate ACKs

τότε υποθέτει ότι κάποιο segment χάθηκε.

Άρα κάνει:

Auto ▾



fast retransmit

Ξαναστέλνει το παλαιότερο μη επιβεβαιωμένο segment.

33. TCP Flow Control

Το flow control προστατεύει τον δέκτη.

Πρόβλημα:

Ο αποστολέας μπορεί να στέλνει πιο γρήγορα από όσο η εφαρμογή του δέκτη διαβάζει από το socket buffer.

Άρα ο buffer του δέκτη μπορεί να γεμίσει.

Receive window - rwnd

Ο δέκτης ενημερώνει τον αποστολέα πόσο χώρο έχει ελεύθερο στον buffer του.

Αυτό γίνεται μέσω του πεδίου:

Auto ▾



rwnd

στην TCP κεφαλίδα.

Κανόνας flow control

Ο αποστολέας περιορίζει τα in-flight δεδομένα ώστε:

Auto ▾



$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$

Έτσι δεν υπερχειλίζει ο buffer του δέκτη.

RcvBuffer

Ο δέκτης έχει receive buffer.

Το rwnd είναι περίπου:

Auto ▾



$\text{rwnd} = \text{RcvBuffer} - \text{buffered data}$

Δηλαδή ελεύθερος χώρος.

34. Flow control vs congestion control

Πολύ SOS διαφορά:

Έλεγχος	Πρόβλημα	Ποιον προστατεύει
Flow control	ο αποστολέας στέλνει πολύ γρήγορα για τον δέκτη	τον δέκτη
Congestion control	πολλοί αποστολείς στέλνουν πολύ γρήγορα για το δίκτυο	το δίκτυο

35. TCP connection management

Πριν σταλούν δεδομένα, το TCP κάνει handshake.

Σκοπός:

- να συμφωνήσουν ότι θέλουν σύνδεση
- να ανταλλάξουν αρχικά sequence numbers
- να αρχικοποιήσουν buffers και μεταβλητές
- να ξέρει κάθε πλευρά ότι η άλλη είναι ζωντανή

36. Γιατί όχι 2-way handshake;

Η διμερής χειραψία μπορεί να έχει προβλήματα λόγω:

- μεταβλητών καθυστερήσεων
- παλιών retransmitted μηνυμάτων
- αναδιάταξης μηνυμάτων
- half-open connections
- duplicate data

Άρα χρειάζεται 3-way handshake.

37. TCP 3-way handshake

Βήματα:

1ο βήμα

Client → Server:

Auto ▾



SYN = 1

Seq = x

Ο client διαλέγει αρχικό sequence number x.

2ο βήμα

Server → Client:

Auto ▾



SYN = 1

ACK = 1

Seq = y

ACKnum = x + 1

Ο server διαλέγει δικό του αρχικό sequence number y και επιβεβαιώνει το SYN του client.

3ο βήμα

Client → Server:

Auto ▾



ACK = 1

ACKnum = y + 1

Ο client επιβεβαιώνει το SYN του server.

Μετά η σύνδεση είναι:

Auto ▾



ESTABLISHED

38. TCP connection closing

Κάθε πλευρά κλείνει τη σύνδεση από τη μεριά της.

Χρησιμοποιείται το flag:

Auto ▾



FIN = 1

Η άλλη πλευρά απαντά με ACK.

Μπορούν να υπάρξουν και ταυτόχρονα FINs.

39. Congestion control

Συμφόρηση σημαίνει:

Auto ▾



Πολλές πηγές στέλνουν πολλά δεδομένα πολύ γρήγορα για να τα χειριστεί το δίκτυο.

Συμπτώματα:

- μεγάλες καθυστερήσεις
- ουρές στους routers
- απώλειες πακέτων
- overflow buffers
- retransmissions

Κόστη συμφόρησης

Η συμφόρηση κοστίζει γιατί:

20990. Αυξάνει η καθυστέρηση.

20991. Χάνονται πακέτα.

20992. Γίνονται retransmissions.

20993. Μερικές retransmissions είναι άχρηστες.

20994. Σπαταλιέται bandwidth.

20995. Σπαταλιέται upstream χωρητικότητα για πακέτα που τελικά χάνονται downstream.

20996. Η ρυθμαπόδοση δεν μπορεί να ξεπεράσει τη χωρητικότητα.

40. Σενάρια συμφόρησης

Σενάριο 1

- δύο ροές
- ένας router
- άπειροι buffers
- χωρητικότητα R
- χωρίς retransmissions

Όταν το input πλησιάζει τη χωρητικότητα:

Auto ▾



η καθυστέρηση αυξάνεται πάρα πολύ

Σενάριο 2

- πεπερασμένοι buffers
- packets μπορούν να χαθούν
- retransmissions

Τότε:

Auto ▾



$\lambda'_{in} \geq \lambda_{in}$

Δηλαδή η πραγματική είσοδος στο δίκτυο περιλαμβάνει και retransmitted data.

Σενάριο 3

Πολλές ροές και πολλά hops.

Αν ένα πακέτο χαθεί downstream, τότε όλη η χωρητικότητα που χρησιμοποίησε upstream πηγή χαμένη.

41. Προσεγγίσεις congestion control

End-to-end congestion control

Δεν υπάρχει άμεση βοήθεια από το δίκτυο.

Ο αποστολέας συμπεραίνει συμφόρηση από:

- απώλειες

- καθυστερήσεις
- duplicate ACKs
- timeouts

Αυτή είναι η κλασική προσέγγιση του TCP.

Network-assisted congestion control

Το δίκτυο δίνει άμεση πληροφορία συμφόρησης.

Παραδείγματα:

- ECN
- ATM
- DECbit

Οι routers μπορούν να μαρκάρουν πακέτα για να δείξουν συμφόρηση.

42. TCP congestion control

Το κλασικό TCP είναι:

Auto ▾



loss-based, end-to-end

Δηλαδή θεωρεί ότι απώλεια πακέτου σημαίνει συμφόρηση.

AIMD

AIMD = Additive Increase, Multiplicative Decrease.

Δηλαδή:

- αυξάνει σταδιακά τον ρυθμό
 - όταν δει απώλεια, μειώνει απότομα
-

Additive Increase

Το TCP αυξάνει το congestion window περίπου κατά:

Auto ▾



1 MSS ανά RTT

στη φάση congestion avoidance.

Multiplicative Decrease

Όταν ανιχνευθεί απώλεια:

- με triple duplicate ACK: μειώνει το cwnd περίπου στο μισό
- με timeout: μειώνει πιο δραστικά, συνήθως σε 1 MSS

43. cwnd

cwnd = congestion window.

Είναι το παράθυρο συμφόρησης.

Ο αποστολέας περιορίζει τα in-flight δεδομένα:

Auto ▾


$$\text{LastByteSent} - \text{LastByteAcked} < \text{cwnd}$$

Προσεγγιστικός ρυθμός TCP:

Auto ▾


$$\text{TCP rate} \approx \text{cwnd} / \text{RTT}$$

44. Slow start

Όταν αρχίζει μια TCP σύνδεση:

Auto ▾


$$\text{cwnd} = 1 \text{ MSS}$$

Με κάθε ACK:

Auto ▾


$$\text{cwnd} += 1 \text{ MSS}$$

Άρα περίπου κάθε RTT το cwnd διπλασιάζεται.

Αυτό είναι εκθετική αύξηση.

Λέγεται slow start, αλλά στην πραγματικότητα ανεβαίνει γρήγορα εκθετικά.

45. ssthresh

ssthresh = slow start threshold.

Όταν γίνει απώλεια:

Auto ▾



$$\text{ssthresh} = \text{cwnd} / 2$$

Μετά:

- αν $\text{cwnd} < \text{ssthresh} \rightarrow \text{slow start}$
- αν $\text{cwnd} \geq \text{ssthresh} \rightarrow \text{congestion avoidance}$

46. Congestion avoidance

Σε congestion avoidance, το cwnd αυξάνεται γραμμικά.

Περίπου:

Auto ▾



$$+1 \text{ MSS ανά RTT}$$

Πιο ακριβώς σε κάθε ACK:

Auto ▾



$$\text{cwnd} = \text{cwnd} + \text{MSS} * (\text{MSS} / \text{cwnd})$$

47. Fast recovery

Μετά από 3 duplicate ACKs:

- γίνεται fast retransmit
- το cwnd μειώνεται
- δεν πέφτει απαραίτητα στο 1 MSS
- μπαίνει σε fast recovery

Σε timeout όμως:

Auto ▾



cwnd = 1 MSS

48. TCP Tahoe vs Reno

Περίπτωση	TCP Tahoe	TCP Reno
Timeout	cwnd = 1 MSS	cwnd = 1 MSS
3 duplicate ACKs	cwnd = 1 MSS	cwnd περίπου στο μισό
Fast recovery	όχι	ναι

49. TCP CUBIC

Το TCP CUBIC είναι πιο σύγχρονη παραλλαγή.

Χρησιμοποιείται πολύ στο Linux.

Ιδέα:

- Θυμάται το W_{max} , δηλαδή το παράθυρο πριν την απώλεια.
- Μετά την απώλεια μειώνει το παράθυρο.
- Αυξάνει γρήγορα όταν είναι μακριά από W_{max} .
- Αυξάνει αργά όταν πλησιάζει το W_{max} .
- Μετά δοκιμάζει ξανά πάνω από W_{max} .

Η αύξηση ακολουθεί κυβική συνάρτηση.

Γι' αυτό λέγεται:

Auto ▾



CUBIC

50. Bottleneck link

Bottleneck link είναι η ζεύξη που περιορίζει τη ρυθμαπόδοση.

Δηλαδή το πιο στενό σημείο του μονοπατιού.

Στόχος:

Auto ▾



Να κρατάμε τον αγωγό οριακά γεμάτο, αλλά όχι υπερβολικά γεμάτο.

Αν είναι υπερβολικά γεμάτος:

- αυξάνει RTT
- γεμίζουν ουρές
- έχουμε απώλειες

51. Delay-based congestion control

Αντί να περιμένει απώλειες, προσπαθεί να καταλάβει τη συμφόρηση από την καθυστέρηση.

Χρησιμοποιεί:

Auto ▾



RTT_{min}

Το RTT_{min} είναι το RTT όταν το μονοπάτι δεν έχει συμφόρηση.

Η θεωρητική μη συμφορημένη ρυθμαπόδοση είναι:

Auto ▾



$cwnd / RTT_{min}$

Αν η πραγματική ρυθμαπόδοση είναι κοντά σε αυτή:

Auto ▾



αύξησε $cwnd$

Αν είναι πολύ μικρότερη:

Auto ▾



μείωσε $cwnd$

Παράδειγμα τέτοιου πρωτοκόλλου:

Auto ▾



BBR

Αναπτύχθηκε από τη Google.

52. ECN

ECN = Explicit Congestion Notification.

Είναι network-assisted μηχανισμός.

Οι routers μαρκάρουν bits στην IP κεφαλίδα για να δηλώσουν συμφόρηση.

Ο δέκτης το βλέπει και βάζει ECE bit στο TCP ACK.

Έτσι ενημερώνεται ο αποστολέας ότι υπάρχει συμφόρηση χωρίς απαραίτητα να χαθεί πακέτο.

53. TCP fairness

Στόχος δικαιοσύνης:

Αν K TCP συνδέσεις μοιράζονται bottleneck link χωρητικότητας R , τότε κάθε σύνδεση ιδανικά παίρνει:

Auto ▾



R / K

Το TCP AIMD είναι δίκαιο υπό ιδανικές συνθήκες:

- ίδιες RTT
 - σταθερός αριθμός ροών
 - όλες σε congestion avoidance
-

TCP fairness και UDP

Το UDP δεν έχει congestion control.

Άρα εφαρμογές UDP μπορούν να στέλνουν σταθερό ρυθμό και να μη μειώνουν όταν υπάρχει συμφόρηση.

Δεν υπάρχει "αστυνομία του Internet" που να τις αναγκάζει να είναι δίκαιες.

Parallel TCP connections

Μια εφαρμογή μπορεί να ανοίξει πολλές TCP συνδέσεις.

Παράδειγμα:

Αν υπάρχουν ήδη 9 TCP συνδέσεις και μία νέα εφαρμογή ανοίξει:

- 1 TCP σύνδεση → παίρνει περίπου $1/10$ του R
- 11 TCP συνδέσεις → παίρνει περίπου $11/20$ του R

Άρα οι πολλαπλές TCP συνδέσεις μπορούν να δώσουν μη δίκαιο πλεονέκτημα.

54. Εξέλιξη επιπέδου μεταφοράς

Για περίπου 40 χρόνια τα βασικά πρωτόκολλα ήταν:

Auto ▾



TCP και UDP

Όμως αναπτύχθηκαν παραλλαγές TCP για ειδικές περιπτώσεις.

Προβλήματα ειδικών περιπτώσεων

Σενάριο	Πρόβλημα
Long fat pipes	πολλά πακέτα in-flight, οι απώλειες σταματούν την pipeline
Wireless networks	απώλειες από θόρυβο/κινητικότητα, όχι απαραίτητα συμφόρηση
Long-delay links	πολύ μεγάλο RTT
Data centers	μεγάλη ευαισθησία σε καθυστέρηση
Background traffic	ροές χαμηλής προτεραιότητας

QUIC και HTTP/3

Νέα τάση:

Auto ▾



Λειτουργίες μεταφοράς υλοποιούνται στο επίπεδο εφαρμογής πάνω από UDP.

Παράδειγμα:

Auto ▾



HTTP/3 χρησιμοποιεί QUIC πάνω από UDP.

55. Όλοι οι βασικοί τύποι SOS

Μετάδοση πακέτου

Auto ▾



$$D_{trans} = L / R$$

όπου:

- L = μέγεθος πακέτου σε bits
- R = ρυθμός ζεύξης σε bits/sec

Stop-and-wait utilization

Auto ▾



$$U_{sender} = (L/R) / (RTT + L/R)$$

Pipelining utilization

Για N πακέτα περίπου:

Auto ▾



$$U_{sender} = N(L/R) / (RTT + L/R)$$

μέχρι να φτάσει το 1.

TCP Estimated RTT

Auto ▾



$$\text{EstimatedRTT} = (1 - \alpha) \text{ EstimatedRTT} + \alpha \text{ SampleRTT}$$

Συνήθως:

Auto ▾



$$\alpha = 0.125$$

TCP DevRTT

Auto ▾



$$\text{DevRTT} = (1 - \beta) \text{ DevRTT} + \beta |\text{SampleRTT} - \text{EstimatedRTT}|$$

Συνήθως:

Auto ▾



$$\beta = 0.25$$

TCP Timeout

Auto ▾



$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \text{ DevRTT}$$

TCP sending rate

Auto ▾



$$\text{TCP rate} \approx \text{cwnd} / \text{RTT}$$

TCP in-flight condition

Για congestion control:

Auto ▾



$$\text{LastByteSent} - \text{LastByteAcked} < \text{cwnd}$$

Για flow control:

Auto ▾



$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

Στην πράξη το TCP περιορίζεται από:

Auto ▾



$$\min(\text{cwnd}, \text{rwnd})$$

Selective Repeat condition

Αν έχουμε k bits sequence number:

Auto ▾



$$\text{sequence number space} = 2^k$$

Για να μην υπάρχει ambiguity:

Auto ▾



$$\text{window size} \leq 2^{(k-1)}$$

56. Τα πιο SOS για εξέταση

Να ξέρεις πολύ καλά:

- 28518. Διαφορά TCP και UDP.
- 28519. Διαφορά flow control και congestion control.
- 28520. Τι είναι multiplexing/demultiplexing.
- 28521. UDP demux με destination port.
- 28522. TCP demux με τετράδα.
- 28523. UDP header.
- 28524. UDP checksum.
- 28525. rdt1.0, rdt2.0, rdt2.1, rdt2.2, rdt3.0.
- 28526. ACK, NAK, sequence number, timeout.
- 28527. Stop-and-wait και γιατί έχει μικρή απόδοση.
- 28528. Go-Back-N.
- 28529. Selective Repeat.
- 28530. Γιατί στο Selective Repeat θέλουμε $\text{window} \leq \text{μισό sequence space}$.
- 28531. TCP sequence numbers είναι bytes, όχι segments.
- 28532. TCP ACK δείχνει το επόμενο αναμενόμενο byte.
- 28533. EstimatedRTT, DevRTT, TimeoutInterval.
- 28534. Fast retransmit με 3 duplicate ACKs.
- 28535. TCP 3-way handshake.
- 28536. TCP flow control με rwnd.
- 28537. Congestion control με cwnd.
- 28538. AIMD.
- 28539. Slow start.
- 28540. Congestion avoidance.
- 28541. ssthresh.
- 28542. CUBIC.
- 28543. ECN.
- 28544. TCP fairness.
- 28545. QUIC/HTTP3 πάνω από UDP.

57. Μικρό τελικό μνημονικό

Auto ▾



UDP = γρήγορο, απλό, χωρίς εγγυήσεις.

TCP = αξιόπιστο, με σύνδεση, σειρά, ACKs, flow control, congestion control.

Auto (Java) ▾



Flow control = προστατεύω τον δέκτη.

Congestion control = προστατεύω το δίκτυο.

Auto ▾



rwnd = χώρος δέκτη.

cwnd = κατάσταση δικτύου.

Auto ▾



ACK στο TCP = επόμενο byte που περιμένω.

Auto (Java) ▾



3 **duplicate ACKs** = fast retransmit.

Timeout = πιο σοβαρή ένδειξη απώλειας.

Auto (Java) ▾



Slow start = εκθετική αύξηση.

Congestion avoidance = γραμμική αύξηση.

AIMD = αύξησε λίγο, κόψε πολύ.

Δίκτυα Επικοινωνιών I

Κεφάλαιο 4: Επίπεδο Δικτύου — Επίπεδο Δεδομένων

1. Τι κάνει το επίπεδο δικτύου

Το επίπεδο δικτύου έχει στόχο να μεταφέρει δεδομένα από τον υπολογιστή αποστολέα στον υπολογιστή παραλήπτη.

Στον αποστολέα:

Auto ▾



Transport segment → IP datagram → Link-layer frame

Στον παραλήπτη:

Auto ▾



IP datagram → Transport segment

Το επίπεδο δικτύου υπάρχει σε:

- υπολογιστές
- δρομολογητές
- συσκευές Internet

Ο δρομολογητής εξετάζει την κεφαλίδα IP και αποφασίζει από ποια έξοδο θα προωθήσει το πακέτο.

2. Δύο βασικές λειτουργίες

Προώθηση — Forwarding

Είναι τοπική λειτουργία μέσα σε έναν δρομολογητή.

Σημαίνει:

Auto ▾



Πακέτο μπαίνει από μία θύρα εισόδου → βγαίνει από κατάλληλη θύρα εξόδου

Παράδειγμα αναλογίας:

Auto ▾



Περνάω έναν κυκλικό κόμβο και διαλέγω έξοδο.

Δρομολόγηση — Routing

Είναι λειτουργία που αφορά όλο το μονοπάτι από πηγή σε προορισμό.

Σημαίνει:

Auto ▾



Υπολογίζω τη διαδρομή που θα ακολουθήσουν τα πακέτα.

Παράδειγμα αναλογίας:

Auto ▾



Σχεδιάζω όλο το ταξίδι από Αθήνα μέχρι Θεσσαλονίκη.

Διαφορά forwarding και routing

Έννοια	Τι κάνει	Πού γίνεται
Forwarding	διαλέγει έξοδο για ένα πακέτο	μέσα σε έναν router
Routing	υπολογίζει διαδρομές	σε επίπεδο δικτύου

3. Επίπεδο δεδομένων και επίπεδο ελέγχου

Επίπεδο δεδομένων — Data plane

Το data plane:

- είναι τοπικό ανά router
- λειτουργεί γρήγορα
- αποφασίζει πώς θα προωθηθεί κάθε εισερχόμενο datagram
- υλοποιείται κυρίως σε hardware
- λειτουργεί σε χρόνους nanoseconds

Παράδειγμα:

Auto ▾



Έρχεται πακέτο με destination IP → κοιτάω πίνακα forwarding → το στέλνω στην έξοδο 2

Επίπεδο ελέγχου — Control plane

To control plane:

- έχει λογική σε επίπεδο δικτύου
- αποφασίζει τις διαδρομές
- γεμίζει τους πίνακες forwarding
- λειτουργεί πιο αργά
- υλοποιείται κυρίως σε software
- λειτουργεί σε χρόνους milliseconds

4. Δύο προσεγγίσεις control plane

1. Παραδοσιακή ανά router προσέγγιση

Κάθε router τρέχει αλγόριθμο δρομολόγησης.

Οι routers επικοινωνούν μεταξύ τους και υπολογίζουν τους πίνακες.

Παράδειγμα:

Auto ▾



OSPF, BGP

2. SDN — Software Defined Networking

Υπάρχει ένας απομακρυσμένος controller.

Ο controller:

- έχει κεντρική εικόνα του δικτύου
- υπολογίζει κανόνες
- ενημερώνει routers/switches
- επικοινωνεί με control agents

Άρα:

Auto ▾



Data plane στους routers
Control plane στον controller

5. Μοντέλο υπηρεσιών δικτύου

Το επίπεδο δικτύου θα μπορούσε θεωρητικά να προσφέρει υπηρεσίες όπως:

Για μεμονωμένα datagrams:

- εγγυημένη παράδοση
- εγγυημένη παράδοση με καθυστέρηση κάτω από 40 ms

Για ροές datagrams:

- παράδοση με σειρά
- εγγυημένο bandwidth
- περιορισμένο jitter
- εγγύηση απωλειών

Το Internet τι προσφέρει;

Το Internet προσφέρει:

Auto ▾



Best effort service

Δηλαδή:

Δεν εγγυάται:

- επιτυχημένη παράδοση
- σειρά παράδοσης
- καθυστέρηση
- bandwidth
- μη απώλεια
- χρονισμό
- ανάδραση συμφόρησης

Με απλά λόγια:

Auto ▾



Το IP κάνει την καλύτερη προσπάθεια, αλλά δεν υπόσχεται τίποτα.

6. Άλλα μοντέλα υπηρεσιών

Αρχιτεκτονική	Μοντέλο	Bandwidth	Loss
Internet	Best effort	όχι	όχι
TM	CBR	σταθερός ρυθμός	ναι
TM	VBR	εγγυημένος ρυθμός	ναι
TM	ABR	ελάχιστος ρυθμός	όχι πάντα
TM	UBR	καμία εγγύηση	όχι
tserv	Guaranteed	ναι	ναι
ffserv	διαφοροποιημένη υπηρεσία	πιθανώς	πιθανώς

7. Γιατί πέτυχε το best effort;

Το best effort πέτυχε επειδή:

- είναι απλό
- κλιμακώνεται εύκολα
- επέτρεψε στο Internet να μεγαλώσει τεράστια
- υπάρχει αρκετό bandwidth πολλές φορές
- οι εφαρμογές προσαρμόζονται
- υπάρχουν CDNs και data centers κοντά στους χρήστες
- το TCP congestion control βοηθάει

Άρα, παρότι δεν έχει εγγυήσεις, είναι πρακτικά πολύ επιτυχημένο.

8. Τι υπάρχει μέσα σε έναν router

Ένας router έχει:

- 4169. Θύρες εισόδου
- 4170. Switching fabric
- 4171. Θύρες εξόδου
- 4172. Routing processor

Routing processor

O routing processor:

- ανήκει στο control plane
- κάνει δρομολόγηση
- ενημερώνει πίνακες
- λειτουργεί σε software
- λειτουργεί σε milliseconds

Switching fabric

To switching fabric:

- συνδέει τις εισόδους με τις εξόδους
- μεταφέρει πακέτα μέσα στον router
- πρέπει να είναι πολύ γρήγορο

Θύρες εισόδου

Κάνουν:

- φυσική λήψη bits
- επεξεργασία link layer
- lookup
- forwarding
- queueing αν χρειάζεται

Θύρες εξόδου

Κάνουν:

- buffering
- scheduling
- link-layer αποστολή
- φυσική μετάδοση bits

9. Λειτουργίες θύρας εισόδου

Η θύρα εισόδου κάνει:

Auto ▾

line termination



```
link layer receive
lookup / forwarding
queueing
```

Δηλαδή:

- 4943. Λαμβάνει bits.
- 4944. Επεξεργάζεται πλαίσιο Ethernet ή άλλο link-layer frame.
- 4945. Κοιτάζει την IP κεφαλίδα.
- 4946. Κάνει lookup στον πίνακα προώθησης.
- 4947. Στέλνει το datagram στο switching fabric.
- 4948. Αν δεν προλαβαίνει, το βάζει σε ουρά.

10. Προώθηση με βάση τον προορισμό

Η παραδοσιακή IP προώθηση γίνεται με βάση:

Auto ▾



```
destination IP address
```

Ο router κοιτάει τη destination IP και αποφασίζει outgoing interface.

11. Longest Prefix Matching

Το routing table έχει prefixes.

Όταν μια destination IP ταιριάζει με πολλά prefixes, επιλέγεται το:

Auto ▾



```
μεγαλύτερο prefix
```

Δηλαδή αυτό που έχει τα περισσότερα κοινά αρχικά bits.

Παράδειγμα

Πίνακας:

Auto (Go) ▾



```
11001000 00010111 00010*** ***** → interface 0
11001000 00010111 00011000 ***** → interface 1
```

```
11001000 00010111 00011*** ***** → interface 2  
otherwise → interface 3
```

Αν η διεύθυνση είναι:

Auto ▾



```
11001000 00010111 00011000 10101010
```

Ταιριάζει και με γενικά prefixes, αλλά το πιο μεγάλο είναι:

Auto ▾



```
11001000 00010111 00011000 *****
```

Άρα:

Auto ▾



```
interface 1
```

12. TCAM

Στην πράξη για γρήγορο lookup χρησιμοποιείται:

Auto ▾



```
TCAM = Ternary Content Addressable Memory
```

Το TCAM:

- κάνει αναζήτηση σε σταθερό χρόνο
- χρησιμοποιεί 0, 1, και wildcard *
- μπορεί να υποστηρίξει τεράστιους forwarding tables

Παράδειγμα:

Auto ▾



```
Cisco Catalyst: περίπου 1 εκατομμύριο entries
```


13. Switching fabric

Το switching fabric μεταφέρει πακέτο:

Auto ▾



από είσοδο → σωστή έξοδο

Ο ρυθμός μεταγωγής μετρίεται συχνά ως πολλαπλάσιο του line rate.

Αν έχουμε:

Auto ▾



N εισόδους
ρυθμό κάθε γραμμής R

ιδανικά θέλουμε switching rate:

Auto ▾



$N \times R$

14. Τρεις τύποι switching fabric

Υπάρχουν 3 βασικοί τύποι:

6700. Μεταγωγή μέσω μνήμης

6701. Μεταγωγή μέσω διαύλου

6702. Μεταγωγή μέσω δικτύου διασύνδεσης

15. Μεταγωγή μέσω μνήμης

Πρώτης γενιάς routers.

Το πακέτο:

Auto ▾



input port → memory → output port

Χαρακτηριστικά:

- ελεγχόταν από CPU
 - το πακέτο αντιγραφόταν στη μνήμη
 - περιορισμός από bandwidth μνήμης
 - χρειάζονται δύο περάσματα από τον bus
-

16. Μεταγωγή μέσω διαύλου

Το πακέτο μεταφέρεται μέσω κοινού bus.

Χαρακτηριστικά:

- input port → shared bus → output port
- υπάρχει ανταγωνισμός για τον bus
- περιορίζεται από bandwidth του bus
- κατάλληλο για access/enterprise routers

Παράδειγμα:

Auto ▾



Cisco 5600: bus 32 Gbps

17. Μεταγωγή μέσω δικτύου διασύνδεσης

Χρησιμοποιεί:

- crossbar
- multistage switches
- παράλληλη μεταγωγή

Χαρακτηριστικά:

- υψηλή απόδοση
- πολλαπλές μεταφορές ταυτόχρονα
- χρησιμοποιείται σε σύγχρονους routers

Μπορεί να τεμαχίζει datagrams σε σταθερού μήκους cells και να τα περνάει από το fabric.

18. Input queueing

Αν το switching fabric είναι πιο αργό από τις εισόδους, δημιουργούνται ουρές στις θύρες εισόδου.

Συνέπειες:

- καθυστέρηση

- πιθανές απώλειες
- buffer overflow

Head-of-Line Blocking

HOL blocking σημαίνει:

Auto ▾



Το πρώτο πακέτο στην ουρά μπλοκάρει άλλα πακέτα πίσω του, ακόμα κι αν αυτά θα μπορούσαν να προωθηθούν.

Παράδειγμα:

Το πρώτο πακέτο θέλει έξοδο που είναι απασχολημένη.

Ένα επόμενο πακέτο στην ίδια input queue θέλει ελεύθερη έξοδο, αλλά δεν μπορεί να περάσει επειδή είναι πίσω από το πρώτο.

19. Output queueing

Στη θύρα εξόδου έχουμε ουρά όταν:

Auto ▾



τα πακέτα φτάνουν από το switching fabric πιο γρήγορα από όσο μπορεί να τα στείλει η γραμμή εξόδου

Συνέπειες:

- buffering
- καθυστέρηση
- απώλειες λόγω γεμάτου buffer
- ανάγκη scheduling

20. Πόση ενταμίευση χρειάζεται;

Παλιός πρακτικός κανόνας RFC 3439:

Auto ▾



$$\text{Buffer} = \text{RTT} \times C$$

όπου:

- RTT = τυπικό RTT , π.χ. 250 ms
- C = χωρητικότητα ζεύξης

Παράδειγμα:

Auto ▾



```
C = 10 Gbps
RTT = 250 ms
Buffer = 2.5 Gbit
```

Νεότερη σύσταση με N ανεξάρτητες ροές:

Auto ▾



```
Buffer = RTT × C / √N
```

Προσοχή:

Πολύ μεγάλο buffer αυξάνει καθυστέρηση.

Αυτό λέγεται συχνά:

Auto ▾



```
bufferbloat
```

21. Buffer management

Όταν γεμίσει ο buffer, πρέπει να αποφασίσουμε ποιο πακέτο θα απορριφθεί.

Πολιτικές:

Drop-tail

Απορρίπτεται το νέο πακέτο που μόλις έφτασε.

Auto ▾



```
buffer full → drop arriving packet
```

Priority drop

Απορρίπτονται πακέτα με βάση προτεραιότητα.

Marking

Αντί να απορρίπτονται, κάποια πακέτα μαρκάρονται για να δηλώσουν συμφόρηση.

Παραδείγματα:

- ECN
 - RED
-

22. Scheduling

Το scheduling αποφασίζει:

Auto ▾



Ποιο πακέτο θα σταλεί μετά στη ζεύξη εξόδου;

Βασικές πολιτικές:

9429. FCFS / FIFO

9430. Priority scheduling

9431. Round Robin

9432. Weighted Fair Queuing

23. FCFS / FIFO

FCFS = First Come First Served.

FIFO = First In First Out.

Δηλαδή:

Auto ▾



Το πρώτο πακέτο που μπαίνει στην ουρά εξυπηρετείται πρώτο.

Πλεονέκτημα:

- απλό

Μειονέκτημα:

- δεν ξεχωρίζει προτεραιότητες
-

24. Priority scheduling

Τα πακέτα χωρίζονται σε κλάσεις προτεραιότητας.

Παράδειγμα:

- high priority queue
- low priority queue

Ο router στέλνει πρώτα τα πακέτα υψηλής προτεραιότητας.

Μέσα σε κάθε ουρά συνήθως χρησιμοποιείται FCFS.

Μειονέκτημα:

Τα χαμηλής προτεραιότητας πακέτα μπορεί να καθυστερούν πολύ.

25. Round Robin

Τα πακέτα χωρίζονται σε κλάσεις.

Ο scheduler περνά κυκλικά από κάθε ουρά και στέλνει ένα πακέτο αν υπάρχει.

Παράδειγμα:

Auto ▾



class 1 → class 2 → class 3 → class 1 → ...

Πιο δίκαιο από strict priority.

26. Weighted Fair Queuing — WFQ

Το WFQ είναι γενίκευση του Round Robin.

Κάθε κλάση έχει βάρος:

Auto ▾



w_i

Η κλάση παίρνει ποσοστό υπηρεσίας ανάλογο με το βάρος της.

Αν έχουμε κλάσεις με βάρη:

Auto ▾



w_1, w_2, w_3

τότε η κλάση i παίρνει περίπου:

Auto ▾



$$w_i / (w_1 + w_2 + w_3)$$

του bandwidth.

Χρησιμοποιείται για εγγύηση ελάχιστου bandwidth.

27. Ουδετερότητα δικτύου

Η ουδετερότητα δικτύου αφορά το ερώτημα:

Auto ▾



Μπορεί ένας πάροχος να μεταχειρίζεται διαφορετικά διαφορετική κίνηση;

Τεχνικά σχετίζεται με:

- scheduling
- priority
- buffer management
- throttling
- blocking

Κοινωνικά και νομικά σχετίζεται με:

- ελευθερία λόγου
- ανταγωνισμό
- καινοτομία
- ίση μεταχείριση εφαρμογών

Βασικοί κανόνες:

- όχι αποκλεισμός περιεχομένου/εφαρμογών
- όχι επιβράδυνση
- όχι πληρωμένη προτεραιότητα

28. To Internet layer

Στο επίπεδο δικτύου του Internet έχουμε:

Auto ▾



IP

ICMP

Routing protocols / SDN controller

Το IP ορίζει:

- μορφή datagram
- διευθυνσιοδότηση
- χειρισμό πακέτων

Το ICMP κάνει:

- αναφορά σφαλμάτων
- signaling

Τα routing protocols όπως OSPF/BGP υπολογίζουν διαδρομές.

29. IPv4 datagram header

Βασικά πεδία IPv4 header:

Auto (CSS) ▾



```
version
header length
type of service
total length
identifier
flags
fragment offset
TTL
upper layer protocol
header checksum
source IP address
destination IP address
options
payload
```

Version

Δείχνει την έκδοση IP.

Για IPv4:

Auto ▾



version = 4

Header length

Δείχνει το μήκος της IP κεφαλίδας σε 32-bit words.

Συνήθως:

Auto ▾



20 bytes

Type of Service

Χρησιμοποιείται για:

- Diffserv bits
- ECN bits

Δηλαδή μπορεί να δείχνει ποιότητα υπηρεσίας ή συμφόρηση.

Total length

Συνολικό μήκος datagram σε bytes:

Auto ▾



IP header + payload

Μέγιστο:

Auto ▾



64 KB

Συνήθως λόγω Ethernet MTU:

Auto ▾



μέχρι 1500 bytes

Identifier, flags, fragment offset

Χρησιμοποιούνται για fragmentation και reassembly.

TTL

TTL = Time To Live.

Μειώνεται κατά 1 σε κάθε router.

Αν γίνει 0:

Auto ▾



το datagram απορρίπτεται

Σκοπός:

Να μη γυρνάνε πακέτα για πάντα σε loops.

Upper layer protocol

Δείχνει σε ποιο πρωτόκολλο ανώτερου επιπέδου θα παραδοθεί το payload.

Παραδείγματα:

Auto ▾



TCP = 6

UDP = 17

Στις διαφάνειες αναφέρεται TCP/UDP ως ανώτερο επίπεδο.

Header checksum

Ελέγχει μόνο την IP κεφαλίδα.

Δεν ελέγχει το payload.

Source και destination IP

32-bit διευθύνσεις πηγής και προορισμού.

Options

Προαιρετικά πεδία, π.χ.:

- timestamp
 - route recording
-

30. Overhead

Τυπικά:

Auto ▾



20 bytes IP header
20 bytes TCP header

Άρα:

Auto ▾



40 bytes overhead

χωρίς application-layer overhead.

31. Fragmentation και reassembly

Κάθε link έχει MTU.

MTU = Maximum Transmission Unit.

Δηλαδή:

Auto ▾



μέγιστο μέγεθος frame που μπορεί να μεταφέρει η ζεύξη

Αν ένα IP datagram είναι μεγαλύτερο από το MTU, γίνεται fragmentation.

Πού γίνεται fragmentation;

Μπορεί να γίνει μέσα στο δίκτυο από router.

Πού γίνεται reassembly;

Μόνο στον τελικό προορισμό.
Όχι στους ενδιάμεσους routers.

32. Παράδειγμα fragmentation

Έχουμε:

Auto ▾



```
datagram = 4000 bytes  
MTU = 1500 bytes  
IP header = 20 bytes
```

Άρα κάθε fragment μπορεί να έχει payload:

Auto ▾



```
1500 - 20 = 1480 bytes
```

Το αρχικό datagram έχει:

Auto ▾



```
4000 - 20 = 3980 bytes data
```

Fragments:

Fragment 1

Auto ▾



```
length = 1500  
ID = x  
fragflag = 1  
offset = 0  
data = 1480
```

Fragment 2

Auto ▾



```
length = 1500
```

```
ID = x
fragflag = 1
offset = 185
data = 1480
```

Γιατί offset 185;

Auto ▾


$$1480 / 8 = 185$$

Fragment 3

Auto ▾



```
length = 1040
ID = x
fragflag = 0
offset = 370
data = 1020
```

Γιατί offset 370;

Auto ▾


$$2960 / 8 = 370$$

Το τελευταίο έχει flag 0 γιατί δεν ακολουθούν άλλα fragments.

33. IP διευθυνσιοδότηση

Μια IPv4 διεύθυνση είναι:

Auto ▾



32 bits

Γράφεται συνήθως σε dotted decimal notation:

Auto ▾



223.1.1.1

Διαδικά:

Auto ▾



223.1.1.1 =
11011111 00000001 00000001 00000001

34. Interface

IP διεύθυνση έχει η διεπαφή, όχι γενικά η συσκευή.

Interface = σύνδεση ανάμεσα σε host/router και φυσική ζεύξη.

Ένας router έχει πολλές interfaces.

Ένας host συνήθως έχει μία ή δύο:

- Ethernet
- WiFi

35. Υποδίκτυο — Subnet

Υποδίκτυο είναι ομάδα interfaces που μπορούν να επικοινωνούν φυσικά μεταξύ τους χωρίς να παρεμβάλλεται router.

Παράδειγμα:

Auto ▾



223.1.1.0/24

Σημαίνει:

- τα πρώτα 24 bits είναι subnet part
- τα υπόλοιπα 8 bits είναι host part

IP διεύθυνση έχει δύο μέρη

Auto ▾



subnet part | host part

Στο ίδιο subnet, οι συσκευές έχουν κοινό subnet part.

36. Πώς βρίσκουμε τα subnets;

Συνταγή:

15048. Αποσυνδέεις νοητά κάθε interface από host/router.

15049. Οι απομονωμένες νησίδες που μένουν είναι subnets.

Παραδείγματα:

Auto ▾



223.1.1.0/24

223.1.2.0/24

223.1.3.0/24

37. CIDR

CIDR = Classless InterDomain Routing.

Μορφή:

Auto ▾



a.b.c.d/x

Το x δείχνει πόσα bits είναι subnet part.

Παράδειγμα:

Auto ▾



200.23.16.0/23

Σημαίνει:

- 23 bits subnet
- 9 bits host

38. Πώς παίρνει IP ένας host;

Υπάρχουν δύο τρόποι:

15496. Στατική ρύθμιση από admin

15497. DHCP

39. DHCP

DHCP = Dynamic Host Configuration Protocol.

Σκοπός:

Auto ▾



Να δώσει δυναμικά IP διεύθυνση σε έναν host όταν συνδεθεί στο δίκτυο.

Είναι plug-and-play.

Τι δίνει το DHCP;

Δεν δίνει μόνο IP.

Μπορεί να δώσει:

- IP address
 - subnet mask
 - first-hop router / default gateway
 - DNS server name
 - DNS server IP
 - lease time
-

40. Τα 4 DHCP βήματα

1. DHCP Discover

Ο client ψάχνει DHCP server.

Auto (CSS) ▾



```
src IP: 0.0.0.0
dest IP: 255.255.255.255
src port: 68
dest port: 67
```

Broadcast:

Auto ▾



Υπάρχει DHCP server;

2. DHCP Offer

Ο server προσφέρει IP.

Auto ▾



yiaddr = προσφερόμενη IP
lifetime = χρόνος μίσθωσης

Broadcast:

Auto ▾



Είμαι DHCP server, πάρε αυτή τη διεύθυνση.

3. DHCP Request

Ο client ζητά επίσημα τη διεύθυνση.

Broadcast:

Auto ▾



Θέλω να χρησιμοποιήσω αυτή τη διεύθυνση.

4. DHCP ACK

Ο server επιβεβαιώνει.

Broadcast:

Auto ▾



Είναι δική σου.

Σημαντικό

Τα πρώτα δύο βήματα μπορούν να παραλειφθούν αν ο client θυμάται παλιά διεύθυνση και θέλει να την ξαναχρησιμοποιήσει.

41. DHCP encapsulation

Το DHCP μήνυμα ενθυλακώνεται ως εξής:

Auto ▾



DHCP → UDP → IP → Ethernet → Physical

Στο τοπικό LAN στέλνεται broadcast Ethernet frame με destination:

Auto ▾



FF-FF-FF-FF-FF-FF

42. Από πού παίρνει διεύθυνση ένα δίκτυο;

Ένα δίκτυο παίρνει subnet block από τον ISP.

Ο ISP έχει μεγαλύτερο block και το χωρίζει.

Παράδειγμα ISP block:

Auto ▾



200.23.16.0/20

Μπορεί να δοθεί σε οργανισμούς ως μικρότερα blocks:

Auto ▾



200.23.16.0/23

200.23.18.0/23

200.23.20.0/23

...

200.23.30.0/23

43. Ιεραρχική διευθυνσιοδότηση

Η ιεραρχική διευθυνσιοδότηση επιτρέπει route aggregation.

Δηλαδή ένας ISP μπορεί να διαφημίσει μία γενική διαδρομή:

Auto ▾



Στείλτε μου ό,τι αρχίζει με `200.23.16.0/20`

Αντί να διαφημίζει πολλές μικρές.

44. Πιο συγκεκριμένες διαδρομές

Αν ένας οργανισμός αλλάξει ISP, τότε ο νέος ISP μπορεί να διαφημίσει πιο συγκεκριμένο prefix.

Παράδειγμα:

Γενικό:

Auto ▾



`200.23.16.0/20`

Πιο συγκεκριμένο:

Auto ▾



`200.23.18.0/23`

Λόγω longest prefix matching, θα επιλεγεί το πιο συγκεκριμένο.

45. ICANN

Η ICANN:

- κατανέμει IP διευθύνσεις μέσω regional registries
- διαχειρίζεται DNS root servers
- αποδίδει top-level domains

Το τελευταίο block IPv4 δόθηκε το 2011.

Το IPv4 έχει 32-bit διευθύνσεις και δεν φτάνουν για όλους.

Λύσεις:

- NAT
- IPv6

46. NAT

NAT = Network Address Translation.

Στο τυπικό σενάριο, πολλές συσκευές ενός LAN μοιράζονται μία δημόσια IPv4 διεύθυνση.

Παράδειγμα:

Auto ▾



Private LAN: 10.0.0.0/24

Public NAT IP: 138.76.29.7

Οι εσωτερικές συσκευές έχουν:

Auto ▾



10.0.0.x

Προς τα έξω φαίνονται όλες ως:

Auto ▾



138.76.29.7

με διαφορετικές θύρες.

47. Private IP blocks

Ιδιωτικές IPv4 διευθύνσεις:

Auto ▾



10.0.0.0/8

172.16.0.0/12

192.168.0.0/16

Αυτές χρησιμοποιούνται μόνο σε τοπικά δίκτυα.

Δεν δρομολογούνται δημόσια στο Internet.

48. Πλεονεκτήματα NAT

Το NAT:

- εξοικονομεί IPv4 διευθύνσεις
 - χρειάζεται μόνο μία δημόσια IP για πολλά hosts
 - επιτρέπει αλλαγές στο LAN χωρίς ενημέρωση έξω κόσμου
 - επιτρέπει αλλαγή ISP χωρίς αλλαγή εσωτερικών IP
 - κρύβει εσωτερικές συσκευές από έξω κόσμο
-

49. Πώς δουλεύει NAT

Για εξερχόμενα datagrams:

Ο NAT router αλλάζει:

Auto ▾



private source IP, source port

σε:

Auto ▾



public NAT IP, νέο source port

και κρατάει εγγραφή στον NAT table.

Για εισερχόμενα datagrams:

Ο NAT router κοιτάει το destination:

Auto ▾



public NAT IP, port

και το μεταφράζει πίσω σε:

Auto ▾



private IP, private port

50. NAT παράδειγμα

Host:

Auto ▾



10.0.0.1:3345

Θέλει να συνδεθεί σε:

Auto ▾



128.119.40.186:80

Ο NAT αλλάζει:

Auto ▾



Source: 10.0.0.1:3345

σε:

Auto ▾



Source: 138.76.29.7:5001

Κρατάει στον πίνακα:

Auto ▾



138.76.29.7:5001 ↔ 10.0.0.1:3345

Όταν απαντήσει ο server:

Auto ▾



Destination: 138.76.29.7:5001

ο NAT το αλλάζει σε:

Auto ▾



Destination: 10.0.0.1:3345

51. NAT και θύρες

Η θύρα είναι 16 bits.

Άρα θεωρητικά υπάρχουν περίπου:

Auto ▾



$$2^{16} = 65536 \text{ θύρες}$$

Στην πράξη περίπου 60.000 ταυτόχρονες συνδέσεις ανά δημόσια IP.

52. Γιατί το NAT είναι αμφιλεγόμενο;

Επειδή:

- οι routers κανονικά πρέπει να κοιτούν μέχρι layer 3, όχι ports
- οι ports ανήκουν στο transport layer
- παραβιάζει την end-to-end αρχή
- δημιουργεί πρόβλημα όταν έξω client θέλει να συνδεθεί σε server πίσω από NAT
- η σωστή λύση θα ήταν IPv6

Αλλά:

Auto ▾



Το NAT χρησιμοποιείται παντού.

Σε:

- σπίτια
- πανεπιστήμια
- εταιρείες
- 4G/5G δίκτυα

53. NAT traversal problem

Πρόβλημα:

Ένας εξωτερικός client θέλει να συνδεθεί σε server πίσω από NAT:

Auto ▾



10.0.0.1

Όμως αυτή η διεύθυνση είναι ιδιωτική και δεν φαίνεται στο Internet.

Λύση 1: Static port forwarding

Ρυθμίζουμε τον NAT router:

Auto ▾



138.76.29.7:2500 → 10.0.0.1:2500

Άρα κάθε εισερχόμενο στη δημόσια IP port 2500 προωθείται στον εσωτερικό server.

Λύση 2: UPnP IGD

Το UPnP επιτρέπει σε host πίσω από NAT να:

- μάθει τη δημόσια IP
- δημιουργήσει port mappings
- αφαιρέσει port mappings
- ορίσει lease times

Είναι αυτοματοποίηση του port forwarding.

Λύση 3: Relay

Χρησιμοποιείται π.χ. στο Skype.

Ιδέα:

20841. Ο host πίσω από NAT ανοίγει σύνδεση προς relay.

20842. Ο εξωτερικός client συνδέεται επίσης στο relay.

20843. Το relay γεφυρώνει την επικοινωνία.

54. IPv6 κίνητρο

Το IPv6 δημιουργήθηκε κυρίως γιατί:

Auto ▾



οι IPv4 διευθύνσεις τελείωναν

Επιπλέον προσφέρει:

- 128-bit διευθύνσεις
 - σταθερή κεφαλίδα 40 bytes
 - ταχύτερη επεξεργασία
 - όχι fragmentation από routers
 - δυνατότητα flow label / QoS
-

55. IPv6 header

Βασικά πεδία IPv6:

Auto (CSS) ▾



```
version
priority / traffic class
flow label
payload length
next header
hop limit
source address
destination address
payload
```

Οι διευθύνσεις είναι:

Auto ▾



```
128 bits
```

56. Τι λείπει από IPv6 σε σχέση με IPv4;

Στο IPv6 δεν υπάρχει:

- header checksum
- fragmentation/reassembly από routers
- options μέσα στην κύρια κεφαλίδα

Γιατί;

Για πιο γρήγορη επεξεργασία από routers.

Options μπορούν να μπουν μέσω next header.

57. IPv6 tunneling

Δεν μπορούν να αναβαθμιστούν όλοι οι routers ταυτόχρονα.

Άρα υπάρχει tunneling.

Tunneling σημαίνει:

Auto ▾



IPv6 datagram μπαίνει ως payload μέσα σε IPv4 datagram

Δηλαδή:

Auto ▾



IPv6 packet inside IPv4 packet

Χρησιμοποιείται όταν δύο IPv6 routers επικοινωνούν μέσω IPv4 δικτύου.

Παράδειγμα tunneling

Λογικά:

Auto ▾



A → B → E → F

Φυσικά μπορεί να υπάρχει IPv4 tunnel:

Auto ▾



B → C → D → E

Το IPv6 datagram έχει:

Auto ▾



IPv6 src = A
IPv6 dest = F

Το εξωτερικό IPv4 datagram έχει:

Auto ▾



```
IPv4 src = B  
IPv4 dest = E
```

Σημαντικό:

Υπάρχουν δύο επίπεδα διευθύνσεων:

- εσωτερικές IPv6 διευθύνσεις
- εξωτερικές IPv4 διευθύνσεις tunnel

58. Μετάβαση IPv4 σε IPv6

Η μετάβαση είναι αργή.

Λόγοι:

- τεράστια εγκατεστημένη βάση IPv4
- NAT καθυστερεί την ανάγκη μετάβασης
- κόστος αναβάθμισης
- πρέπει να δουλεύουν μαζί IPv4 και IPv6

Οι διαφάνειες αναφέρουν ότι η μετάβαση κρατά πάνω από 25 χρόνια.

59. Γενικευμένη προώθηση και SDN

Στην παραδοσιακή προώθηση:

Auto ▾



```
match destination IP → forward to output port
```

Στη γενικευμένη προώθηση:

Auto ▾



```
match σε διάφορα πεδία κεφαλίδας → action
```

Αυτό είναι η λογική:

Auto ▾



```
match + action
```

60. OpenFlow

Στο OpenFlow κάθε switch/router έχει flow table.

Κάθε entry έχει:

23054. Match fields

23055. Counters

23056. Actions

Match fields

Μπορούν να περιλαμβάνουν:

- switch port
 - MAC source
 - MAC destination
 - Ethernet type
 - VLAN ID
 - IP source
 - IP destination
 - IP protocol
 - TCP source port
 - TCP destination port
-

Actions

Ενέργειες:

- forward σε θύρα
 - drop
 - modify fields
 - send to controller
 - encapsulate and send to controller
-

Counters

Μετρητές:

- αριθμός πακέτων
 - αριθμός bytes
-

61. OpenFlow παραδείγματα

Προώθηση με βάση προορισμό

Auto ▾



IP Dst = 51.6.0.8 → forward port 6

Firewall rule για SSH

Auto ▾



TCP destination port = 22 → drop

Δηλαδή μπλοκάρει SSH.

Firewall rule για source IP

Auto ▾



IP Src = 128.119.1.1 → drop

Μπλοκάρει όλα τα datagrams από αυτή την IP.

62. Match + action σε διαφορετικές συσκευές

Συσκευή	Match	Action
Router	longest prefix destination IP	forward σε output link
Switch	MAC destination	forward ή flood
Firewall	IP και TCP/UDP ports	allow ή deny
NAT	IP και port	αλλαγή IP/port

63. OpenFlow παράδειγμα με hosts

Στο παράδειγμα:

- hosts h5 και h6 βρίσκονται στο 10.3.0.x
- hosts h3 και h4 βρίσκονται στο 10.2.0.x
- θέλουμε datagrams από h5/h6 προς h3/h4 να περάσουν μέσω s1 και s2

Κανόνες τύπου:

Auto ▾

```
IP Src = 10.3.*.*  
IP Dst = 10.2.*.*  
→ forward(3)
```



ή ανάλογα με το switch:

Auto ▾

```
ingress port = 2  
IP Dst = 10.2.0.3  
→ forward(3)
```



Auto ▾

```
ingress port = 2  
IP Dst = 10.2.0.4  
→ forward(4)
```



64. Middleboxes

Middlebox είναι κάθε ενδιάμεση συσκευή που κάνει κάτι παραπάνω από την απλή IP δρομολόγηση.

Ορισμός:

Auto ▾

Ενδιάμεσο κουτί που εκτελεί λειτουργίες διαφορετικές από τις τυπικές λειτουργίες ενός IP router στη διαδρομή δεδομένων.



Παραδείγματα middleboxes

- NAT
- Firewalls
- IDS
- Load balancers
- Caches
- Application-layer gateways

Υπάρχουν σε:

- εταιρικά δίκτυα
- ISP δίκτυα
- data centers
- cloud υποδομές

65. Εξέλιξη middleboxes

Αρχικά ήταν ξεχωριστές εξειδικευμένες συσκευές hardware.

Σήμερα μετακινούνται προς:

- software-based implementations
- SDN
- NFV

NFV

NFV = Network Functions Virtualization.

Σημαίνει:

Auto (Bash) ▾



Δικτυακές λειτουργίες ως προγραμματιζόμενες υπηρεσίες πάνω σε κοινό hardware/cloud.

Παραδείγματα:

- virtual firewall
- virtual NAT
- virtual load balancer

66. Η κλεψύδρα του IP

Το Internet έχει αρχιτεκτονική κλεψύδρας.

Στη "λεπτή μέση" βρίσκεται:

Auto ▾



IP

Πάνω από IP υπάρχουν πολλά:

- TCP
- UDP
- QUIC
- HTTP
- SMTP
- DASH
- RTP

Κάτω από IP υπάρχουν πολλά:

- Ethernet
- WiFi
- Bluetooth
- PPP
- radio
- fiber
- copper

Άρα:

Auto ▾



Πολλά πάνω, πολλά κάτω, ένα κοινό IP στη μέση.

67. Αρχές του Internet

Βασική αρχή RFC 1958:

Auto ▾



Ο στόχος είναι η συνδεσιμότητα,
το εργαλείο είναι το IP,
και η εξυπνάδα βρίσκεται στα άκρα.

Τρεις ιδέες:

- 25940. Απλή συνδεσιμότητα
 - 25941. IP ως λεπτή μέση
 - 25942. Πολυπλοκότητα στα άκρα
-

68. Πού βρίσκεται η εξυπνάδα;

Τηλεφωνικό δίκτυο

Η εξυπνάδα ήταν στους μεταγωγείς.

Internet πριν το 2005

Η εξυπνάδα ήταν κυρίως στα άκρα:

- hosts
 - applications
 - servers
-

Internet μετά το 2005

Η εξυπνάδα υπάρχει και σε:

- προγραμματιζόμενες δικτυακές συσκευές
 - middleboxes
 - cloud
 - data centers
 - application-layer υποδομές
-

69. Τελική σύνοψη κεφαλαίου

Το κεφάλαιο καλύπτει:

- 26428. Επισκόπηση επιπέδου δικτύου
- 26429. Data plane και control plane
- 26430. Forwarding και routing
- 26431. Αρχιτεκτονική router
- 26432. Input ports
- 26433. Output ports
- 26434. Switching fabric
- 26435. Queueing
- 26436. Buffer management

26437. Scheduling
26438. Network neutrality
26439. IPv4 datagram
26440. Fragmentation
26441. IP addressing
26442. Subnets
26443. CIDR
26444. DHCP
26445. Hierarchical addressing
26446. NAT
26447. NAT traversal
26448. IPv6
26449. Tunneling
26450. SDN
26451. OpenFlow
26452. Middleboxes
26453. IP hourglass

70. SOS για εξέταση

Να ξέρεις οπωσδήποτε:

Auto ▾



Forwarding ≠ Routing

Auto (Java) ▾



Data plane = τοπικό, γρήγορο, hardware

Control plane = δρομολόγηση, πιο αργό, software/controller

Auto ▾



Internet = best effort

Auto ▾



Longest prefix matching = διάλεξε το πιο συγκεκριμένο prefix

Auto (Lua) ▾



Router = input ports + switching fabric + output ports + routing processor

Auto ▾



HOL blocking = πρώτο πακέτο μπλοκάρει τα επόμενα

Auto ▾



Output queueing = πολλά πακέτα φτάνουν στην ίδια έξοδο

Auto ▾



Drop-tail = γεμάτος buffer → πετάω το νέο πακέτο

Auto ▾



FIFO = πρώτο μέσα, πρώτο έξω

Auto ▾



WFQ = δίκαιη κατανομή με βάρη

Auto ▾



IPv4 = 32-bit διευθύνσεις

IPv6 = 128-bit διευθύνσεις

Auto ▾



CIDR = a.b.c.d/x

Auto ▾



DHCP = Discover, Offer, Request, ACK

Auto ▾



NAT = πολλές private IP μοιράζονται μία public IP

Auto ▾



Private IP blocks:

10.0.0.0/8

172.16.0.0/12

192.168.0.0/16

Auto ▾



IPv6 tunneling = IPv6 μέσα σε IPv4

Auto ▾



OpenFlow = match + action + counters

Auto ▾



Middlebox = NAT, firewall, IDS, load balancer, cache

71. Οι βασικοί τύποι

Buffer παλιός κανόνας

Auto ▾



$$\text{Buffer} = \text{RTT} \times C$$

Buffer με N ροές

Auto ▾



$$\text{Buffer} = \text{RTT} \times C / \sqrt{N}$$

CIDR

Auto ▾



$$a.b.c.d/x$$

x = bits υποδικτύου.

Hosts σε subnet

Αν έχεις /x, τότε host bits:

Auto ▾



$$32 - x$$

Πλήθος διευθύνσεων:

Auto ▾



$$2^{(32-x)}$$

Συνήθως usable hosts:

Auto ▾



$$2^{(32-x)} - 2$$

Fragment offset

Auto ▾



$$\text{offset} = \text{προηγούμενα data bytes} / 8$$

72. Πολύ σύντομο μνημονικό

Auto (Bash) ▾



Router:

μπαίνει πακέτο → lookup → switching fabric → output queue → [link](#)

Auto ▾



IP:

best effort, 32-bit IPv4, fragmentation, TTL, checksum

Auto ▾



DHCP:

Discover → Offer → Request → ACK

Auto ▾



NAT:
private IP:port ↔ public IP:port

Auto (Visual Basic .NET) ▾



IPv6:
128-bit, 40-byte header, no checksum, no router fragmentation

Auto ▾



SDN/OpenFlow:
match + action

Δίκτυα Επικοινωνιών I

Ενότητα 5: Επίπεδο Δικτύου — Επίπεδο Ελέγχου

1. Τι είναι το επίπεδο ελέγχου

Στο επίπεδο δικτύου έχουμε δύο βασικές λειτουργίες:

Λειτουργία	Τι κάνει
Πρώθηση / Forwarding	Μεταφέρει πακέτο από είσοδο router σε σωστή έξοδο
Δρομολόγηση / Routing	Υπολογίζει τη διαδρομή από πηγή σε προορισμό

Το **επίπεδο ελέγχου** ασχολείται κυρίως με τη **δρομολόγηση**.

Δηλαδή απαντά:

Auto ▾



Από ποια διαδρομή πρέπει να περάσει το πακέτο;

2. Δύο προσεγγίσεις επιπέδου ελέγχου

1. Έλεγχος ανά δρομολογητή

Παραδοσιακή προσέγγιση.

Κάθε router:

- τρέχει δικό του routing algorithm
- επικοινωνεί με άλλους routers
- υπολογίζει πίνακα προώθησης

Auto ▾



Κάθε router έχει και data plane και control plane.

2. Λογικά κεντριοποιημένος έλεγχος — SDN

Στο SDN υπάρχει απομακρυσμένος controller.

Ο controller:

- υπολογίζει τους πίνακες
- τους στέλνει στους routers/switches
- έχει κεντρική εικόνα του δικτύου

Auto ▾



Data plane στους switches
Control plane στον controller

3. Στόχος routing protocols

Τα πρωτόκολλα δρομολόγησης βρίσκουν «καλά» μονοπάτια από πηγές σε προορισμούς.

Ένα μονοπάτι είναι:

Auto ▾



ακολουθία routers από source μέχρι destination

«Καλό» μονοπάτι μπορεί να σημαίνει:

- μικρότερο κόστος
- λιγότερα hops
- μικρότερη καθυστέρηση
- λιγότερη συμφόρηση
- μεγαλύτερο bandwidth

4. Μοντέλο γράφου

Το δίκτυο μοντελοποιείται ως γράφος:

Auto ▾



$G = (N, E)$

όπου:

- N = σύνολο routers / κόμβων
- E = σύνολο ζεύξεων

Κάθε ζεύξη έχει κόστος:

Auto ▾



$c(x, y)$

Αν δύο κόμβοι δεν είναι γείτονες:

Auto ▾



$c(x, y) = \infty$

Το κόστος μπορεί να είναι:

- πάντα 1

- αντίστροφο του bandwidth
- ανάλογο της συμφόρησης
- τιμή που ορίζει ο διαχειριστής

5. Κατηγορίες routing algorithms

Με βάση την πληροφορία

Τύπος	Περιγραφή
Καθολικοί / Global	Όλοι οι routers ξέρουν όλη την τοπολογία
Αποκεντρωμένοι / Decentralized	Κάθε router ξέρει αρχικά μόνο τους γείτονές του

Link State

Στους αλγόριθμους κατάστασης ζεύξης:

- όλοι ξέρουν όλη την τοπολογία
- όλοι ξέρουν τα κόστη των ζεύξεων
- χρησιμοποιείται Dijkstra

Distance Vector

Στους αλγόριθμους διανύσματος απόστασης:

- κάθε router ξέρει αρχικά μόνο τους γείτονές του
- ανταλλάσσει πληροφορίες με γείτονες
- χρησιμοποιείται Bellman-Ford

Με βάση το πόσο συχνά αλλάζουν τα μονοπάτια

Τύπος	Περιγραφή
Στατικοί	τα μονοπάτια αλλάζουν σπάνια
Δυναμικοί	

	τα μονοπάτια αλλάζουν συχνά λόγω κόστους/ συμφόρησης
--	--

6. Link State Routing — Dijkstra

Ο αλγόριθμος Dijkstra:

- είναι κεντριοποιημένος ως προς την πληροφορία
- κάθε router έχει πλήρη γνώση της τοπολογίας
- βρίσκει διαδρομές ελάχιστου κόστους από μία πηγή προς όλους τους άλλους κόμβους
- δίνει πίνακα δρομολόγησης για την πηγή

Συμβολισμοί Dijkstra

Σύμβολο	Σημασία
$c(x,y)$	κόστος απευθείας ζεύξης $x-y$
$D(v)$	τρέχουσα εκτίμηση κόστους από πηγή προς v
$p(v)$	προηγούμενος κόμβος στη διαδρομή προς v
N'	σύνολο κόμβων με οριστικοποιημένη ελάχιστη διαδρομή

Ιδέα Dijkstra

Ξεκινάμε από την πηγή.

Σε κάθε βήμα:

3179. Διαλέγουμε τον μη οριστικοποιημένο κόμβο με το μικρότερο D .

3180. Τον βάζουμε στο N' .

3181. Ενημερώνουμε τους γείτονές του.

Τύπος ενημέρωσης:

Auto ▾



$$D(v) = \min(D(v), D(w) + c(w,v))$$

Ψευδοκώδικας

Auto (Visual Basic .NET) ▾



$$N' = \{u\}$$

Για κάθε κόμβο v :

αν v είναι γείτονας του u :

$$D(v) = c(u,v)$$

αλλιώς:

$$D(v) = \infty$$

Επανάλαβε:

βρες w εκτός N' με ελάχιστο $D(w)$

πρόσθεσε w στο N'

για κάθε γείτονα v του w εκτός N' :

$$D(v) = \min(D(v), D(w) + c(w,v))$$

μέχρι όλοι οι κόμβοι να μπουν στο N'

7. Παράδειγμα Dijkstra

Αν από τον u τελικά έχουμε δέντρο συντομότερων διαδρομών:

Auto ▾



$$u \rightarrow v$$

$$u \rightarrow x$$

$$u \rightarrow x \rightarrow y$$

$$u \rightarrow x \rightarrow y \rightarrow w$$

$$u \rightarrow x \rightarrow y \rightarrow z$$

Τότε ο πίνακας προώθησης στον u δείχνει το πρώτο βήμα:

--	--

Προορισμός	Εξερχόμενη ζεύξη
v	u-v
x	u-x
y	u-x
w	u-x
z	u-x

Σημαντικό:

Ο πίνακας forwarding δεν αποθηκεύει όλη τη διαδρομή, αλλά το **επόμενο άλμα**.

8. Πολυπλοκότητα Dijkstra

Για n κόμβους:

Auto ▾



$O(n^2)$

Με πιο αποδοτικές υλοποιήσεις:

Auto ▾



$O(n \log n)$

Πολυπλοκότητα μηνυμάτων Link State

Κάθε router πρέπει να διαδώσει την πληροφορία των ζεύξεών του σε όλους.

Με n κόμβους και E ζεύξεις:

Auto ▾



$O(nE)$ μηνύματα

9. Πρόβλημα ταλαντώσεων στον Dijkstra

Αν το κόστος ζεύξης εξαρτάται από την κίνηση, μπορεί να έχουμε ταλαντώσεις.

Δηλαδή:

4534. Οι routers βλέπουν μικρό κόστος σε μια διαδρομή.

4535. Όλη η κίνηση πάει εκεί.

4536. Η διαδρομή συμφόρει.

4537. Το κόστος της αυξάνεται.

4538. Όλη η κίνηση φεύγει αλλού.

4539. Μετά συμφόρει η άλλη διαδρομή.

Άρα το routing μπορεί να αλλάζει συνέχεια.

10. Distance Vector Routing

Ο αλγόριθμος Distance Vector βασίζεται στην εξίσωση Bellman-Ford.

Bellman-Ford

Έστω:

Auto ▾



$Dx(y)$ = κόστος ελάχιστης διαδρομής από x προς y

Τότε:

Auto ▾



$Dx(y) = \min_v \{ c(x,v) + Dv(y) \}$

όπου v είναι γείτονας του x .

Δηλαδή:

Auto ▾



Για να πάω από x στο y , δοκιμάζω κάθε γείτονα v :
κόστος $x \rightarrow v$ + κόστος $v \rightarrow y$
και παίρνω το ελάχιστο.

11. Παράδειγμα Bellman-Ford

Αν ο u έχει γείτονες v, x, w και θέλει να πάει στο z :

Auto ▾



$$D_v(z) = 5$$

$$D_x(z) = 3$$

$$D_w(z) = 3$$

και:

Auto ▾



$$c(u, v) = 2$$

$$c(u, x) = 1$$

$$c(u, w) = 5$$

τότε:

Auto ▾



$$\begin{aligned} D_u(z) &= \min\{2+5, 1+3, 5+3\} \\ &= \min\{7, 4, 8\} \\ &= 4 \end{aligned}$$

Άρα ο u πάει στο z μέσω x .

12. Ιδέα Distance Vector

Κάθε κόμβος:

5501. Διατηρεί ένα διάνυσμα αποστάσεων.
5502. Στέλνει περιοδικά το DV στους γείτονές του.
5503. Λαμβάνει DVs από γείτονες.
5504. Εφαρμόζει Bellman-Ford.
5505. Αν αλλάξει το DV του, ενημερώνει τους γείτονες.

Χαρακτηριστικά DV

Το Distance Vector είναι:

- επαναληπτικό
- ασύγχρονο
- κατανεμημένο

- αυτο-τερματιζόμενο

Αν δεν αλλάζει τίποτα:

Auto ▾



κανείς δεν στέλνει νέα ενημέρωση

13. Distance Vector — γενικός αλγόριθμος

Κάθε κόμβος κάνει:

Auto ▾



Περίμενε για:

αλλαγή κόστους τοπικής ζεύξης
ή μήνυμα από γείτονα

Ξαναυπολόγισε τις αποστάσεις με Bellman-Ford.

Αν άλλαξε το DV:

στείλε το νέο DV στους γείτονες.

14. Διάδοση πληροφορίας στο DV

Η πληροφορία διαδίδεται σταδιακά.

Αν ο κόμβος c ξέρει κάτι στο $t=0$:

Auto ▾



$t=0$: το ξέρει μόνο ο c

$t=1$: το ξέρουν οι γείτονες του c

$t=2$: το ξέρουν κόμβοι 2 hops μακριά

$t=3$: το ξέρουν κόμβοι 3 hops μακριά

$t=4$: το ξέρουν κόμβοι 4 hops μακριά

Άρα η πληροφορία ταξιδεύει hop-by-hop.

15. Αλλαγές κόστους στο DV

Αν αλλάξει το κόστος μιας τοπικής ζεύξης:

6543. Ο κόμβος το ανιχνεύει.

6544. Ενημερώνει το DV του.

6545. Αν άλλαξε κάτι, ειδοποιεί τους γείτονες.

6546. Οι γείτονες κάνουν το ίδιο.

Καλά νέα ταξιδεύουν γρήγορα

Αν ένα κόστος μειωθεί, οι κόμβοι συνήθως βρίσκουν γρήγορα καλύτερες διαδρομές.

Κακά νέα ταξιδεύουν αργά

Αν ένα κόστος αυξηθεί ή χαλάσει μια ζεύξη, μπορεί να εμφανιστεί:

Auto ▾



count-to-infinity problem

Δηλαδή οι routers αυξάνουν σιγά σιγά την εκτίμηση κόστους μέχρι να καταλάβουν ότι ο προορισμός δεν είναι προσβάσιμος.

16. Poisoned Reverse

Το poisoned reverse μειώνει το count-to-infinity.

Ιδέα:

Αν ο Z δρομολογεί προς X μέσω Y, τότε ο Z λέει στον Y:

Auto ▾



Η απόστασή μου προς X είναι ∞ .

Έτσι ο Y δεν θα προσπαθήσει να πάει προς X μέσω Z.

17. Σύγκριση Link State και Distance Vector

Χαρακτηριστικό	Link State	Distance Vector
Πληροφορία	πλήρης τοπολογία	μόνο από γείτονες
Αλγόριθμος	Dijkstra	Bellman-Ford

Ανταλλαγή	flooding σε όλους	μόνο γείτονες
Σύγκλιση	συνήθως γρήγορη	μπορεί να αργήσει
Προβλήματα	ταλαντώσεις	loops, count-to-infinity
Μηνύματα	$O(nE)$	εξαρτάται από αλλαγές
Λάθος router	επηρεάζει δικές του ανακοινώσεις	λάθη διαδίδονται

18. Ιεραρχική δρομολόγηση

Μέχρι τώρα θεωρούσαμε flat δίκτυο.

Στην πράξη αυτό δεν γίνεται, γιατί:

- υπάρχουν εκατοντάδες εκατομμύρια προορισμοί
- δεν χωράνε όλοι στους πίνακες
- οι ενημερώσεις θα κατέκλυζαν το δίκτυο
- κάθε οργανισμός θέλει αυτονομία

Άρα χρησιμοποιείται ιεραρχική δρομολόγηση.

19. Αυτόνομα Συστήματα — AS

Το Internet είναι δίκτυο δικτύων.

Οι routers ομαδοποιούνται σε:

Auto ▾



AS = Autonomous System

Ένα AS είναι δίκτυο υπό κοινή διαχείριση.

Παραδείγματα:

- ISP
- πανεπιστημιακό δίκτυο
- μεγάλο εταιρικό δίκτυο

Intra-AS routing

Δρομολόγηση μέσα στο ίδιο AS.

Λέγεται και:

Auto ▾



Interior Gateway Protocol / IGP

Παραδείγματα:

- RIP
- OSPF
- IGRP

Inter-AS routing

Δρομολόγηση ανάμεσα σε διαφορετικά AS.

Κύριο πρωτόκολλο:

Auto ▾



BGP

Gateway router

Gateway router είναι router στην άκρη ενός AS.

Έχει σύνδεση με router άλλου AS.

20. RIP

RIP = Routing Information Protocol.

Χαρακτηριστικά:

- παλιό πρωτόκολλο
- χρησιμοποιήθηκε στο BSD Unix το 1982
- βασίζεται σε Distance Vector
- μετρική = αριθμός hops
- κάθε ζεύξη έχει κόστος 1
- μέγιστο 15 hops
- 16 σημαίνει άπειρο / μη προσβάσιμο
- ανταλλάσσει ενημερώσεις κάθε 30 sec

- κάθε advertisement έχει μέχρι 25 υποδίκτυα
 - χρησιμοποιεί UDP port 520
 - διαχειρίζεται από διεργασία route-d / routed
-

RIP failure recovery

Αν δεν ακουστεί advertisement για:

Auto ▾



180 sec

ο γείτονας θεωρείται νεκρός.

Τότε:

- οι διαδρομές μέσω αυτού ακυρώνονται
 - στέλνονται νέες διαφημίσεις
 - οι γείτονες ενημερώνουν άλλους γείτονες
 - χρησιμοποιείται poison reverse για αποφυγή loops
-

21. OSPF

OSPF = Open Shortest Path First.

Χαρακτηριστικά:

- ανοικτή δημόσια προδιαγραφή
 - intra-AS protocol
 - βασίζεται σε Link State
 - χρησιμοποιεί Dijkstra
 - κάθε router έχει χάρτη τοπολογίας
 - κάνει flooding των LS advertisements
 - μεταφέρεται απευθείας πάνω από IP
 - δεν χρησιμοποιεί TCP ή UDP
 - IP protocol number = 89
-

Τι περιέχει OSPF advertisement

Η OSPF δημοσιοποίηση μεταφέρει πληροφορία για κάθε γειτονικό router.

Δηλαδή λέει:

Auto ▾



Ποιοι είναι οι γείτονές μου και με τι κόστος συνδέομαι μαζί τους.

22. Προηγμένα χαρακτηριστικά OSPF

Το OSPF έχει χαρακτηριστικά που δεν έχει το RIP:

- 10027. Αυθεντικοποίηση μηνυμάτων.
- 10028. Πολλαπλές ίσου κόστους διαδρομές.
- 10029. Πολλαπλές μετρικές κόστους ανά TOS.
- 10030. Υποστήριξη unicast και multicast.
- 10031. MOSPF για multicast.
- 10032. Ιεραρχικό OSPF για μεγάλα domains.

23. Ιεραρχικό OSPF

Το OSPF μπορεί να χωρίσει ένα μεγάλο AS σε περιοχές.

Υπάρχουν δύο επίπεδα:

Auto ▾



τοπικές περιοχές
backbone area

Χαρακτηριστικά ιεραρχικού OSPF

- Οι LS advertisements μένουν μέσα στην περιοχή.
- Κάθε router ξέρει λεπτομερώς την τοπολογία της περιοχής του.
- Για άλλες περιοχές ξέρει μόνο κατεύθυνση/διαδρομή.
- Μειώνεται το μέγεθος των πινάκων.
- Μειώνονται τα routing messages.

Τύποι routers στο OSPF

Router	Ρόλος
Internal router	μέσα σε μία περιοχή

Area border router	συνδέει περιοχή με backbone
Backbone router	τρέχει OSPF στο backbone
Boundary router	συνδέει το AS με άλλα AS

24. Inter-AS routing

Αν ένας router σε AS1 λάβει datagram για προορισμό έξω από το AS1, πρέπει να αποφασίσει:

Auto ▾



Σε ποιον gateway router θα το στείλω;

Το AS πρέπει:

11093. Να μάθει ποιοι προορισμοί είναι προσβάσιμοι μέσω άλλων AS.

11094. Να διαδώσει αυτή την πληροφορία στους εσωτερικούς routers.

11095. Να επιλέξει gateway.

25. Hot Potato Routing

Αν ένας προορισμός είναι προσβάσιμος από πολλούς gateways, μπορεί να χρησιμοποιηθεί hot potato routing.

Ιδέα:

Auto (Visual Basic .NET) ▾



Ξεφορτώσου την κίνηση από το δικό σου AS όσο πιο γρήγορα γίνεται.

Άρα επιλέγουμε τον πιο κοντινό gateway με βάση το intra-AS κόστος.

26. BGP

BGP = Border Gateway Protocol.

Είναι το βασικό πρωτόκολλο δρομολόγησης ανάμεσα σε AS.

Είναι:

Auto ▾



de facto inter-domain routing protocol

Το BGP επιτρέπει σε κάθε AS:

- να μαθαίνει reachable prefixes από γειτονικά AS
- να διαδίδει πληροφορία μέσα στο AS
- να επιλέγει διαδρομές με βάση πολιτική
- να διαφημίζει την ύπαρξή του στο Internet

27. eBGP και iBGP

eBGP

External BGP.

Χρησιμοποιείται ανάμεσα σε routers διαφορετικών AS.

Auto ▾



AS1 ↔ AS2

iBGP

Internal BGP.

Χρησιμοποιείται μέσα στο ίδιο AS για διάδοση πληροφορίας BGP.

Auto ▾



routers μέσα στο AS1

28. BGP sessions

Δύο BGP routers που επικοινωνούν λέγονται peers.

Ανταλλάσσουν BGP messages πάνω από TCP.

Auto ▾



TCP port 179

Το BGP είναι πρωτόκολλο:

Auto ▾



path vector

Δηλαδή δεν διαφημίζει απλά κόστος, αλλά ολόκληρο AS path.

29. BGP messages

Μήνυμα	Ρόλος
OPEN	ανοίγει TCP σύνδεση και ταυτοποιεί τον αποστολέα
UPDATE	διαφημίζει νέα διαδρομή ή αποσύρει παλιά
KEEPALIVE	κρατά τη σύνδεση ζωντανή και επιβεβαιώνει OPEN
NOTIFICATION	αναφέρει σφάλμα και κλείνει σύνδεση

30. Διανομή reachability στο BGP

Παράδειγμα:

- 12744. Το AS3 στέλνει μέσω eBGP στο AS1 ότι ξέρει ένα prefix.
- 12745. Ο router 1c του AS1 το μαθαίνει.
- 12746. Ο 1c το διαδίδει με iBGP στους άλλους routers του AS1.
- 12747. Το AS1 μπορεί να το ξαναδιαφημίσει στο AS2 μέσω eBGP.
- 12748. Οι routers βάζουν entries στους forwarding tables.

31. BGP route

Μια BGP διαδρομή είναι:

Auto ▾



prefix + attributes

Δύο σημαντικά attributes:

13115. AS-PATH

13116. NEXT-HOP

AS-PATH

Το AS-PATH περιέχει τα AS από τα οποία πέρασε η διαφήμιση.

Παράδειγμα:

Auto ▾



AS-PATH = 1755 1239 7018 6341

Χρήσεις:

- αποφυγή loops
- επιλογή διαδρομής
- πολιτικές routing

NEXT-HOP

Το NEXT-HOP δείχνει τον συγκεκριμένο router προς τον οποίο πρέπει να σταλεί η κίνηση για αυτή τη διαδρομή.

Κάθε φορά που μια ανακοίνωση περνά σύνορα AS, το NEXT-HOP αλλάζει στη διεύθυνση του border router που ανακοίνωσε τη διαδρομή.

32. Import policy

Όταν ένας gateway router λαμβάνει BGP route, εφαρμόζει πολιτική εισαγωγής.

Μπορεί να πει:

Auto ▾



δέχομαι τη διαδρομή

ή

Auto ▾



την απορρίπτω

Παράδειγμα policy:

Auto ▾



Ποτέ μη δρομολογείς μέσω του AS X.

33. Συνδυασμός intra-AS και inter-AS

Για να μπει entry στον forwarding table:

13928. Το BGP λέει ποιο prefix είναι reachable και ποιο είναι το NEXT-HOP.

13929. Το intra-AS routing λέει πώς φτάνω στο NEXT-HOP.

13930. Ο router βάζει forwarding entry προς την κατάλληλη εσωτερική διεπαφή.

34. Επιλογή διαδρομής BGP

Αν υπάρχουν πολλές BGP διαδρομές, επιλέγεται με σειρά:

14215. Μεγαλύτερη local preference.

14216. Μικρότερο AS-PATH.

14217. Πλησιέστερο NEXT-HOP, δηλαδή hot potato.

14218. Πρόσθετα κριτήρια.

35. Local Preference

Το Local Preference χρησιμοποιείται μέσα στο iBGP.

Μεγαλύτερη τιμή σημαίνει προτιμότερη διαδρομή.

Παράδειγμα:

Auto ▾



local pref = 100 καλύτερο από local pref = 90

Είναι κυρίως πολιτική απόφαση.

36. Το μικρότερο AS-PATH δεν σημαίνει πάντα πραγματικά μικρότερη διαδρομή

Το BGP μετρά AS hops, όχι πραγματική καθυστέρηση ή φυσικό μήκος.

Άρα:

Auto ▾



AS path 4 1

μπορεί να επιλεγεί αντί για:

Auto ▾



AS path 3 2 1

ακόμα κι αν φυσικά δεν είναι πιο γρήγορο.

37. BGP πολιτική δρομολόγησης

Στο BGP η πολιτική είναι πολύ σημαντική.

Παράδειγμα:

- A, B, C είναι provider networks.
- X, W, Y είναι customer networks.
- X είναι dual-homed, δηλαδή συνδεδεμένο σε δύο providers.

Αν το X δεν θέλει να μεταφέρει traffic από B προς C, τότε:

Auto ▾



δεν διαφημίζει στο B διαδρομή προς C.

Provider policy

Ένας provider θέλει συνήθως να δρομολογεί traffic:

- προς/από πελάτες του

Δεν θέλει να μεταφέρει traffic ανάμεσα σε δύο μη πελάτες χωρίς όφελος.

Παράδειγμα:

Αν το B μάθει route προς W μέσω A, δεν πρέπει απαραίτητα να τη διαφημίσει στο C, γιατί θα κουβαλάει traffic του C προς W χωρίς να πληρώνεται.

38. Γιατί διαφορετικό intra-AS και inter-AS routing;

Πολιτική

Στο inter-AS routing οι πολιτικές είναι κρίσιμες.

Κάθε AS θέλει να ελέγχει:

- ποιος περνά από το δίκτυό του
 - από πού βγαίνει η κίνηση
 - ποιες διαδρομές διαφημίζει
-

Κλίμακα

Η ιεραρχική δρομολόγηση μειώνει:

- μέγεθος πινάκων
 - routing updates
 - πολυπλοκότητα
-

Απόδοση

Στο intra-AS routing μπορούμε να κοιτάμε κυρίως απόδοση.

Στο inter-AS routing η πολιτική μπορεί να είναι πιο σημαντική από την απόδοση.

39. SDN

SDN = Software Defined Networking.

Ιδέα:

Auto ▾



Διαχωρισμός control plane και data plane.

Οι switches είναι απλοί και γρήγοροι.

Ο controller παίρνει τις αποφάσεις.

40. Γιατί SDN;

Το SDN βοηθά γιατί προσφέρει:

- ευκολότερη διαχείριση
- λιγότερα λάθη ρύθμισης
- ευελιξία στις ροές
- γενικευμένη προώθηση
- προγραμματισμό δικτύου
- ανοιχτές διεπαφές
- όχι κλειστές ιδιοταγείς λύσεις

41. Παραδοσιακή δρομολόγηση vs SDN

Στην παραδοσιακή δρομολόγηση είναι δύσκολο να πεις:

Auto ▾



Η ροή $u \rightarrow z$ να πάει από $u \rightarrow v \rightarrow z$
και η ροή $x \rightarrow z$ να πάει από $x \rightarrow w \rightarrow y \rightarrow z$.

Ή:

Auto ▾



Χώρισε την κίνηση $u \rightarrow z$ σε δύο paths.

Ή:

Auto ▾



Δρομολόγησε διαφορετικά κόκκινη και πορτοκαλί ροή.

Με SDN γίνεται πιο εύκολο γιατί οι κανόνες μπορούν να βασίζονται σε ροές και όχι μόνο destination IP.

42. Βασικές ιδέες SDN

Το SDN έχει:

16863. Γενικευμένη προώθηση με βάση ροή.
16864. Διαχωρισμό control και data plane.
16865. Λειτουργίες ελέγχου έξω από τους switches.

16866. Προγραμματιζόμενο δίκτυο.

43. SDN data plane switches

Οι SDN switches:

- είναι γρήγορα hardware κουτιά
 - εκτελούν κανόνες flow table
 - δεν υπολογίζουν μόνοι τους routing
 - παίρνουν κανόνες από controller
 - ελέγχονται μέσω API όπως OpenFlow
-

44. SDN controller

Ο controller:

- κρατά κατάσταση δικτύου
 - επικοινωνεί με switches μέσω southbound API
 - επικοινωνεί με εφαρμογές μέσω northbound API
 - υπολογίζει και εγκαθιστά flow tables
 - υλοποιείται ως κατακευκτικό σύστημα για κλιμάκωση και ανθεκτικότητα
-

Northbound API

Συνδέει:

Auto ▾



network-control applications → controller

Παραδείγματα εφαρμογών:

- routing
 - access control
 - load balancing
-

Southbound API

Συνδέει:

Auto ▾



controller → switches

Παράδειγμα:

Auto ▾



OpenFlow

45. SDN control applications

Οι control applications είναι το «μυαλό».

Υλοποιούν λειτουργίες όπως:

- routing
- access control
- load balancing

Χρησιμοποιούν τις υπηρεσίες του controller.

Μπορούν να αναπτυχθούν από τρίτους, όχι μόνο από vendors hardware.

46. Συστατικά SDN controller

Ένας SDN controller έχει:

18103. Επίπεδο διεπαφών με εφαρμογές.

18104. Διαχείριση κατάστασης δικτύου.

18105. Επίπεδο επικοινωνίας με συσκευές.

Κατάσταση δικτύου

Περιλαμβάνει:

- γράφο δικτύου
 - link-state information
 - statistics
 - host information
 - switch information
 - flow tables
-

Επικοινωνία

Μπορεί να χρησιμοποιεί:

- OpenFlow

- SNMP
 - άλλα πρωτόκολλα
-

47. OpenFlow protocol

Το OpenFlow λειτουργεί ανάμεσα σε:

Auto ▾



controller ↔ switch

Χρησιμοποιεί TCP:

Auto ▾



port 6653

Μπορεί να έχει προαιρετική κρυπτογράφηση.

48. Τρεις κλάσεις OpenFlow μηνυμάτων

18672. Controller-to-switch

18673. Asynchronous, δηλαδή switch-to-controller

18674. Symmetric

49. OpenFlow controller-to-switch messages

Ο controller μπορεί να στείλει:

Read-state

Ζητά στατιστικά και counters από:

- flow table
- ports

Configuration

Ρωτά ή θέτει παραμέτρους.

Modify-state

Προσθέτει, διαγράφει ή τροποποιεί entries στο flow table.

Packet-out

Στέλνει πακέτο σε συγκεκριμένη θύρα του switch.

50. OpenFlow switch-to-controller messages

Ο switch μπορεί να στείλει:

Packet-in

Όταν φτάσει πακέτο που δεν ταιριάζει σε flow table entry.

Flow-removed

Όταν αφαιρεθεί flow entry.

Port-status

Όταν αλλάξει η κατάσταση μιας θύρας.

51. Παράδειγμα SDN αλληλεπίδρασης

Αν χαλάσει ζεύξη S1-S2:

- 19415. Ο S1 ενημερώνει τον controller.
 - 19416. Ο controller ενημερώνει τη link-state βάση.
 - 19417. Η routing application ειδοποιείται.
 - 19418. Η routing application τρέχει Dijkstra.
 - 19419. Υπολογίζει νέες διαδρομές.
 - 19420. Ο flow-table manager αποφασίζει ποιοι πίνακες αλλάζουν.
 - 19421. Ο controller χρησιμοποιεί OpenFlow για να εγκαταστήσει νέους κανόνες.
-

52. ICMP

ICMP = Internet Control Message Protocol.

Χρησιμοποιείται από hosts και routers για ανταλλαγή πληροφορίας επιπέδου δικτύου.

Χρήσεις ICMP

Το ICMP χρησιμοποιείται για:

- αναφορά σφαλμάτων
- host unreachable

- network unreachable
- protocol unreachable
- port unreachable
- echo request/reply
- ping
- TTL expired
- bad IP header

ICMP πάνω από IP

Το ICMP ανήκει στο επίπεδο δικτύου, αλλά τα ICMP μηνύματα μεταφέρονται μέσα σε IP datagrams.

Το ICMP message περιέχει:

- type
- code
- τα πρώτα 8 bytes του IP datagram που προκάλεσε το σφάλμα

53. Βασικοί τύποι ICMP

Type	Code	Περιγραφή
0	0	echo reply
3	0	destination network unreachable
3	1	destination host unreachable
3	2	destination protocol unreachable
3	3	destination port unreachable
3	6	destination network unknown
3	7	destination host unknown
4	0	source quench, δεν χρησιμοποιείται

8	0	echo request
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

54. Ping

Το ping χρησιμοποιεί ICMP:

Auto ▾



```
echo request
echo reply
```

Ο host στέλνει ICMP echo request.

Ο άλλος απαντά με ICMP echo reply.

Έτσι μετρίεται αν ο host είναι προσβάσιμος και περίπου το RTT.

55. Traceroute

Το traceroute χρησιμοποιεί UDP και ICMP.

Η πηγή στέλνει σειρά από UDP segments με αυξανόμενο TTL.

Auto ▾



```
1o datagram: TTL = 1
2o datagram: TTL = 2
3o datagram: TTL = 3
...
```

Χρησιμοποιεί ασυνήθιστο port.

Τι γίνεται στον n-οστό router

Όταν το n-οστό datagram φτάσει στον n-οστό router:

21392. Το TTL γίνεται 0.

21393. Ο router απορρίπτει το datagram.

21394. Στέλνει ICMP μήνυμα:

Auto ▾



Type 11, Code 0
TTL expired

4. Η πηγή μετρά RTT.

Το traceroute στέλνει συνήθως:

Auto ▾



3 probes ανά TTL

Πότε σταματά το traceroute;

Όταν το UDP segment φτάσει στον τελικό προορισμό.

Επειδή έχει ασυνήθιστο port, ο προορισμός απαντά:

Auto ▾



ICMP Type 3, Code 3
port unreachable

Τότε το traceroute σταματά.

56. Διαχείριση δικτύου

Στην ενότητα αναφέρονται:

- SNMP
- NETCONF/YANG

Χρησιμοποιούνται για διαχείριση δικτύων.

SNMP

SNMP = Simple Network Management Protocol.

Χρησιμοποιείται για:

- παρακολούθηση συσκευών
 - συλλογή στατιστικών
 - διαχείριση routers/switches
 - λήψη πληροφοριών κατάστασης
-

NETCONF/YANG

Το NETCONF/YANG είναι πιο σύγχρονη προσέγγιση για διαχείριση και διαμόρφωση δικτύων.

- NETCONF: πρωτόκολλο διαχείρισης
 - YANG: γλώσσα μοντελοποίησης δεδομένων δικτύου
-

57. Τελική σύνοψη κεφαλαίου

Η Ενότητα 5 καλύπτει:

- 22378. Επίπεδο ελέγχου.
- 22379. Forwarding vs routing.
- 22380. Έλεγχος ανά router.
- 22381. SDN κεντριοποιημένο control plane.
- 22382. Routing protocols.
- 22383. Graph model.
- 22384. Link State routing.
- 22385. Dijkstra.
- 22386. Distance Vector routing.
- 22387. Bellman-Ford.
- 22388. Count-to-infinity.
- 22389. Poisoned reverse.
- 22390. Σύγκριση LS και DV.
- 22391. Ιεραρχική δρομολόγηση.
- 22392. Autonomous Systems.
- 22393. Intra-AS routing.
- 22394. RIP.
- 22395. OSPF.
- 22396. Hierarchical OSPF.
- 22397. Inter-AS routing.
- 22398. BGP.
- 22399. eBGP και iBGP.
- 22400. BGP messages.
- 22401. AS-PATH.
- 22402. NEXT-HOP.
- 22403. Local Preference.

22404. Hot potato routing.

22405. BGP policy.

22406. SDN.

22407. OpenFlow.

22408. ICMP.

22409. Ping.

22410. Traceroute.

22411. SNMP.

22412. NETCONF/YANG.

58. SOS για εξέταση

Auto ▾



Forwarding = τοπική προώθηση πακέτου μέσα στον router.

Routing = υπολογισμός διαδρομής από πηγή σε προορισμό.

Auto (Java) ▾



Link State = όλοι ξέρουν όλη την τοπολογία, Dijkstra.

Distance Vector = ανταλλαγή με γείτονες, Bellman-Ford.

Auto ▾



Dijkstra update:

$$D(v) = \min(D(v), D(w) + c(w, v))$$

Auto ▾



Bellman-Ford:

$$D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$$

Auto ▾



Good news travels fast.
Bad news travels slowly.

Auto (Visual Basic .NET) ▾



Count-to-infinity = πρόβλημα DV όταν αυξάνεται κόστος ή χαλάει ζεύξη.

Auto (Java) ▾



Poisoned reverse = λέω στον γείτονα ∞ για διαδρομή που χρησιμοποιώ μέσω αυτού.

Auto ▾



RIP = DV, hop count, max 15 hops, UDP port 520.

Auto ▾



OSPF = Link State, Dijkstra, flooding, IP protocol 89.

Auto ▾



BGP = inter-AS, path vector, TCP port 179.

Auto ▾



eBGP = μεταξύ AS.
iBGP = μέσα στο ίδιο AS.

Auto ▾



BGP route = prefix + attributes.

Auto (Visual Basic .NET) ▾



AS-PATH = λίστα AS από όπου πέρασε η ανακοίνωση.

NEXT-HOP = router επόμενου άλματος.

Auto (Visual Basic .NET) ▾



BGP επιλογή:

local pref → shortest AS-PATH → closest NEXT-HOP → άλλα κριτήρια.

Auto (Java) ▾



Hot potato = βγάζω την κίνηση από το AS μου όσο πιο γρήγορα γίνεται.

Auto ▾



SDN = διαχωρισμός data plane και control plane.

Auto ▾



OpenFlow = controller ↔ switch, TCP port 6653.

Auto ▾



ICMP = μηνύματα ελέγχου/σφαλμάτων, ping, traceroute.

Auto (Visual Basic .NET) ▾



Traceroute:

TTL 1,2,3 ...

routers απαντούν TTL expired.

destination απαντά port unreachable.

59. Οι βασικοί τύποι

Dijkstra

Auto ▾



$$D(v) = \min(D(v), D(w) + c(w, v))$$

Bellman-Ford

Auto ▾



$$D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$$

RIP infinity

Auto ▾



$$\infty = 16 \text{ hops}$$

BGP επιλογή

Auto (Visual Basic .NET) ▾



1. Local Preference
2. Shortest AS-PATH

3. Closest **NEXT-HOP**
4. Other criteria

60. Μικρό μνημονικό

Auto ▾



LS:

Ξέρω όλο τον χάρτη → τρέχω Dijkstra.

Auto ▾



DV:

Ρωτάω τους γείτονες → Bellman-Ford.

Auto ▾



OSPF:

μέσα στο AS, link state.

Auto ▾



BGP:

ανάμεσα σε AS, πολιτική, AS-PATH.

Auto ▾



SDN:

ο controller σκέφτεται, οι switches εκτελούν.

Auto ▾



ICMP:
το δίκτυο μου λέει τι πήγε στραβά.

Δίκτυα Επικοινωνιών I

Ενότητα 6: Επίπεδο Ζεύξης

1. Τι είναι το επίπεδο ζεύξης

Το **επίπεδο ζεύξης** έχει ευθύνη να μεταφέρει ένα datagram:

Auto ▾



από έναν κόμβο στον αμέσως γειτονικό κόμβο

πάνω από μία φυσική ζεύξη.

Βασική ορολογία

Όρος	Σημασία
Κόμβοι / nodes	υπολογιστές και δρομολογητές
Ζεύξεις / links	κανάλια που ενώνουν γειτονικούς κόμβους
Frame / πλαίσιο	πακέτο επιπέδου ζεύξης
Datagram	πακέτο επιπέδου δικτύου που μπαίνει μέσα στο frame

Είδη ζεύξεων

Οι ζεύξεις μπορεί να είναι:

- ενσύρματες
- ασύρματες
- LANs

2. Frame

Το επίπεδο ζεύξης ενθυλακώνει το IP datagram μέσα σε frame.

Auto ▾

IP datagram → Link-layer frame

Το ίδιο IP datagram μπορεί να μεταφερθεί από διαφορετικά πρωτόκολλα ζεύξης σε διαφορετικά σημεία της διαδρομής.

Παράδειγμα:

Auto ▾

Ethernet → Frame Relay → 802.11 WiFi

3. Αναλογία ταξιδιού

Ταξίδι από Πρίνστον στη Λωζάνη:

Δίκτυο	Αναλογία
Datagram	τουρίστας
Link	τμήμα ταξιδιού
Link protocol	μέσο μεταφοράς
Routing algorithm	ταξιδιωτικός πράκτορας

Παράδειγμα:

Auto ▾

Ταξί → Αεροπλάνο → Τρένο

Όπως αλλάζει μέσο μεταφοράς, έτσι ένα datagram μπορεί να αλλάζει πρωτόκολλο ζεύξης σε κάθε link.

4. Υπηρεσίες επιπέδου ζεύξης

Το επίπεδο ζεύξης μπορεί να προσφέρει:

- 1354. Πλαισίωση.
 - 1355. Πρόσβαση στη ζεύξη.
 - 1356. MAC addressing.
 - 1357. Αξιόπιστη παράδοση μεταξύ γειτονικών κόμβων.
 - 1358. Έλεγχο ροής.
 - 1359. Ανίχνευση σφαλμάτων.
 - 1360. Διόρθωση σφαλμάτων.
 - 1361. Half-duplex ή full-duplex λειτουργία.
-

5. Πλαισίωση και πρόσβαση στη ζεύξη

Πλαισίωση

Το επίπεδο ζεύξης:

Auto ▾



ενθυλακώνει datagram σε frame

και προσθέτει:

- header
 - trailer
-

Πρόσβαση στη ζεύξη

Αν το μέσο είναι κοινόχρηστο, πρέπει να αποφασιστεί:

Auto ▾



ποιος κόμβος θα μεταδώσει και πότε

Αυτό γίνεται με πρωτόκολλα MAC.

6. MAC διευθύνσεις

Οι MAC διευθύνσεις:

- μπαίνουν στην κεφαλίδα του frame
- αναγνωρίζουν πηγή και προορισμό στο τοπικό δίκτυο
- είναι διαφορετικές από τις IP διευθύνσεις

Auto ▾



IP address = επίπεδο δικτύου

MAC address = επίπεδο ζεύξης

7. Αξιόπιστη παράδοση στο επίπεδο ζεύξης

Σε κάποιες ζεύξεις μπορεί να υπάρχει reliable delivery ανάμεσα σε γειτονικούς κόμβους.

Χρησιμοποιείται κυρίως σε:

- ασύρματες ζεύξεις
- ζεύξεις με υψηλό bit error rate

Σπάνια χρησιμοποιείται σε:

- οπτικές ίνες
- καλής ποιότητας ενσύρματες ζεύξεις

Γιατί;

Επειδή εκεί τα σφάλματα είναι λίγα.

Γιατί αξιοπιστία και στο link layer και end-to-end;

Επειδή:

- το link layer διορθώνει τοπικά σφάλματα γρήγορα
- το TCP διορθώνει σφάλματα από άκρο σε άκρο
- σε ασύρματες ζεύξεις είναι καλύτερο να διορθώνεις τοπικά αντί να περιμένεις end-to-end retransmission

8. Έλεγχος ροής

Το link layer μπορεί να ρυθμίζει τον ρυθμό μεταξύ δύο γειτονικών κόμβων.

Δηλαδή:

Auto ▾



ο αποστολέας να μη στέλνει πιο γρήγορα από όσο μπορεί να λάβει ο γείτονας

9. Ανίχνευση και διόρθωση σφαλμάτων

Ανίχνευση σφαλμάτων

Ο δέκτης καταλαβαίνει ότι υπάρχουν σφάλματα λόγω:

- θορύβου
- εξασθένησης σήματος
- παρεμβολών

Αν εντοπιστεί λάθος:

- απορρίπτει το frame
 - ή ζητά αναμετάδοση
-

Διόρθωση σφαλμάτων

Ο δέκτης μπορεί να διορθώσει κάποια bits χωρίς αναμετάδοση.

10. Half-duplex και full-duplex

Half-duplex

Και οι δύο πλευρές μπορούν να μεταδώσουν, αλλά όχι ταυτόχρονα.

Auto ▾



ή μιλάω ή ακούω

Full-duplex

Και οι δύο πλευρές μπορούν να μεταδίδουν ταυτόχρονα.

Auto ▾



μιλάω και ακούω ταυτόχρονα

11. Πού υλοποιείται το επίπεδο ζεύξης;

Το link layer υλοποιείται:

- σε κάθε host
- σε network adapter
- σε NIC
- σε Ethernet/WiFi chipset

Περιλαμβάνει:

- hardware
- software
- firmware

Η NIC συνδέεται στο host bus, π.χ. PCI.

12. Προσαρμογείς που επικοινωνούν

Στην αποστολή

Ο adapter:

3838. Παίρνει datagram από network layer.

3839. Το ενθυλακώνει σε frame.

3840. Προσθέτει error detection bits.

3841. Προσθέτει πληροφορίες flow control ή reliable transfer, αν υπάρχουν.

3842. Το στέλνει στο φυσικό μέσο.

Στη λήψη

Ο adapter:

4074. Λαμβάνει frame.

4075. Ελέγχει για σφάλματα.

4076. Κάνει link-layer λειτουργίες.

4077. Βγάζει το datagram.

4078. Το παραδίδει στο network layer.

13. Είδη ζεύξεων

Υπάρχουν δύο βασικά είδη:

Point-to-point

Σύνδεση ανάμεσα σε δύο κόμβους.

Παραδείγματα:

- PPP για dial-up
- Ethernet switch με υπολογιστή

Broadcast

Κοινόχρηστο μέσο.

Παραδείγματα:

- παραδοσιακό Ethernet
- upstream HFC
- 802.11 WiFi
- δορυφορικό κανάλι

14. Πολλαπλή πρόσβαση

Σε broadcast κανάλι, πολλοί κόμβοι μοιράζονται το ίδιο μέσο.

Αν δύο ή περισσότεροι μεταδώσουν ταυτόχρονα:

Auto ▾



collision / σύγκρουση

Τότε τα σήματα παρεμβάλλονται.

15. Πρωτόκολλο πολλαπλής πρόσβασης

Είναι κατανεμημένος αλγόριθμος που αποφασίζει:

Auto ▾



πότε μπορεί ένας κόμβος να μεταδώσει

Σημαντικό:

Δεν υπάρχει ξεχωριστό κανάλι συντονισμού.

Η επικοινωνία για τον συντονισμό γίνεται πάνω στο ίδιο κοινόχρηστο κανάλι.

16. Ιδανικό πρωτόκολλο πολλαπλής πρόσβασης

Για broadcast κανάλι ρυθμού R θέλουμε:

- 5074. Αν μόνο ένας κόμβος θέλει να στείλει, να στέλνει με ρυθμό R.
 - 5075. Αν M κόμβοι θέλουν να στείλουν, ο καθένας να παίρνει μέσο ρυθμό R/M.
 - 5076. Να είναι πλήρως αποκεντρωμένο.
 - 5077. Να μην έχει ειδικό master κόμβο.
 - 5078. Να μη χρειάζεται συγχρονισμό ρολογιών.
 - 5079. Να είναι απλό.
-

17. Κατηγορίες MAC πρωτοκόλλων

Υπάρχουν τρεις βασικές κατηγορίες:

- 5416. Διαμέριση καναλιού.
 - 5417. Τυχαία πρόσβαση.
 - 5418. Εκ περιτροπής λειτουργία.
-

18. Διαμέριση καναλιού

Το κανάλι χωρίζεται σε κομμάτια:

- χρόνου
- συχνότητας
- κώδικα

Κάθε κόμβος παίρνει αποκλειστική χρήση ενός κομματιού.

Παραδείγματα:

- TDMA
 - FDMA
-

19. TDMA

TDMA = Time Division Multiple Access.

Το κανάλι χωρίζεται σε χρονοθυρίδες.

Χαρακτηριστικά:

- πρόσβαση σε γύρους
- κάθε σταθμός παίρνει σταθερή θυρίδα
- αν ο σταθμός δεν έχει να στείλει, η θυρίδα μένει άδεια

Παράδειγμα:

6 σταθμοί, στέλνουν μόνο οι 1, 3, 4:

Auto ▾



1 | idle | 3 | 4 | idle | idle

Μειονέκτημα:

Auto ▾



σπατάλη όταν κάποιοι κόμβοι δεν στέλνουν

20. FDMA

FDMA = Frequency Division Multiple Access.

Το φάσμα χωρίζεται σε ζώνες συχνοτήτων.

Κάθε σταθμός παίρνει δική του ζώνη.

Αν δεν έχει να στείλει:

Auto ▾



η ζώνη του μένει ανενεργή

Μειονέκτημα:

Auto ▾



σπατάλη συχνότητας

21. Τυχαία πρόσβαση

Στα πρωτόκολλα τυχαίας πρόσβασης:

- το κανάλι δεν χωρίζεται
- ο κόμβος στέλνει όταν έχει πλαίσιο
- μπορούν να υπάρξουν collisions
- χρειάζεται ανάνηψη από collisions

Παραδείγματα:

- Slotted ALOHA
 - ALOHA
 - CSMA
 - CSMA/CD
 - CSMA/CA
-

22. Slotted ALOHA

Υποθέσεις

- όλα τα frames έχουν ίδιο μέγεθος
 - ο χρόνος χωρίζεται σε ίσες θυρίδες
 - μία θυρίδα = χρόνος μετάδοσης ενός frame
 - οι κόμβοι μεταδίδουν μόνο στην αρχή θυρίδας
 - οι κόμβοι είναι συγχρονισμένοι
 - αν δύο μεταδώσουν στην ίδια θυρίδα, υπάρχει collision
-

Λειτουργία

Όταν ένας κόμβος έχει frame:

6916. Το στέλνει στην επόμενη θυρίδα.

6917. Αν δεν υπάρξει collision, επιτυχία.

6918. Αν υπάρξει collision, αναμεταδίδει σε επόμενες θυρίδες με πιθανότητα p μέχρι επιτυχία.

Πλεονεκτήματα Slotted ALOHA

- αν ένας κόμβος είναι μόνος, χρησιμοποιεί πλήρως το κανάλι
 - απλό
 - αρκετά αποκεντρωμένο
 - χρειάζεται μόνο συγχρονισμό θυρίδων
-

Μειονεκτήματα Slotted ALOHA

- collisions
 - χαμένες θυρίδες
 - idle slots
 - χρειάζεται συγχρονισμός
 - μπορεί collision να ανιχνεύεται νωρίτερα αλλά χάνεται όλη η θυρίδα
-

23. Αποδοτικότητα Slotted ALOHA

Με N κόμβους και πιθανότητα μετάδοσης p :

Πιθανότητα συγκεκριμένος κόμβος να πετύχει:

Auto ▾



$$p(1-p)^{(N-1)}$$

Πιθανότητα να πετύχει κάποιος κόμβος:

Auto ▾



$$Np(1-p)^{(N-1)}$$

Για πολλά N , η μέγιστη αποδοτικότητα είναι:

Auto ▾



$$1/e \approx 0.37$$

Άρα στην καλύτερη περίπτωση:

Auto ▾



το κανάλι χρησιμοποιείται ωφέλιμα 37% του χρόνου

24. Απλό ALOHA

Στο απλό ALOHA δεν υπάρχουν θυρίδες.

Ο κόμβος:

Auto ▾



στέλνει αμέσως όταν έχει frame

Δεν χρειάζεται συγχρονισμός.

Όμως η πιθανότητα σύγκρουσης είναι μεγαλύτερη.

Ένα frame που ξεκινά στο t_0 μπορεί να συγκρουστεί με frames που ξεκινούν στο:

Auto ▾

 $[t_0 - 1, t_0 + 1]$

25. Αποδοτικότητα απλού ALOHA

Πιθανότητα επιτυχίας συγκεκριμένου κόμβου:

Auto ▾

 $p(1-p)^{(2(N-1))}$

Για πολλά N , μέγιστη αποδοτικότητα:

Auto ▾

 $1/(2e) \approx 0.18$

Άρα είναι χειρότερο από το Slotted ALOHA.

26. CSMA

CSMA = Carrier Sense Multiple Access.

Ιδέα:

Auto ▾



Άκου πριν μεταδώσεις.

Αν το κανάλι είναι αδρανές:

Auto ▾



μετάδωσε

Αν είναι απασχολημένο:

Auto ▾



περίμενε

Αναλογία:

Auto ▾



Μη διακόπτεις όταν μιλάει άλλος.

27. Συγκρούσεις στο CSMA

Παρότι οι κόμβοι ακούν πριν στείλουν, συγκρούσεις μπορούν να γίνουν λόγω καθυστέρησης διάδοσης.

Δηλαδή:

- ο A ξεκινά να στέλνει
- το σήμα δεν έχει φτάσει ακόμα στον B
- ο B νομίζει ότι το κανάλι είναι ελεύθερο
- στέλνει και αυτός
- γίνεται collision

Όσο μεγαλύτερη απόσταση/καθυστέρηση διάδοσης, τόσο μεγαλύτερη πιθανότητα collision.

28. CSMA/CD

CSMA/CD = CSMA with Collision Detection.

Χρησιμοποιείται στο κλασικό Ethernet.

Ιδέα:

9056. Άκου πριν μεταδώσεις.

9057. Αν γίνει collision, εντόπισέ το γρήγορα.

9058. Σταμάτα τη μετάδοση.

9059. Στείλε jam signal.

9060. Περίμενε τυχαίο χρόνο.

9061. Ξαναπροσπάθησε.

Collision detection

Εύκολη σε ενσύρματα LANs, γιατί μπορείς να μετρήσεις το σήμα.

Δύσκολη σε ασύρματα LANs, γιατί η δική σου μετάδοση καλύπτει το λαμβανόμενο σήμα.

29. Ethernet CSMA/CD αλγόριθμος

9432. Η NIC παίρνει datagram από network layer και φτιάχνει frame.

9433. Αν το κανάλι είναι idle, αρχίζει μετάδοση.

9434. Αν είναι busy, περιμένει να γίνει idle.

9435. Αν τελειώσει χωρίς collision, επιτυχία.

9436. Αν εντοπίσει collision, σταματά και στέλνει jam signal.

9437. Μετά κάνει binary exponential backoff.

Binary exponential backoff

Μετά την m-οστή σύγκρουση:

Auto ▾



K επιλέγεται τυχαία από $\{0, 1, 2, \dots, 2^m - 1\}$

Η NIC περιμένει:

Auto ▾



$K \times 512 \text{ bit times}$

και ξαναδοκιμάζει.

Όσο περισσότερες συγκρούσεις, τόσο μεγαλώνει το πιθανό διάστημα αναμονής.

30. Ρυθμαπόδοση CSMA/CD

Ορίζουμε:

Auto ▾



t_{prop} = μέγιστη καθυστέρηση διάδοσης μεταξύ 2 κόμβων

t_{trans} = χρόνος μετάδοσης μέγιστου frame

Ρυθμαπόδοση:

Auto ▾



$\text{Efficiency} = 1 / (1 + 5 t_{prop} / t_{trans})$

Η απόδοση πάει στο 1 όταν:

Auto ▾

 $t_{prop} \rightarrow 0$

ή όταν:

Auto ▾

 $t_{trans} \rightarrow \infty$

Είναι καλύτερο από ALOHA.

31. Πρωτόκολλα εκ περιτροπής

Στόχος:

Auto ▾



να πάρουμε τα καλά της διαμέρισης και της τυχαίας πρόσβασης

Η διαμέριση είναι καλή σε υψηλό φορτίο αλλά κακή σε χαμηλό.

Η τυχαία πρόσβαση είναι καλή σε χαμηλό φορτίο αλλά έχει collisions σε υψηλό.

32. Polling

Υπάρχει master κόμβος.

Ο master προσκαλεί κάθε slave να μεταδώσει με τη σειρά.

Μειονεκτήματα:

- polling overhead
- καθυστέρηση
- single point of failure στον master

33. Token passing

Υπάρχει token.

Μόνο ο κόμβος που έχει το token μπορεί να μεταδώσει.

Το token περνά σειριακά από κόμβο σε κόμβο.

Μειονεκτήματα:

- token overhead
 - καθυστέρηση
 - αν χαθεί το token υπάρχει πρόβλημα
-

34. MAC και ARP

IP διεύθυνση

Η IP διεύθυνση:

- είναι 32-bit στο IPv4
 - ανήκει στο network layer
 - χρησιμοποιείται για δρομολόγηση επιπέδου 3
-

MAC διεύθυνση

Η MAC διεύθυνση:

- είναι 48-bit
- ανήκει στο link layer
- αποθηκεύεται στη ROM της NIC
- χρησιμοποιείται τοπικά μέσα στο LAN
- λέγεται και LAN address, physical address, Ethernet address

Παράδειγμα:

Auto ▾



1A-2F-BB-76-09-AD

Είναι δεκαεξαδική.

Κάθε hex ψηφίο = 4 bits.

35. Broadcast MAC address

Η broadcast MAC είναι:

Auto ▾



FF-FF-FF-FF-FF-FF

Σημαίνει:

Auto ▾



όλοι οι κόμβοι στο LAN

36. Ποιος δίνει MAC addresses;

Το IEEE διαχειρίζεται τον χώρο MAC διευθύνσεων.

Οι κατασκευαστές αγοράζουν τμήματα MAC addresses ώστε να διασφαλίζεται μοναδικότητα.

37. MAC vs IP

MAC	IP
φυσική/LAN διεύθυνση	λογική/network διεύθυνση
48 bits	32 bits στο IPv4
επίπεδη	ιεραρχική
φορητή με την κάρτα	εξαρτάται από το δίκτυο
χρησιμοποιείται τοπικά	χρησιμοποιείται για routing
σαν ΑΜΚΑ	σαν ταχυδρομική διεύθυνση

38. ARP

ARP = Address Resolution Protocol.

Σκοπός:

Auto ▾



Βρίσκω MAC διεύθυνση όταν ξέρω IP διεύθυνση.

ARP table

Κάθε IP κόμβος στο LAN έχει ARP table.

Περιέχει entries:

Auto ▾



<IP address, MAC address, TTL>

TTL συνήθως περίπου:

Auto ▾



20 λεπτά

39. ARP στο ίδιο LAN

Ο Α θέλει να στείλει στον Β.

Ξέρει IP του Β, αλλά όχι MAC του Β.

Βήματα:

12454. Ο Α στέλνει ARP query broadcast.

12455. Destination MAC:

Auto ▾



FF-FF-FF-FF-FF-FF

3. Όλοι στο LAN λαμβάνουν το query.
4. Ο Β βλέπει ότι το IP είναι δικό του.
5. Ο Β απαντά με ARP reply unicast στον Α.
6. Ο Α αποθηκεύει IP→MAC στον ARP table.

Το ARP είναι:

Auto ▾



plug-and-play

Δεν χρειάζεται διαχειριστής.

40. Soft state

Οι ARP εγγραφές είναι soft state.

Δηλαδή:

Auto ▾



αν δεν ανανεωθούν, λήγουν και διαγράφονται

41. Αποστολή σε άλλο LAN

Αν ο A θέλει να στείλει στον B που είναι σε άλλο LAN:

Δεν στέλνει frame απευθείας στη MAC του B.

Στέλνει frame στη MAC του router πρώτου άλματος.

Σημαντικό

Το IP datagram έχει:

Auto ▾



IP src = A

IP dest = B

Σε όλη τη διαδρομή.

Αλλά το Ethernet frame αλλάζει σε κάθε ζεύξη.

Πρώτο LAN

Ο A φτιάχνει frame:

Auto ▾



MAC src = MAC του A

MAC dest = MAC του router R

To frame περιέχει datagram:

Auto ▾



```
IP src = A
IP dest = B
```

Στον router

Ο router:

- 13423. Λαμβάνει το frame.
- 13424. Βγάζει το IP datagram.
- 13425. Κοιτάει destination IP.
- 13426. Αποφασίζει επόμενη έξοδο.
- 13427. Φτιάχνει νέο frame.

Δεύτερο LAN

Ο router φτιάχνει frame:

Auto (Java) ▾



```
MAC src = MAC του router στη δεύτερη διεπαφή
MAC dest = MAC του B
```

Το IP datagram παραμένει:

Auto ▾



```
IP src = A
IP dest = B
```

42. Ethernet

Το Ethernet είναι η κυρίαρχη τεχνολογία ενσύρματων LAN.

Πλεονεκτήματα:

- φθινό
- απλό
- ευρέως χρησιμοποιούμενο
- πιο απλό από token LANs και ATM
- υποστήριξε πολλές ταχύτητες

Ταχύτητες:

Auto ▾



10 Mbps έως 10 Gbps

43. Ethernet φυσική τοπολογία

Bus topology

Παλαιότερα δημοφιλής.

Όλοι οι κόμβοι στο ίδιο collision domain.

Άρα μπορούσαν να συγκρουστούν μεταξύ τους.

Star topology

Σήμερα κυριαρχεί.

Υπάρχει switch στο κέντρο.

Κάθε host έχει ξεχωριστή σύνδεση με switch.

Σε full-duplex συνδέσεις δεν υπάρχουν collisions.

44. Ethernet frame

Ο αποστολέας ενθυλακώνει IP datagram σε Ethernet frame.

Πεδία Ethernet frame:

- preamble
- destination MAC
- source MAC
- type
- data
- CRC

Preamble

To preamble έχει:

Auto ▾



7 bytes: 10101010

1 byte: 10101011

Χρησιμοποιείται για συγχρονισμό ρολογιού πομπού και δέκτη.

MAC addresses

Έχει:

Auto ▾



destination MAC = 6 bytes

source MAC = 6 bytes

Αν η destination MAC είναι:

- η MAC του adapter
- ή broadcast MAC

τότε ο adapter παραδίδει το payload στο network layer.

Αλλιώς απορρίπτει το frame.

Type

Το Type δείχνει ποιο πρωτόκολλο ανώτερου επιπέδου υπάρχει στο payload.

Συνήθως:

Auto ▾



IP

Αλλά μπορεί να είναι και άλλα, π.χ.:

- Novell IPX
 - AppleTalk
-

CRC

CRC = Cyclic Redundancy Check.

Χρησιμοποιείται για ανίχνευση σφαλμάτων.

Αν εντοπιστεί λάθος:

Auto ▾



το frame απορρίπτεται

45. Ethernet υπηρεσία

Το Ethernet είναι:

Connectionless

Δεν γίνεται handshake μεταξύ adapters.

Unreliable

Δεν στέλνει ACKs ή NACKs.

Αν ένα frame χαθεί ή απορριφθεί, ανακτάται μόνο αν υπάρχει αξιοπιστία σε ανώτερο επίπεδο, π.χ. TCP.

Αν χρησιμοποιείται UDP, μπορεί απλά να χαθεί.

Ethernet MAC

Το Ethernet χρησιμοποιεί:

Auto ▾



unslotted CSMA/CD με binary backoff

στο κλασικό shared Ethernet.

46. Ethernet 802.3 πρότυπα

Υπάρχουν πολλά Ethernet standards.

Έχουν κοινά:

- MAC protocol
- frame format

Διαφέρουν σε:

- ταχύτητα
- φυσικό μέσο

Παραδείγματα ταχυτήτων:

- 2 Mbps
- 10 Mbps
- 100 Mbps
- 1 Gbps
- 10 Gbps

Μέσα:

- συνεστραμμένο ζεύγος χαλκού
- οπτική ίνα

Παραδείγματα:

- 100BASE-TX
 - 100BASE-FX
 - 100BASE-SX
 - 100BASE-BX
-

47. Ethernet switch

Ο Ethernet switch είναι συσκευή επιπέδου ζεύξης.

Κάνει:

- store and forward
 - εξετάζει MAC διευθύνσεις
 - προωθεί frames επιλεκτικά
 - χρησιμοποιεί switch table
 - είναι διαφανής για hosts
 - είναι plug-and-play
 - μαθαίνει μόνος του
-

48. Switches και ταυτόχρονες μεταδόσεις

Με switch:

- κάθε host έχει αποκλειστική σύνδεση
- κάθε link είναι ξεχωριστό collision domain
- σε full-duplex δεν υπάρχουν collisions
- μπορούν να γίνουν πολλές μεταδόσεις ταυτόχρονα

Παράδειγμα:

Auto ▾



A → A'

B → B'

μπορούν να συμβούν ταυτόχρονα.

49. Switch table

Κάθε switch έχει πίνακα:

Auto ▾



MAC address | interface | timestamp/TTL

Παράδειγμα:

Auto ▾



A → interface 1

A' → interface 4

50. Αυτοεκμάθηση switch

Όταν ο switch λάβει frame:

16761. Κοιτάζει τη source MAC.

16762. Μαθαίνει από ποια interface ήρθε.

16763. Βάζει στον πίνακα:

Auto ▾



source MAC → incoming interface

Έτσι μαθαίνει πού βρίσκονται οι hosts.

51. Forwarding / filtering σε switch

Όταν λάβει frame:

16997. Καταγράφει source MAC και incoming interface.

16998. Κοιτάει destination MAC στον switch table.

Αν βρει destination:

- Αν destination είναι στην ίδια interface από όπου ήρθε:

Auto ▾



drop

- Αλλιώς:

Auto ▾



forward στη σωστή interface

Αν δεν βρει destination:

Auto ▾



flood

δηλαδή στέλνει σε όλες τις interfaces εκτός από αυτή που ήρθε.

52. Flooding

Flooding γίνεται όταν ο switch δεν ξέρει πού είναι η destination MAC.

Τότε στέλνει το frame παντού εκτός από την είσοδο.

Μετά, όταν ο προορισμός απαντήσει, ο switch μαθαίνει και τη δική του θέση.

53. Διασύνδεση switches

Οι switches μπορούν να συνδέονται μεταξύ τους.

Η αυτοεκμάθηση λειτουργεί και σε πολλούς switches.

Αν ο C στείλει στον I:

- οι switches μαθαίνουν πού είναι ο C από τη source MAC
 - αν δεν ξέρουν τον I κάνουν flooding
 - όταν ο I απαντήσει, μαθαίνουν και τη θέση του I
-

54. Switches vs Routers

Switch	Router
επίπεδο ζεύξης	επίπεδο δικτύου
εξετάζει MAC header	εξετάζει IP header
προωθεί frames	προωθεί datagrams
μαθαίνει με self-learning	υπολογίζει routes
χρησιμοποιεί MAC addresses	χρησιμοποιεί IP addresses
flood όταν δεν ξέρει	routing table
plug-and-play	χρειάζεται routing config/protocol

Και οι δύο είναι:

Auto ▾



store-and-forward

55. VLANs

VLAN = Virtual Local Area Network.

Επιτρέπει να χωρίσουμε ένα φυσικό LAN σε πολλά λογικά LANs.

56. Γιατί θέλουμε VLANs;

Προβλήματα χωρίς VLAN:

- ένα μεγάλο broadcast domain
 - ARP/DHCP broadcasts πάνε παντού
 - λιγότερη ασφάλεια
 - λιγότερη ιδιωτικότητα
 - χαμηλότερη αποδοτικότητα
 - δυσκολία όταν χρήστης μετακινείται φυσικά αλλά θέλει να μείνει στο ίδιο λογικό δίκτυο
-

57. Port-based VLAN

Στο port-based VLAN, ομαδοποιούμε θύρες switch.

Παράδειγμα:

Auto ▾



```
ports 1-8 → Electrical Engineering VLAN  
ports 9-15 → Computer Science VLAN
```

Ο ίδιος φυσικός switch λειτουργεί σαν πολλοί εικονικοί switches.

58. Ιδιότητες VLAN

Απομόνωση κίνησης

Frames από ports ενός VLAN μπορούν να πάνε μόνο σε ports του ίδιου VLAN.

Dynamic membership

Οι θύρες μπορούν να ανατεθούν δυναμικά σε VLANs.

VLAN με MAC addresses

Μπορεί VLAN να οριστεί και με βάση MAC addresses, όχι μόνο ports.

Επικοινωνία μεταξύ VLANs

Για να επικοινωνήσουν διαφορετικά VLANs χρειάζεται router.

Δηλαδή:

Auto ▾



```
inter-VLAN communication = routing
```

Στην πράξη υπάρχουν switches που έχουν και routing δυνατότητες.

59. VLANs σε πολλούς switches

Αν VLANs εκτείνονται σε πολλούς φυσικούς switches, χρειάζονται trunk ports.

Trunk port

Trunk port μεταφέρει frames πολλών VLANs ανάμεσα σε switches.

Επειδή πρέπει να ξέρουμε σε ποιο VLAN ανήκει κάθε frame, χρησιμοποιείται tagging.

802.1Q

Το 802.1Q προσθέτει/αφαιρεί επιπλέον πεδία κεφαλίδας για VLAN ID.

Δηλαδή:

Auto ▾



Ethernet frame + VLAN tag

Το VLAN ID δείχνει σε ποιο VLAN ανήκει το frame.

60. MPLS

Στις σημειώσεις αναφέρεται ως:

Auto ▾



Εικονικές Ζεύξεις: MPLS

Το MPLS λειτουργεί σαν τεχνολογία που δημιουργεί εικονικές διαδρομές/ζεύξεις μέσα στο δίκτυο.

Βασική ιδέα:

Auto ▾



προώθηση με labels αντί μόνο με IP destination

61. Δικτύωση κέντρων δεδομένων

Αναφέρεται ως μέρος της ενότητας.

Τα data center networks είναι δίκτυα υψηλής ταχύτητας που συνδέουν:

- servers
- switches
- storage
- load balancers

- services

Στόχος:

- υψηλή απόδοση
 - χαμηλή καθυστέρηση
 - αξιοπιστία
 - κλιμάκωση
-

62. Μία ημέρα στη ζωή μιας web αίτησης

Σενάριο:

Φοιτητής συνδέει laptop στο δίκτυο του πανεπιστημίου και ζητά:

Auto ▾



`www.google.com`

Πρωτόκολλα που εμφανίζονται:

- DHCP
 - ARP
 - DNS
 - UDP
 - IP
 - Ethernet
 - TCP
 - HTTP
 - routing protocols
 - switches
 - routers
-

63. Βήμα 1: DHCP

Το laptop χρειάζεται:

- IP address
- first-hop router IP
- DNS server IP/name

Χρησιμοποιεί DHCP.

Το DHCP request ενθυλακώνεται:

Auto ▾



DHCP → UDP → IP → Ethernet

Στέλνεται ως Ethernet broadcast:

Auto ▾



FF-FF-FF-FF-FF-FF

Το λαμβάνει ο router που τρέχει DHCP server.

64. DHCP ACK

Ο DHCP server στέλνει DHCP ACK που περιέχει:

- IP address του client
- IP του first-hop router
- DNS server name
- DNS server IP

Μετά ο client ξέρει:

Auto ▾



τη δική του IP
τον gateway
τον DNS server

65. Βήμα 2: ARP πριν το DNS

Πριν στείλει DNS query, ο client πρέπει να στείλει frame στον first-hop router.

Ξέρει IP του router, αλλά χρειάζεται MAC του router.

Άρα κάνει ARP.

21459. Στέλνει ARP query broadcast.

21460. Ο router απαντά με ARP reply.

21461. Ο client μαθαίνει MAC του gateway.

66. Βήμα 3: DNS

Ο client θέλει την IP του:

Auto ▾

`www.google.com`

Στέλνει DNS query:

Auto ▾

`DNS → UDP → IP → Ethernet`

To frame πάει στον first-hop router.

To IP datagram δρομολογείται προς τον DNS server.

Ο DNS server απαντά με την IP του www.google.com.

67. Βήμα 4: TCP σύνδεση

Για HTTP, ο client χρειάζεται TCP σύνδεση με τον web server.

Γίνεται TCP three-way handshake:

21970. SYN

21971. SYN-ACK

21972. ACK

Μετά η TCP σύνδεση είναι έτοιμη.

68. Βήμα 5: HTTP αίτηση και απόκριση

Ο browser στέλνει HTTP request μέσα από TCP socket.

Auto ▾

`HTTP → TCP → IP → Ethernet`

To IP datagram δρομολογείται προς τον Google web server.

Ο server απαντά με HTTP response που περιέχει την ιστοσελίδα.

Η ιστοσελίδα εμφανίζεται στον browser.

69. Τελική σύνοψη επιπέδου ζεύξης

Η ενότητα καλύπτει:

- 22392. Τι είναι κόμβος, ζεύξη, frame.
- 22393. Υπηρεσίες link layer.
- 22394. Πλαισίωση.
- 22395. MAC addressing.
- 22396. Reliable delivery στο link.
- 22397. Flow control.
- 22398. Error detection/correction.
- 22399. Half/full duplex.
- 22400. NIC.
- 22401. Point-to-point και broadcast links.
- 22402. Multiple access protocols.
- 22403. TDMA.
- 22404. FDMA.
- 22405. Slotted ALOHA.
- 22406. ALOHA.
- 22407. CSMA.
- 22408. CSMA/CD.
- 22409. Binary exponential backoff.
- 22410. Polling.
- 22411. Token passing.
- 22412. MAC addresses.
- 22413. ARP.
- 22414. Ethernet.
- 22415. Ethernet frame.
- 22416. Ethernet service.
- 22417. Switches.
- 22418. Self-learning.
- 22419. Flooding.
- 22420. VLANs.
- 22421. 802.1Q trunking.
- 22422. MPLS.
- 22423. Data center networking.
- 22424. Web request full scenario.

70. SOS για εξέταση

Auto ▾



Link layer = μεταφορά datagram από κόμβο σε γειτονικό κόμβο.

Auto ▾



Frame = πακέτο επιπέδου ζεύξης.

Auto ▾



MAC address = τοπική διεύθυνση 48-bit.

Auto ▾



IP address = λογική ιεραρχική διεύθυνση επιπέδου δικτύου.

Auto ▾



ARP = βρίσκω MAC όταν ξέρω IP.

Auto ▾



Broadcast MAC = FF-FF-FF-FF-FF-FF.

Auto ▾



TDMA = μοιράζω χρόνο.

FDMA = μοιράζω συχνότητα.

Auto (Java) ▾



Slotted ALOHA max efficiency = $1/e$ = 0.37.

ALOHA max efficiency = $1/(2e)$ = 0.18.

Auto (PowerShell) ▾



CSMA = άκου πριν μεταδώσεις.

CSMA/CD = άκου, μετάδωσε, αν γίνει collision σταμάτα.

Auto ▾



Ethernet = connectionless και unreliable.

Auto ▾



Switch = layer 2, MAC addresses, self-learning.

Auto ▾



Router = layer 3, IP addresses, routing protocols.

Auto (Java) ▾



Unknown destination σε switch = flood.

Known destination = forward.

Same incoming interface = drop.

Auto ▾



VLAN = πολλά λογικά LANs πάνω σε ένα φυσικό LAN.

Auto ▾



Trunk port = μεταφέρει πολλά VLANs.

802.1Q = VLAN tag.

Auto ▾



Web request:

DHCP → ARP → DNS → TCP handshake → HTTP.

71. Βασικοί τύποι

Slotted ALOHA

Auto ▾



$P(\text{success specific node}) = p(1-p)^{(N-1)}$

Auto ▾



$P(\text{success any node}) = Np(1-p)^{(N-1)}$

Auto ▾



Max efficiency = $1/e \approx 0.37$

Pure ALOHA

Auto ▾



$P(\text{success specific node}) = p(1-p)^{(2(N-1))}$

Auto ▾


$$\text{Max efficiency} = 1/(2e) \approx 0.18$$

CSMA/CD efficiency

Auto ▾


$$\text{Efficiency} = 1 / (1 + 5t_{\text{prop}}/t_{\text{trans}})$$

Binary exponential backoff

Μετά από m collisions:

Auto ▾


$$K \in \{0, 1, 2, \dots, 2^m - 1\}$$

Αναμονή:

Auto ▾


$$K \times 512 \text{ bit times}$$

72. Μικρό μνημονικό

Auto (PowerShell) ▾



IP σε πάει μέχρι το σωστό δίκτυο.
MAC σε πάει στον σωστό γείτονα μέσα στο LAN.
ARP μεταφράζει IP σε MAC.
Switch μαθαίνει MAC.
Router κοιτάει IP.
VLAN χωρίζει το LAN λογικά.
Ethernet στέλνει frames χωρίς εγγυήσεις.

